

01 · 현장 데이터 수집: 먼저 흐름 잡기

보강 안내 1/4 · 기존 46쪽 교본은 뒤에 원형 보존

이 권의 핵심 질문

설비의 실제 상태가 어떻게 **시간·태그·값·품질**이 있는 메시지가 되는가?

한 값의 길: 교반기 전류 IT-102

① 가상 AR-100

교반기 전류가 바뀐다. 이 실습은 실제 PLC가 아닌 시뮬레이터다.



② Modbus TCP

계측 레지스터의 숫자를 읽는다. 장치 주소·자료형·단위를 맞춘다.



③ EdgeX (기본 경로)

device-modbus가 읽고 core-data가 다루며 MQTT export가 Event를 보낸다.



④ EMQX → Telegraf bridge

MQTT 메시지를 받은 뒤 Kafka 입력 형식으로 정규화한다.



⑤ Kafka raw

sensor.telemetry.raw에 원시 관측을 기록한다. 다음 권이 여기서 시작한다.

바로 보이는 별도 길

FUXA 서버는 시뮬레이터에 **Modbus로 직접** 연결해 현재 값을 표시하고 기동·정지 명령도 쓴다. FUXA 값이 보인다는 것만으로 Kafka·Flink 정상은 증명되지 않는다.

01 · 같은 설비, 서로 다른 두 입력

보강 안내 2/4 · 기본 경로와 lite 경로를 혼동하지 않기

비교	기본 EdgeX 경로	lite 경로
센서 수집	시뮬레이터 → Modbus → EdgeX	시뮬레이터가 MQTT로 직접 발행
MQTT 입력	EdgeX Event(JSON, readings 배열)	평면 JSON(이미 태그·값 포함)
Telegraf 설정	bridge-edgex.conf: Event를 파싱·정규화	bridge-lite.conf: 평면 값을 파싱
공통 출력	sensor.telemetry.raw	sensor.telemetry.raw
뒤쪽 흐름	Kafka 이후에는 Flink·저장·화면 경로가 같다	Kafka 이후에는 같은 경로가 이어진다

메시지가 바뀌는 자리

Modbus의 관점

주소에 저장된 숫자다. 태그명과 단위는 설정이 해석한다.

EdgeX의 관점

장치명·resourceName·origin·readings로 측정을 감싼 Event다.

Kafka의 관점

ts·site·device·tag·value·quality로 정규화한 관측이다.

교재용 예시: 실제 실행값 아님

```
{"ts": "측정 시각", "site": "AR-100", "device": "reactor-line-01", "tag": "IT-102", "value": 9.8, "quality": "GOOD"}
```

실제 bridge 설정의 공통 출력 필드는 ts·site·device·tag·value·quality다. 시뮬레이터 상태 API의 seq는 이 Kafka 원시 JSON 필드와 혼동하지 않는다.

01 · 이 권의 스택 역할 사전

보강 안내 3/4 · 프로그램과 통신 규칙을 구분하기

이름 · 종류	쉬운 역할	이 값의 길에서 확인할 것
시뮬레이터 · 프로그램	가상 공정이 센서·구동기 상태를 만든다.	고장 주입 전후 값·운전 상태
Modbus TCP · 통신 규칙	주소의 숫자/켜짐 상태를 읽고 쓴다.	레지스터·코일 주소와 자료형
EdgeX · 수집 서비스	설비 통신을 공통 Event로 바꾼다.	device-modbus-core-data-export
MQTT · 통신 규칙	토픽에 발행하고 구독한다.	발행 토픽, QoS, 구독자
EMQX · 프로그램	MQTT 메시지를 구독자에게 전한다.	EdgeX/lite 발행이 들어왔는가
Telegraf bridge · 프로그램	MQTT 형식을 Kafka 공통 스키마로 바꾼다.	필드·태그·시각·품질
FUXA · HMI 프로그램	직접 값을 읽고 운전원 명령을 보낸다.	현재 값과 조작 후 상태

왜 EdgeX와 FUXA가 모두 Modbus를 쓰나?

같은 설비에 서로 다른 목적의 클라이언트가 붙는다. EdgeX는 수집 파이프라인에 값을 보내고, FUXA는 운전원 화면과 직접 조작을 담당한다.

01 · 이해를 확인하는 관찰 순서

보강 안내 4/4 · 이 교본의 2026-09-17 설명과 현재 코드를 구분

관찰	보야 할 변화	아직 증명하지 않는 것
센서값	시뮬레이터 상태 또는 Modbus 주소 값이 변함	Kafka에 기록됐는지
MQTT	EMQX에 입력 토픽의 메시지가 도착	Flink가 판정했는지
Kafka raw	같은 태그·시각·값·품질을 가진 JSON 기록	FUXA 조작이 가능한지
FUXA	현재 계측값이 보이고 명령 후 상태가 바뀜	이력 DB·Grafana 경로 정상

실제 코드와 대조한 근거

fxa/build_project.py · telegraf/bridge-edgex.conf · bridge-lite.conf · kafka/create-topics.sh. 기존 교본의 성능 수치와 화면은 작성 당시 기록이다.

이후 읽기: 제2권은 Kafka에서 Flink 결과·알람까지, 제3권은 저장·관제·명령의 되읽기까지 이어진다.

01

현장·데이터 수집

센서의 숫자가 공통 계약의 메시지가 되기까지

중복 설명을 통합한 3 권 읽기용 편집본

원문: 1_기술이해.pdf · 294 쪽

원문 기준일: 2026-09-17 | 편집일: 2026-09-21

이 권의 흐름

시작하기 — 전체 문제와 AR-100 공정

1 센서와 계측 — 계측·품질·물리 모델·현장 인터록

2 Modbus와 IoT 미들웨어 — 레지스터·정규화·EdgeX·저장 후 전송

3 MQTT와 브로커 — 토픽·전달 보장·세션·브리지

원리 → 실제 구현 → 예외·한계 → 대안의 순서를 유지합니다.

같은 표·설정·설명은 정보 위치에 모으고, 원문의 설명 차이는 편집 주석으로 구분했습니다.

읽기 전에

이 문서는 첨부된 교본의 내용을 읽기 편하게 재편집한 것입니다. 원문에 인용된 저장소나 시스템을 새로 실행·검증하지 않았습니다. 제품 버전, 설정값, 라이선스 언급, 성능 수치는 원문 작성 시점의 설명이며 현재 상태에 대한 별도 검증이 아닙니다.

본문의 “설계 보고서”는 교본이 인용한 15 쪽 자료를 뜻합니다. “원문” 또는 “원문 교본”은 첨부된 294 쪽 PDF 입니다. “원문의 별도 실습서”는 이 편집본의 2 권과 다른 자료입니다. 원문의 소절 번호를 가리키는 표현에는 “원문”을 붙였고, 장 번호 1~9는 그대로 유지했습니다.

원문에는 같은 주제가 다른 측정 구간이나 구현 단계에서 다시 등장합니다. 이런 수치·조건·예외는 단순 중복으로 지우지 않았습니다. 특히 설계 요구와 구현 완료, 상태의 정확히 한 번 처리와 외부 출력 중복, 데이터 감사 추적과 명령 감사 로그를 구분합니다.

읽는 순서: 1 권에서 현장과 수집을 이해하고, 2 권에서 Kafka·Flink·CEP·ML 을 읽은 뒤, 3 권에서 저장·화면·운영 검증을 연결합니다.

표·코드·도해의 출처는 해당 장의 원문 범위입니다. 별도의 편집 주석은 원문 쪽수를 함께 표시합니다. 코드는 원리와 설정을 읽기 위한 발췌이며, 중간 생략이나 줄바꿈이 있는 블록을 그대로 실행하는 설치 매뉴얼은 아닙니다.

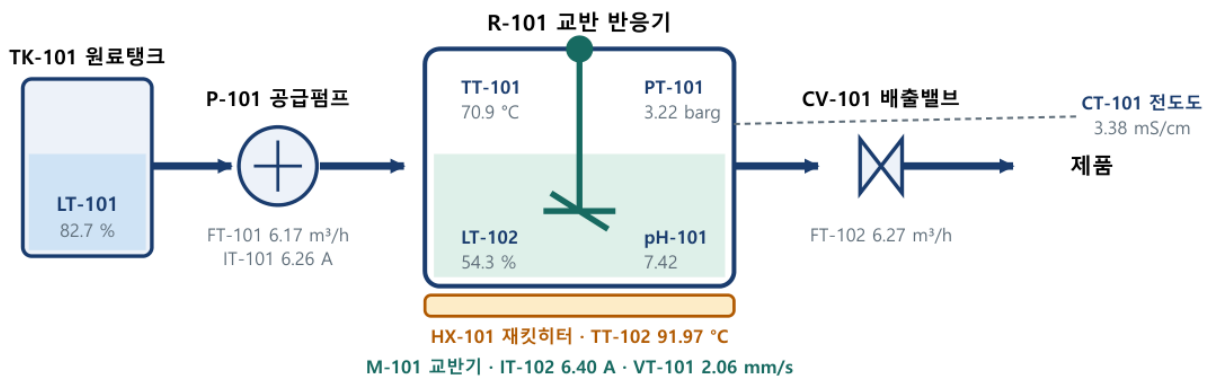
시작하기 · 이 시스템이 풀려는 문제

원문 교본 2~4 쪽의 설명을 통한

AR-100 은 원료탱크에서 펌프로 반응기에 원료를 공급하고, 교반기와 재킷히터로 반응 조건을 만들며, 제어밸브를 통해 배출하는 반응기 라인입니다. 센서값이 올라가는 길과 조작 명령이 내려가는 길을 함께 읽어야 전체를 이해할 수 있습니다.

해야 할 일	이를 맡는 기술
계속 관측하고 기록한다	센서·Modbus·EdgeX·MQTT·Kafka
단발 변화와 지속 이상을 구분한다	Flink 의 시간·상태·규칙 처리
사건의 순서와 태그 관계를 살핀다	CEP·머신러닝 이상 탐지
근거를 보고 조치하고 다시 확인한다	시계열 기록·SCADA·사람의 판단

설계 목표와 구현 범위를 구분해서 읽습니다. 원문의 도입부에는 매뉴얼 기반 조치와 사람의 승인, 재측정까지 포함한 목표가 나옵니다. 반면 뒤의 구현 설명에는 자동 예고 체크·명령 감사 로그가 없는 것으로 적혀 있습니다. 목표가 등장했다고 그 절차까지 구현되었다고 보지는 않습니다. 구체적인 경계는 3 권 9 장에 모았습니다.



원문 도해 · AR-100 반응기 라인과 정상 운전점 (원문 4 쪽)

녹화값이나 독립된 난수만 재생하는 시스템과 달리, 이 예제는 질량수지·에너지수지·헤드스페이스 압력을 계산합니다. 따라서 제어 입력이 물리 상태를 바꾸고 그 결과가 다시 계측값으로 올라옵니다. 다만 실제 감지 소자·전류 루프·ADC 까지 재현하는 것은 아닙니다.

1 센서와 계측 — 숫자는 어디서 오는가

원문 교본 5~40 쪽의 설명을 통합

1.1 센서와 트랜스미터 — 물리량이 전기가 되는 곳

센서와 트랜스미터

산업 현장에 계측 장비가 들어오기 전에는, 공장 운전원이 설비마다 붙은 유리관 온도계와 부르동관 압력계를 눈으로 읽고 종이 일지에 적었습니다. 부르동관(Bourdon tube) 압력계는 지금도 배관에 붙어 있는 그 둥근 시계 모양 압력계입니다. 납작하게 늘린 금속 관을 C자로 구부려 한쪽 끝을 막아 두 고, 반대쪽으로 압력이 들어오면 납작한 관이 둥글어지려 하면서 C자가 조금 펴집니다. 그 끝의 움직임을 톱니로 확대해 바늘을 돌리는 구조입니다. 전기가 전혀 필요 없고, 사람이 눈으로 읽는 것 말고는 아무 데도 값을 보내지 못합니다.

문제는 세 가지였습니다. 첫째, 사람이 돌아다니는 주기(보통 한 시간에 한 번)보다 빠른 변화는 아무도 보지 못했습니다. 둘째, 위험한 위치의 계기는 읽으려 갈 수 없었습니다. 셋째, 기록이 사람의 손글씨로만 남아 나중에 검증할 수 없었습니다. 계측 기술은 이 세 가지를 해결하려고 생겼습니다.

오늘날 계측점 하나는 보통 네 부분으로 이루어집니다.

- ① 감지 소자(sensing element) — 물리량에 따라 자기 성질이 변하는 물체입니다. 온도계라면 백금 저항체(RTD, Resistance Temperature Detector)가 쓰입니다. 대표적인 Pt100은 0 °C에서 저항이 정확히 100 Ω이고, 온도가 1 °C 오를 때마다 약 0.385 Ω씩 커집니다. 압력계라면 얇은 금속판(다이어프램)이 압력을 받아 휘고, 그 휘어짐이 붙어 있는 스트레인 게이지(strain gauge)의 저항을 바꿉니다. 스트레인 게이지는 아주 가는 금속 저항선을 지그재그로 접어 얇은 필름에 붙여 놓은 우표만 한 조각입니다. 이 조각을 다이어프램에 접착해 두면, 판이 휘면서 저항선도 같이 늘어납니다. 금속선은 늘어나면 더 가늘고 길어지므로 저항이 커지고, 줄면 저항이 작아집니다. 그래서 “압력 → 판의 휨 → 선의 늘어남 → 저항 변화”라는 사슬이 만들어집니다. 유량계라면 배관에 만든 좁은 목(오리피스) 앞뒤의 압력 차이를 잽니다. 물리량을 직접 재는 게 아니라, 물리량에 비례해 변하는 전기적 성질을 재는 것이 계측의 출발점입니다.
- ② 변환(transducer) — 저항·전하·주파수 같은 값을 다루기 쉬운 전압이나 전류로 바꿉니다.
- ③ 신호 조정(signal conditioning) — 신호가 너무 작으면 증폭하고, 잡음을 걸러내고, 감지 소자 고유의 비선형성을 보정합니다. 비선형성이란 입력이 두 배가 되어도 출력이 정확히 두 배가 되지는 않는 성질입니다. 앞에서 Pt100이 “1 °C마다 0.385 Ω”이라고 했지만, 그 0.385라는 기울기 자체가 온도 에 따라 조금씩 달라집니다. 0 °C 근처에서보다 300 °C 근처에서 더 완만해집니다. 열전대(서로 다른 두 금속을 붙여 온도차만큼 전압이 생기는 소자)는 훨씬 더 심하게 휩니다. 그래서 제조사는 미리 여러 온도에서 실제 저항을 재어 곡선을 표로 만들어 두고, 계기는 그 곡선을 거꾸로 적용해 흰 것을 직선으로 펴 줍니다. 이때 쓰는 보정값을 캘리브레이션 계수(calibration coefficient)라고 부릅니다. 이 계수가 어디서 오는지는 원문 1.7 절에서 다룹니다.
- ④ 전송(transmitter) — 정리된 신호를 표준 형식으로 바깥에 내보냅니다. 여기까지 하는 장치를 트랜스미터(transmitter)라고 부릅니다.

현장에서 “센서”라고 부르는 물건은 대개 ①만 가리키고, 태그가 붙어 관리되는 장비는 ①~④를 다 가진 트랜스미터입니다. 그래서 태그 이름이 “온도 센서”가 아니라 TT-101 (Temperature Transmitter 101)인 것입니다. 그리고 트랜스미터가 내보내는 값은 물리량 그 자체가 아니라 지시값입니다. 이 구분은 원문 1.8 절에서 이 장의 가장 중요한 주제로 다시 나옵니다.

4-20 mA 와 그 후계자들

트랜스미터가 값을 내보내는 가장 오래되고 아직도 가장 널리 쓰이는 방식은 4-20 mA 아날로그 전류 루프입니다. 측정 범위의 최저값(LRV, Lower Range Value)을 4 mA, 최고값(URV, Upper Range Value)을 20 mA에 대응시키고 그 사이를 직선으로 잇습니다. 예를 들어 0-10 barg 범위의 압력 트랜스미터가 12 mA를 흘리고 있다면, 지시값은 $(12-4)/(20-4) \times 10 = 5.0$ barg입니다.

왜 0-20 mA가 아니라 4-20 mA 일까요. 0 mA를 “측정 최저값”이 아니라 “선이 끊어졌음”으로 쓰기 위해서입니다. 만약 0 mA가 0 barg를 뜻한다면, 케이블이 단선됐을 때 제어실은 “압력이 0입니다”라는 그럴듯한 거짓말을 보게 됩니다. 4 mA를 바닥으로 잡으면 0 mA는 물리적으로 나올 수 없는 값이 되고, 시스템은 이를 고장으로 판정할 수

있습니다. 정상 범위 밖에 “고장”을 위한 자리를 남겨 두는 이 발상은 뒤에 나오는 데이터 품질 설계 전체에 그대로 이어집니다.

전류를 쓰는 이유도 있습니다. 전압은 케이블이 길어지면 저항 때문에 떨어지지만, 직렬 회로의 전류는 어디서 재도 같습니다. 그래서 수백 미터 떨어진 현장 계기도 제어실에서 정확히 읽을 수 있습니다.

아날로그 루프의 한계는 분명합니다. 선 한 쌍이 값 하나만 실어 나릅니다. 기기 상태, 시리얼 번호, 고장 진단, 교정 이력을 같이 보내고 싶어도 보낼 자리가 없습니다. 이 요구에서 두 갈래의 디지털 통신이 나왔습니다.

HART(1980년대 중반) — 기존 4~20 mA 전류 위에 작은 디지털 신호를 엮습니다. 배선을 그대로 두고 진단 정보를 얻을 수 있어 가장 널리 퍼졌습니다. 필드버스(fieldbus)(1990년대) — 현장(field) 계기 여러 대를 디지털 통신선 한 쌍(bus)에 나란히 매달아 값·상태·설정을 주고받게 만든 규격군을 통틀어 부르는 말입니다. 계기마다 선을 따로 끌지 않아도 되고, 값 말고 다른 것도 보낼 수 있습니다. PROFIBUS-PA와 Foundation Fieldbus가 대표입니다.

여기서 초보가 잘못 배우기 쉬운 순서를 바로잡아 둡니다. Modbus는 이 계보의 후계자가 아닙니다. Modbus는 1979년에 나왔으므로 HART보다도 필드버스보다도 앞섰고, 진단 정보를 실기 위해 만들어진 것도 아닙니다. Modbus는 PLC와 그 주변 장치를 잇는 가장 단순한 디지털 프로토콜로 태어났고, 그래서 지금도 레지스터 번호와 값밖에 실어 나르지 못합니다(원문 1.12 절). 즉 “아날로그 → HART → 필드 버스”가 진단 정보를 향해 간 갈래이고, Modbus는 그 옆에서 단순함으로 살아남은 다른 갈래입니다.

이 교본이 실제로 다루는 현장 프로토콜은 Modbus 하나뿐입니다. 구현 경로는 Modbus → EdgeX → MQTT 하나이고, OPC-UA는 원문 1.12 절의 비교 대상으로만 나옵니다(docs/journal/11-teaching-guide.md §11.6 “이 교재가 다루지 않는 것” 표에 OPC-UA | EdgeX device-opcua로 확장 가능하나 미포함 이라고 적혀 있습니다). 그리고 현장 계기 대부분은 여전히 아날로그로 PLC에 들어간 다음, PLC 안에서 디지털 숫자가 되어 바깥으로 나갑니다.



원문 도해 · 물리량에서 네트워크 메시지까지의 변환 (원문 8 쪽)

아날로그 전류가 정수가 되는 과정

전류는 연속적인 양이지만 컴퓨터는 정수만 다룹니다. 그 사이를 잇는 부품이 ADC(Analog-to-Digital Converter, 아날로그-디지털 변환기)입니다. ADC는 입력 범위를 2의 거듭제곱 개수로 잘게 나누어 몇 번째 칸인지를 정수로 내놓습니다. 12비트 ADC라면 4096칸, 16비트라면 65536칸입니다. 이 칸 개수를 분해능(resolution)이라 하고, 칸 하나의 폭보다 작은 변화는 원리적으로 구분할 수 없습니다. 0~10 barg를 12비트로 재면 한 칸이 약 0.0024 barg입니다.

PLC는 그 정수를 자기 메모리에 보관합니다. 여기서 흔한 오해를 하나 짚습니다. “PLC 메모리의 기본 단위는 16비트 레지스터”라는 말은 정확하지 않습니다. PLC 내부 메모리는 제조사마다 다른 주소 체계로 비트·바이트·워드(16비트)·더블워드(32비트)·실수형을 함께 씁니다. “한 칸이 16비트인 레지스터”는 PLC 일반의 사실이 아니라, Modbus라는 프로토콜이 바깥에 드러내는 형식입니다. 이 장에서 우리가 다루는 숫자는 전부 그 Modbus 창구를 통해 나온 것이므로, 아래 설명은 Modbus의 데이터 모델 이야기로 읽으시면 됩니다(홀딩 레지스터의 정확한 정의는 원문 1.11 절과 다음 장에서 다룹니다).

문제는 72.43 같은 소수를 16비트 정수 한 칸에 담을 수 없다는 점입니다. 해결책은 두 가지입니다.

스케일된 정수 — 값을 10배·100배 해서 정수로 싣고, 받는 쪽이 다시 나눕니다. 온도 72.0 °C를 720으로 보내는 식입니다. 단순하지만 받는 쪽이 배율을 모르면 값이 10배 틀립니다. 부호가 필요한 값은 Int16으로 담습니다. Int16은 부호 있는 16비트 정수로 -32768부터 32767까지를 나타냅니다. float32(32비트 부동소수점) — 레지스터 두 칸을 붙여 실수 하나를 담습니다. 이때 쓰는 국제 규격이 IEEE 754입니다. 32비트를 부호 1비트 + 지수 8비트 + 가수 23비트로 나뉘, 실수를 “부호 × (1. 가수) × (2의 지수 제곱)” 꼴로 근사해 담기로 한 약속입니다. 십진수로 유효숫자 약 7자리를 담을 수 있어서 72.43 같은 값을 그대로 실을 수 있습니다. 대신 두 워드를 어떤 순서로 붙일지를 약속이 똑같이 약속해야 합니다. 이 순서를 엔디안(endianness)이라 합니다. 여러 바이트로 된 수를 메모리와 통신선에 늘어놓는 순서를 뜻하고, 큰 자리(상위) 바이트를 먼저 놓는 방식이 빅엔디안(big-endian)입니다. 순서가 어긋나면 값이 조금 틀리는 게

아니라 완전히 엉뚱한 숫자가 나옵니다. UINT32(Uint32) — 부호 없는 32 비트 정수입니다. 앞의 U는 unsigned(부호 없음)를 뜻하고, 0 부터 4,294,967,295 까지를 담습니다. 음수가 될 일이 없는 카운터에 씁니다. 이것도 레지스터 두 칸을 씁니다.

이 교본이 다루는 시스템은 계측값에 float32 를, 제어 설정치에 Int16 스케일 정수를, 스캔 카운터에 Uint32 를 씁니다. 왜 그렇게 나누는지는 원문 1.11 절에서 실제 설정 파일로 확인합니다.

주의. “센서에서 값을 읽는다”는 문장은 대부분 거짓입니다. 실제로는 PLC 의 메모리 칸을 읽습니다. 그 칸의 내용은 PLC 가 마지막으로 스캔했을 때 ADC 가 내놓은 정수를 PLC 가 자기 방식대로 변환해 둔 결과입니다. 우리가 읽는 시점과 물리량이 그 값이었던 시점 사이에는 언제나 시차가 있습니다.

AR-100 · ①~③ 이 없습니다. 여기서 정직하게 말해 둡니다

이 교본의 실습 시스템에는 감지 소자도, 4-20 mA 전류 루프도, ADC 도 없습니다. 지금까지 읽은

①~③ 은 실제 공장을 이해하기 위한 내용이고, 코드로 존재하지 않습니다. simulator/plant.py 의 _measure() 는 물리 계산값에 잡음을 더하고 하한에서 자른 뒤 그대로 돌려줍니다.

```
value += random.gauss(0.0, self.noise.get(tag, 0.05))
lo = self.clamp_lo.get(tag)
return value if lo is None else max(value, lo)
```

그리고 simulator/sim.py 의 f32_regs() 가 그 실수를 곧바로 float32 두 워드로 만들어 홀딩 레지스터에 씁니다.

```
def f32_regs(value: float) -> list[int]:
    """float32 → 홀딩레지스터 2 워드 (big-endian, ABCD)."""
    return list(struct.unpack(">HH", struct.pack(">f", float(value))))
```

실제 공장이라면 PT-101 의 다이어프램이 휘고 → 스트레인 게이지 저항이 변하고 → 트랜스미터가 4-20 mA 로 바꾸고 → PLC 아날로그 입력 카드의 ADC 가 정수로 자르고 → 그 정수가 레지스터에 있습니다. AR-100 은 이 네 단계를 “잡음에 섞인 지시값”이라는 한 줄로 대신합니다. 그림의 사슬로 말하면 ④ 디지털화부터가 실물입니다.

그래서 얻는 것과 잃는 것이 분명합니다. 잃는 것은 ADC 양자화 계단(값이 0.0024 단위로 층져 보이는 현상), 단선(0 mA) 고장, 루프 전원 고장처럼 ①~③ 에서만 생기는 고장 유형입니다. 이 시스템 에서는 재현할 수 없습니다. 얻는 것은 그 아래 계층을 있는 그대로 연습할 수 있다는 점입니다. 통신 불량은 센티널 값 BAD_QUALITY = -999999.0 으로, 계기 고장은 spike 주입으로, 열화는 bearing_wear · drift 로 넣을 수 있습니다(원문 1.9 절·원문 1.11 절).

실습 중 “우리 시스템의 4-20 mA 와 ADC 는 어디 있나”라는 질문의 답은 없습니다, Modbus 레지스터가 출발점입니다. 이 한 줄을 알고 넘어가야 뒤 절의 숫자들이 헛갈리지 않습니다.

1.2 표본 주기와 Hz — 왜 초당 몇 번인가

샘플링과 주파수

물리량은 시간에 대해 끊김 없이 이어지지만, 우리가 가질 수 있는 건 정해진 순간들의 스냅샷뿐입니다. 이 스냅샷을 뜨는 행위를 표본 추출(sampling, 샘플링), 스냅샷 사이의 간격을 표본 주기 (sampling period), 1 초 동안 뜨는 스냅샷 개수를 표본 주파수(sampling frequency)라고 하고 단위 Hz(헤르츠)로 씁니다.

1 Hz 는 1 초에 한 번입니다. 표본 주기로는 1000 ms 입니다. 10 Hz 는 100 ms 마다 한 번, 1 kHz 는 1 ms 마다 한 번입니다. 둘은 서로 역수 관계이므로 어느 쪽으로 말해도 같은 뜻입니다.

스냅샷을 뜨는 순간에 실제로 벌어지는 일을 래치(latch)라고 합니다. 정해진 그 시각에 모든 값을 한꺼번에 붙잡아 복사해 두는 동작입니다. 열두 개 값을 하나씩 순서대로 읽으면 첫 값과 마지막 값이 서로 다른 시각의 것이 되지만, 한 번에 래치하면 열두 개가 같은 순간의 묶음이 됩니다. 뒤에서 태그 열두 개가 “한 이벤트”로 묶여 나가는 것이 바로 이 이야기입니다.

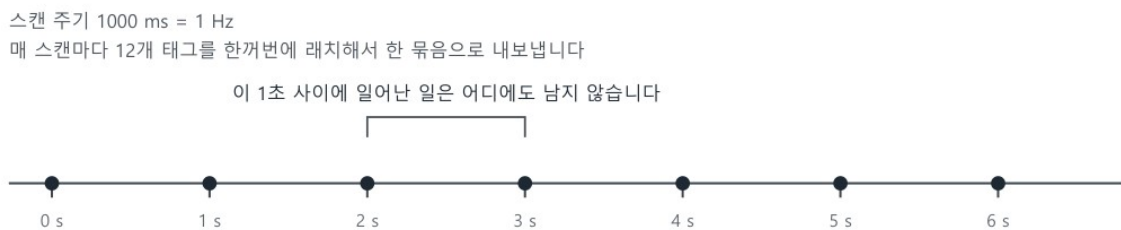
주기를 얼마로 잡을지는 취향이 아니라 물리가 정합니다. 신호 처리에는 나이퀴스트-새넨 표본화 정리라는 규칙이 있습니다. 관심 있는 현상 중 가장 빠른 변화 주파수의 최소 두 배로 표본을 떼야 그 현상을 복원할 수 있다는 것입니다. 이보다 느리게 뜨면 에일리어싱(aliasing, 되접힘)이 일어납니다. 빠른 진동이 사라지는 정도가 아니라, 존재하지 않는 느린 변동으로 둔갑해서 보입니다. 자동차 바퀴가 영상에서 거꾸로 도는 것처럼 보이는 현상과 같은 원리입니다.

그래서 현장에서는 물리량 종류마다 주기가 완전히 다릅니다.

온도 — 액체가 담긴 탱크의 온도는 수십 초에 걸쳐 변합니다. 게다가 온도계 자체가 자기 열용량 때문에 늦게 따라옵니다. 이 늦음의 크기를 시정수(time constant)로 나타냅니다. 온도가 계단처럼 갑자기 바뀌었을 때 지시값이 그 차이의 약 63 %를 따라잡는 데 걸리는 시간입니다. 보호관 (thermowell)에 들어 있는 산업용 온도계는 시정수가 수 초에서 수십 초입니다. 즉 100 ms 마다 값을 떠 봐야 새 정보가 없습니다. 1 초 주기면 충분하고, 더 빠른 주기는 잡음만 늘립니다. 압력·유량 — 초 단위로 변합니다. 0.1~1 초 주기가 일반적입니다. 진동 — 베어링 결함 주파수는 수백~수천 Hz 에 있습니다. 어느 부품이 상했는지 가려내려면 수 kHz~수십 kHz 로 파형을 떠서 주파수 성분으로 분해해야 합니다. 1 Hz 로 뜯 진동 값은 “지금 세기 가 이 정도”라는 요약일 뿐입니다. 이 요약을 RMS 실효값이라고 합니다. 측정값들을 각각 제공해 평균을 낸 뒤 제공근을 씌운 값입니다. 제공하는 순간 부호가 사라지므로, 위아래로 왔다 갔다 하는 진동을 하나의 양수로 요약할 수 있습니다(그냥 산술평균을 내면 위와 아래가 상쇄되어 0 에 가까워져 쓸모가 없습니다).

초보가 반드시 헷갈리는 지점이 있습니다. 파이프라인에는 서로 다른 주기가 최소 세 개 존재합니다.

스캔 주기 — 현장 장치가 자기 메모리를 갱신하는 주기 수집·발행 주기 — 수집기가 그 메모리를 읽어 네트워크로 내보내는 주기 저장·집계 주기 — 데이터베이스가 원본을 유지하거나 요약(다운샘플링)하는 주기 이 셋이 다르면 값이 중복되거나 건너뛰어집니다. 수집 주기가 스캔 주기보다 짧으면 같은 값이 여러 번 수집되고(가짜 고착처럼 보입니다), 길면 그 사이의 값들이 아예 사라집니다. 둘을 일치시키거나, 최소한 왜 다른지를 문서로 남겨야 합니다.



표본 주기를 정하는 일은 “무엇을 보지 않기로 할지”를 정하는 일입니다.

원문 도해 · 표본 주기 사이에서 관측되지 않는 변화 (원문 11 쪽)

AR-100 · 진동도 1 Hz 입니다. 그래서 할 수 있는 것과 없는 것

simulator/plant.yaml 의 scan_interval_ms: 1000 이 모든 태그에 똑같이 적용됩니다. 진동 태그 VT-101 도 1 초에 한 번입니다. 바로 위에서 “진동은 수 kHz 로 떠야 한다”고 했으니 정면으로 어긋나 보입니다. 해명하면 이렇습니다.

plant.py 의 _vibration() 은 진동 파형을 만들지 않습니다. 세기 하나를 바로 계산합니다.

```
base = self.p_agit["base_vibration_mms"]    # plant.yaml: 2.1
v = base * (0.82 + 0.30 * level_frac)
if self.vib_wear > 0.0:
    v *= 1.0 + 3.0 * self.vib_wear
```

즉 VT-101 이 내는 값은 앞에서 말한 세기 요약(RMS 실효값)에 해당하며, 실제 현장에서도 이런 값 은 별도의 진동 감시 장비가 내부에서 수 kHz 로 떠서 1 초에 한 번 요약해 내보내는 형태로 자주 유통됩니다. 이 값으로는 어느 부품이 상했는지 가려낼 수 없습니다. 베어링 외륜 결함인지 내륜 결함 인지 판별하려면 파형을 주파수 성분으로 분해해야 하고, 이 시스템에는 그 데이터도 그 기능도 없습니다.

그래도 1 Hz 요약값으로 할 수 있는 일이 있습니다. 세기가 커지는 추세와, 다른 태그와의 시간 선 후 관계입니다.

bearing_wear 고장을 주입하면 교반기 전류 IT-102 가 먼저 오르고, plant.yaml 의 lag_s: 6 만큼 뒤에 VT-101 이 따라 오릅니다. 실측한 간격은 시행마다 2~6 초였고, 뒤 장의 CEP 가 이 순서를 이렇게 잡아냅니다 — 교반기 전류 9.69A (정격 120% 초과) 후 7 초 내 진동 7.21mm/s 상회 → 베어링 열화 의심 . 1 Hz 는 “부품 진단”에는 모자라고 “열화 추세 감지”에는 충분합니다. 어느 쪽을 하려는지에 따라 필요한 주기가 달라진다는 것이 이 절의 요점입니다.

1.3 단위 — 숫자에 반드시 붙여 다녀야 하는 것

단위와 그 함정

72.43 이라는 숫자 혼자서는 아무 뜻이 없습니다. °C 인지 °F 인지, barg 인지 bara 인지에 따라 정상일 수도 있고 폭발 직전일 수도 있습니다. 그래서 단위는 값과 같은 격의 데이터이며 파이프라인 전 구간 에서 값과 함께 다녀야 합니다.

이 교본에 나오는 단위를 하나씩 풀어 둡니다.

% — 탱크 레벨에 쓰입니다. 미터가 아니라 퍼센트를 쓰는 이유는, 운전원이 “2.1 m”보다 “60 % 찼다”를 더 빨리 판단하기 때문입니다. 다만 0 %가 빈 탱크를 뜻하지 않는 경우가 많습니다. 계측 하한 아래에도 잔량이 남습니다. °C 와 K — 섭씨 온도와 절대온도(켈빈)입니다. 여기서 초보가 자주 거꾸로 배웁니다. 온도 “차이”는 °C 와 K 가 수치상 완전히 같습니다. 1 °C 오른 것은 1 K 오른 것입니다(두 눈금의 간격이 같고 출발점만 다르기 때문입니다). 그러니 온도차를 다루는 계산에서는 굳이 바꿀 필요가 없습니다. 절대 온도로 바뀌어야 하는 곳은 온도의 “비율”이 들어가는 계산입니다. 기체 법칙이 대표적이고, 이 교본의 코드가 바로 그 예입니다. plant.py 의 압력 식에는 $(temp_c + 273.15) / (ambient_c + 273.15)$ 라는 분수가 들어 있습니다. 72 °C 와 22 °C 를 넣으면 $345.15/295.15 = 1.169$ 지만, 섭씨를 그대로 넣으면 $72/22 = 3.27$ 이 됩니다. 압력이 세 배 가까이 틀리는 것입니다. barg 와 bara — 초보가 가장 많이 틀리는 곳입니다. barg 는 게이지압, 즉 대기압을 0 으로 놓고 잰 압력입니다. bara 는 절대압, 즉 진공을 0 으로 놓고 잰 압력입니다. 둘은 약 1.013 bar 차이 납니다. 현장 압력계는 거의 다 게이지압이고, 기체 법칙 계산은 반드시 절대압으로 해야 합니다. 이 차이를 놓치면 압력 모델이 통째로 어긋납니다. m³/h — 시간당 세제곱미터, 부피 유량입니다. 질량 유량(kg/h)과 다릅니다. 밀도가 변하면 둘의 관계도 변합니다. A — 암페어, 전류입니다. 모터 전류는 부하를 간접적으로 재는 가장 싼 방법입니다. 별도 토크 센서 없이 “지금 힘을 많이 쓰고 있다”를 알 수 있습니다. mm/s — 진동 속도의 실효값(원문 1.2 절의 RMS)입니다. 진동은 변위(μm)·속도(mm/s)·가속도(g) 셋 중 무엇으로도 표현할 수 있고, 회전 기계 상태 감시의 국제 표준(ISO 10816 계열)이 속도 실효값을 쓰기 때문에 mm/s 가 관례입니다. pH — 단위가 없는 로그 스케일입니다. pH 6 과 7 의 차이는 수소이온 농도 10 배 차이입니다. 그래서 pH 값의 산술 평균은 화학적으로 의미가 약합니다. mS/cm — 밀리시멘스 퍼 센티미터, 전기전도도입니다. 용액에 이온이 얼마나 녹아 있는지를 봅니다. 농도를 직접 재기 어려울 때 대신 쓰는 값입니다.

1.4 태그 명명 규칙 — TT·PT·FT·LT·IT·VT 가 무슨 뜻인가

태그는 이름이 아니라 주소입니다.

설비 하나에 계측점이 수십, 공장 하나에 수만 개가 붙습니다. 각각을 “3 번 탱크 위쪽 온도계”처럼 문장으로 부르면 도면·제어 로직·데이터베이스·경보 목록이 전부 어긋납니다. 그래서 산업계는 태그(tag) 라는 짧고 규칙적인 식별자를 씁니다. 미국계측제어학회의 ISA-5.1 표준이 그 규칙이고, 전 세계 P&ID(Piping and Instrumentation Diagram, 배관 계장도 — 배관과 설비, 그 위에 붙은 계기와 제어 선을 한 장에 기호로 그린 도면)가 이를 따릅니다.

태그는 문자부 + 숫자부로 나뉩니다. 문자부의 첫 글자는 무엇을 재는가, 그다음 글자는 그걸로 무엇을 하는가를 뜻합니다. 같은 알파벳이라도 첫 글자일 때와 후속 글자일 때 뜻이 다릅니다. 이것이 이 표를 읽는 열쇠입니다.

첫 글자 (측정 변수)	뜻	후속 글자 (기능)	뜻
L	Level — 액위	T	Transmitter — 신호를 만들어 보냄
T	Temperature — 온도	I	Indicator — 현장에 표시
P	Pressure — 압력	C	Controller — 설정치와 비교해 제어
F	Flow — 유량	V	Valve — 구동 밸브
A	Analysis — 성분 분석(pH, 전도도)	S	Switch — 임계에서 점점 동작
V	Vibration — 진동	A	Alarm — 경보
I	Current — 전기 전류	R	Recorder — 기록
E	Voltage(EMF) — 전압	H	High — 상한 쪽

이 조합 규칙을 알면 태그를 읽을 수 있습니다.

LT = Level Transmitter — 액위 트랜스미터 TT = Temperature Transmitter — 온도 트랜스미터 PT = Pressure Transmitter — 압력 트랜스미터 FT = Flow Transmitter — 유량 트랜스미터 VT = Vibration Transmitter — 진동 트랜스미터 IT = Current Transmitter — 전류 트랜스미터. ISA-5.1 에서 첫 글자 I 는 Current(전기 전류)입니다. Indicator 는 I 가 후속 글자일 때의 뜻이고, 전압은 첫 글자 E(EMF)가 말합니다. 그러니 IT-101 · IT-102 는 현장 관행이 아니라 표준 그대로의 표기입니다. TIC 라면 Temperature Indicating Controller — 온도를 표시하면서 제어까지 하는 장치입니다.

숫자부는 “라인 번호 + 계기 순번”입니다. 헛갈리기 쉬우니 AR-100 의 실제 예로 못 박아 두겠습니다. 이 공장에는 반응기 라인이 하나뿐이고 그 라인의 번호가 101 입니다. 그래서 라인 위의 설비가 전부 101 을 답니다 — TK-101 (원료탱크),

P-101 (공급펌프), R-101 (반응기), CV-101 (제어밸브), M-101 (교반기), HX-101 (재킷히터). 여기에 붙는 계기는 같은 종류가 둘 이상이면 끝자리를 올려 구분합니다.

TT-101 은 반응기 R-101 의 온도, TT-102 는 재킷히터 HX-101 의 온도입니다. 둘은 같은 설비에 붙은 두 개가 아니라, 같은 라인의 서로 다른 설비에 붙은 온도계입니다. LT-101 은 원료탱크 TK-101 의 레벨, LT-102 는 반응기 R-101 의 레벨입니다. FT-101 은 공급 유량, FT-102 는 배출 유량입니다. 공장에 라인이 더 생기면 TT-201, TT-301 처럼 백의 자리가 올라갑니다.

설비 태그 약어도 같은 규칙입니다. TK 는 Tank(탱크), P 는 Pump(펌프), R 은 Reactor(반응기), CV 는 Control Valve(제어밸브), M 은 Motor(모터), HX 는 Heat Exchanger(열교환기)입니다.

초보가 반드시 짚어야 할 점은 이것입니다. 태그는 사람이 부르는 별명이 아니라, 파이프라인 전 구간 에서 데이터를 찾아가는 주소입니다. 태그 문자열 하나가 Modbus 레지스터 주소, 규격 한계값, 데이터베이스 인덱스, 화면 바인딩, 경보 규칙에 동시에 박힙니다. 그래서 태그 정의는 반드시 파일 한 곳에만 존재해야 하고, 나머지는 거기서 생성돼야 합니다. 두 곳에 적어 두면 언젠가 반드시 어긋나고, 어긋난 날 화면에는 여전히 그럴듯한 숫자가 떠 있습니다.

1.5 규격 — LSL 과 USL

정상과 비정상을 가르는 선

숫자 하나만 놓고는 정상인지 알 수 없습니다. 70.9 라는 값은 온도라면 평범하고, 압력이라면 재난입니다. 그래서 태그마다 허용 범위를 따로 정의해 둡니다.

LSL(Lower Specification Limit, 하한 규격치) — 이 아래로 내려가면 안 되는 값 USL(Upper Specification Limit, 상한 규격치) — 이 위로 올라가면 안 되는 값이 값들은 통제로 구하는 게 아니라 사람이 결정해서 주는 값입니다. 근거는 세 가지입니다. 설비의 설계 허용치(압력용기의 설계압력, 모터의 정격 — 그 기기가 계속 돌아도 문제없다고 제조사가 보증하는 최대 운전값), 제품 품질 기준(반응 온도 범위를 벗어나면 규격품이 안 나옴), 그리고 법규·안전 기준입니다.

한쪽만 있는 태그가 많습니다. 압력은 높으면 위험하지만 낮다고 위험하진 않으므로 USL 만 둡니다. 유량은 운전 조건에 따라 0 부터 최대까지 정상이므로 양쪽 다 없을 수 있습니다. “한계가 없음”과 “한계를 아직 안 정했음”은 다른 상태이며, 설정 파일에서 이를 구분해 적어야 합니다.

초보가 거의 반드시 헷갈리는 두 가지를 미리 짚습니다.

① 규격 한계(specification limit) ≠ 관리 한계(control limit). 규격 한계는 “넘으면 안 되는 선”으로 사람이 줍니다. 관리 한계는 “평균 흔들리는 폭”으로 측정 데이터의 평균과 표준편차에서 계산합니다. 관례적으로 평균 $\pm 3\sigma$ 를 씁니다. 여기서 3 이라는 숫자는 임의가 아닙니다. 값이 정규분포(원문 1.6 절)를 따른다고 보면 평균 $\pm 3\sigma$ 밖으로 나가는 일은 전체의 약 0.3 %, 즉 1000 번에 3 번쯤입니다. 우연으로는 좀처럼 안 생기는 일이라, 그런 값이 나오면 “운이 나빴다”보다 “뭔가 달라졌다”로 읽는 편이 맞습니다. $\pm 2\sigma$ 로 잡으면 우연히도 5 %가 벗어나므로 경보가 너무 자주 울리고, $\pm 4\sigma$ 로 잡으면 실제 변화를 오래 놓칩니다. 규격 이탈은 “불량”, 관리 한계 이탈은 “변화”입니다. 이 교본의 시스템은 규격 검사와 통계 검사를 서로 다른 검출기로 따로 돌립니다.

② 규격 한계 ≠ 안전 트립 설정치. 규격을 넘으면 경보가 울려 사람이 대응합니다. 안전 트립은 사람을 기다리지 않고 설비를 즉시 멈춥니다. 트립 설정치는 규격보다 높게 잡습니다. 그래야 “경보 → 사람 대응 → 그래도 안 되면 강제 정지”라는 층이 생깁니다. 두 값을 같게 잡으면 층이 사라져 사소한 잡음에도 공장이 멈춥니다.

1.6 계측 잡음 — 값이 흔들리는 게 정상입니다

잡음의 정체와 다루는 법

같은 물리량을 연속으로 재면 값이 조금씩 다릅니다. 이 흔들림을 잡음(noise)이라고 합니다. 원인은 겹겹입니다.

열잡음 — 저항체 안 전자의 무작위 운동. 물리 법칙상 없앨 수 없습니다. 전자기 간섭(EMI) — 옆을 지나는 인버터·용접기·모터 케이블이 신호선에 유도 전압을 만듭니다. 여기서 인버터는 모터 속도를 바꾸려고 들어오는 전기를 초당 수천 번 잘라 원하는 주파수를 다시 만들어 내는 장치(VFD, 가변속 구동장치)입니다. 그 “자르는” 동작 자체가 강한 전자기 잡음을 뿜습니다. 공장에서 가장 흔한 원인입니다. 양자화 오차 — 원문 1.1 절의 ADC 칸 나누기 때문에 생기는 계단 모양 오차. 공정 자체의 요동 — 배관 안 난류(유체가 곧게 흐르지 않고 소용돌이치며 흐르는 상태), 액면의 출렁임, 교반 날개가 만드는 맥동(날개가 한 바퀴 돌 때마다 압력·유량이 규칙적으로 오르내리는 것).

이건 계측 오류가 아니라 실제로 일어나는 일입니다.

잡음의 크기는 보통 표준편차 σ (시그마)로 표현합니다. σ 는 값들이 자기들의 평균 둘레에서 얼마나 널리 흩어지는지를 재는 수치입니다. 값의 분포가 정규분포(평균 근처에 값이 가장 많이 모이고 멀어질수록 급격히 드물어지는 좌우 대칭의 종 모양 분포)를 따른다고 보면, 전체의 약 68 %가 평균 $\pm 1\sigma$ 안 에, 약 95 %가 평균 $\pm 2\sigma$ 안에 들어옵니다.

여기서 반드시 구분해야 합니다. σ 가 말해 주는 중심은 참값이 아니라 “그 계기가 내는 값들의 평균”입니다. 계기가 교정이 틀어져 늘 0.5 °C 씩 높게 읽고 있어도, 그 높은 값들끼리는 여전히 암전히 모여 있으므로 σ 는 작게 나옵니다. σ 가 작다는 것은 “흔들림이 적다”일 뿐, “참값에 가깝다”가 아닙니다. 교정 드리프트·설치 위치·동적 지연이 만드는 치우침은 σ 안에 들어 있지 않습니다. 이 구분이 원문 1.7 절과 원문 1.8 절의 주제입니다.

여기서 중요한 사실이 하나 있습니다. 잡음이 전혀 없는 계측값은 정상이 아니라 고장 신호입니다. 살아 있는 센서는 반드시 조금씩 떨립니다. 소수점 아래가 완벽히 고정된 값이 이어진다면 센서가 얼어붙었거나, 중간 어딘가가 마지막 값을 복사해 내보내고 있는 것입니다. 원문 1.9 절의 고착이 바로 이 상태입니다.

잡음을 줄이려고 필터를 세게 거는 것은 공짜가 아닙니다. 평균을 많이 낼수록 값은 매끄러워지지만 응답이 느려집니다. 압력이 실제로 튀어 올랐는데 필터가 그걸 몇 초 늦게 통과시키면, 그 몇 초 동안 안전 로직은 아무것도 모릅니다. 매끄러움과 빠름은 맞바꾸는 관계이고, 계측 설계의 상당 부분이 이 교환비를 정하는 일입니다.

잡음이 만드는 성가신 부작용도 있습니다. 값이 0 근처일 때 잡음이 음수 쪽으로 붙으면 물리적으로 불가능한 음수가 나옵니다. 정지한 펌프의 유량이 $-0.03 \text{ m}^3/\text{h}$ 로 찍히는 식입니다. 실제 트랜스미터는 측정 범위 하한(LRV) 아래를 잘라내므로, 시스템도 같은 클램프를 넣어야 현실과 맞습니다.

1.7 정확도와 정밀도, 그리고 교정 — 흩어짐과 치우침은 다른 문제입니다

과녁 두 장으로 끝나는 구분

병원 비유로 돌아가 보겠습니다. 어떤 체온계로 열 번을 재니 37.2 · 37.2 · 37.3 · 37.2 · 37.2 도가 나왔습니다. 아주 일관됩니다. 그런데 이 체온계는 공장 출고 때부터 0.6 도 높게 읽도록 틀어져 있었습니다. 열 번을 재도, 백 번을 재도, 평균을 내도 0.6 도 높은 값이 흔들림 없이 나옵니다. 화면만 봐서는 이 사실을 알 길이 없습니다.

여기서 두 축이 갈립니다.

정밀도(precision) — 흩어짐. 같은 대상을 반복해서 잴 때 값들이 서로 얼마나 모여 있는가. 원문 1.6 절의 σ 가 바로 이것을 잽니다. 잡음이 작으면 정밀도가 높습니다. 정확도(accuracy) — 치우침. 그 값들의 중심이 참값에서 얼마나 떨어져 있는가. 이 떨어진 양을 바이어스(bias, 편의)라고 합니다.

과녁에 비유하면, 정밀도는 탄착군이 얼마나 뭉쳐 있는가이고 정확도는 그 뭉친 자리가 과녁 한복판에서 얼마나 벗어났는가입니다. 둘은 완전히 별개라서 네 가지 조합이 다 나옵니다.

실무에서 중요한 결론이 여기서 나옵니다. 평균을 내거나 필터를 걸면 흩어짐은 줄지만 치우침은 전혀 줄지 않습니다. 오히려 흔들림이 사라지는 만큼 치우친 값이 더 또렷하고 더 믿음직해 보입니다. 데이터를 매끄럽게 만드는 어떤 처리도 교정을 대신하지 못합니다.



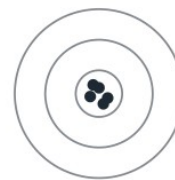
정밀도 높음 · 정확도 낮음

σ 작다 · 치우쳐 있다
교정으로만 고쳐집니다



정밀도 낮음 · 정확도 높음

σ 크다 · 평균은 맞다
평균·필터로 개선됩니다



둘 다 높음

교정 직후의 상태
시간이 지나면 왼쪽으로

원문 도해 · 정밀도와 정확도: 흩어짐과 치우침 (원문 17 쪽)

교정(calibration)이 실제로 하는 작업

교정은 “계기를 청소하는 일”도 “값을 예쁘게 만드는 일”도 아닙니다. 믿을 수 있는 기준과 대 보고, 어긋난 만큼 계수를 다시 잡는 일입니다. 절차는 이렇습니다.

① 기준을 준비합니다. 대상 계기보다 훨씬 정확하고 그 자체가 상위 표준으로 교정되어 있는 계기 (기준기), 또는 물리적으로 값이 정해진 기준점을 씁니다. 온도라면 얼음물(0 °C)과 끓는 물(100 °C), 압력이라면 표준 분동식 압력계, pH 라면 값이 보증된 표준 완충액입니다.

② 여러 점에서 나란히 잹니다. 보통 측정 범위의 0 · 25 · 50 · 75 · 100 % 지점에서 기준값과 계기 의 지시값을 짝지어 적습니다. 올라가면서 한 번, 내려오면서 한 번 재서 이력 현상(같은 지점인데 올라올 때와 내려갈 때 값이 다른 것)까지 봅니다.

③ 두 개의 계수를 다시 잡습니다. 표 전체가 위아래로 통째로 밀려 있으면 영점(zero, 오프셋)을 옮기고, 위로 갈수록 오차가 커지면 스펜(span, 기울기)을 고칩니다. 곡선으로 휘어 있으면 원문 1.1 절

③의 비선형 보정 계수를 다시 넣습니다. 원문 1.1 절에서 말한 캘리브레이션 계수는 바로 이 작업의 산물입니다.

원문 설명의 차이: 이 고장 설정의 설명문은 “Autoencoder 만 탐지”라고 쓰지만, 원문 225·293 쪽의 후속 실측에서는 Z-Score 도 탐지했습니다. 여기서 보존할 사실은 “교정 계층이 없고, drift 가 상관 구조를 깨도록 주입된다”는 구현입니다. 탐지 비교는 2 권 7 장에서 두 기록을 함께 읽습니다. [원문 18, 225, 293 쪽]

④ 성적서를 남깁니다. 언제, 어떤 기준기로, 어떤 오차가 나왔고 어떻게 조정했는지를 기록합니다. 규제 산업에서는 이 서류가 없으면 그 계기로 잹 값 전체가 무효가 됩니다.

교정을 하지 않으면 무슨 일이 생기는지가 핵심입니다. 감지 소자는 열·진동·화학적 부식에 오래 노출 되면 특성이 서서히 변합니다. 이 느린 치우침을 캘리브레이션 드리프트라고 합니다. 드리프트는 σ 를 키우지 않습니다. 값은 여전히 암전히 흔들리고, 규격 안에 있고, 그래프도 예뻐합니다. 그래서 화면을 아무리 들여다봐도 발견되지 않습니다. 주기적으로 기준기를 대 보는 것 말고는 알아낼 방법이 없고, 그래서 공장은 계기마다 교정 주기(보통 6 개월~2 년)를 정해 관리합니다.

AR-100 · 교정 계층이 없습니다

이 실습 시스템에는 교정 절차도, 캘리브레이션 계수를 담는 필드도 없습니다. 레포 전체를 검색해도 calibration 이라는 이름의 설정이나 코드가 없습니다. plant.yaml 의 noise 절은 흩어짐(σ)만 정의합니다. 치우침을 만드는 항은 고장 주입 쪽에만 있습니다.

```
drift:
  desc: "pH-101/CT-101 동시 미세 드리프트 → 단일 임계치 무반응, Autoencoder 만 탐지"
  default_duration_s: 90
  target: [pH-101, CT-101]
  magnitude: 0.55
```

plant.py 의 _measure() 가 이 고장을 ramp = min(1.0, f.elapsed / 45.0) 로 45 초에 걸쳐 서서히 었습니다. 즉 이 시스템의 “드리프트”는 계기가 늙어 가는 캘리브레이션 드리프트라기보다, 공정과 센서가 함께 서서히 치우쳐 상관 구조가 깨지는 상황의 재현입니다. 단일 임계치로는 안 잡히고 다변량 모델만 잡아낸다는 것을 보이기 위한 장치입니다.

원문 1.10 절에 나오는 “디지털 트윈 메타데이터(캘리브레이션 계수·설치 좌표)”도 마찬가지입니다. 설계 보고서가 수집 미들웨어의 역할로 드는 항목이지만, AR-100 에는 구현돼 있지 않습니다. 개념으로 알아 두고, 이 시스템에서 찾지는 마십시오.

1.8 트랜스미터 지시값과 물리 진값 — 이 장에서 가장 중요한 구분

아무도 진값을 모릅니다

반응기 안의 실제 압력을 물리 진값(true value)이라고 합시다. 우리가 손에 쥔 것은 PT-101 이라는 트랜스미터가 내놓은 지시값(indicated value, measured value)입니다. 이 둘은 원리적으로 같을 수 없습니다. 차이를 만드는 원인은 다음과 같습니다.

잡음 — 원문 1.6 절. 매 순간 랜덤하게 어긋납니다(흩어짐). 캘리브레이션 드리프트 — 원문 1.7 절. 시간이 지나면 감지 소자의 특성이 변합니다. 6 개월 전 교정한 계기는 지금 조금씩 틀립니다(치우침). 설치 위치 — 반응기 바닥에 붙은 압력계와 위쪽에 붙은 압력계는 액주 높이만큼 다른 값을 냅니다. 교반기 반대편에 붙은 온도계는 국소적으로 더 찰니다. 동적 지연 — 온도계가 제 온도에 도달하는 데 걸리는 시간, 즉 원문 1.2 절의 시정수 때문입니다. 온도가 빠르게 오르는 중이라면 지시값은 늘 진값보다 낮습니다. 계기 고장 — 소자 파손, 배선 접촉 불량, 신호 증폭단 이상. 이때 지시값은 진값과 아무 상관 없는 숫자가 됩니다.

여기서 설계상 가장 중요한 결론이 나옵니다. 제어 로직과 안전 로직이 볼 수 있는 것은 지시값뿐입니다. 실제 PLC는 반응기 안에 손을 넣을 수 없습니다. 그러므로 계기가 고장 나서 잘못된 값을 내면, 안전 로직은 그 잘못된 값을 믿고 동작합니다.

이것을 스푸리어스 트립(spurious trip, 오작동 정지)이라고 하며, 실제 플랜트의 주요 골칫거리입니다. 압력은 멀쩡한데 압력계가 고장 나서 7 barg를 지시하면 펌프가 멈춥니다. 생산이 서고, 원인을 찾으려 몇 시간이 갑니다. 그렇다고 “진짜 위험할 때만 멈추게” 만들 방법은 없습니다. 무엇이 진짜인지 알 방법 자체가 없기 때문입니다.

현실의 대응은 값을 더 믿는 게 아니라 계기를 늘리는 것입니다. 같은 물리량을 재는 트랜스미터를 두 개 두거나(이중화), 세 개 두고 2oo3 투표로 판정합니다. 2oo3은 two out of three, 즉 “셋 중 둘”이라는 뜻입니다. 세 계기 중 두 개 이상이 같은 판정을 내릴 때만 그 판정을 채택합니다. 그러면 계기 하나가 고장 나 혼자 엉뚱한 값을 내도 다수에 묻혀 설비를 세우지 않으면서, 진짜 이상은 둘 이상이 동의하므로 잡아냅니다. 비용이 세 배 드는 대신 가동률을 사는 것입니다.

시뮬레이터를 만들 때 이 구분을 놓치면 훈련 도구로서 쓸모가 없어집니다. 시뮬레이터는 진값을 “알고” 있기 때문에, 안전 로직이 진값을 참조하도록 짜기가 아주 쉽습니다. 그렇게 만들면 계기 고장을 주입해도 아무 일이 일어나지 않습니다. 현실에서 가장 자주 일어나는 사고 유형이 재현되지 않는 것입니다.

1.9 데이터 품질 — 결측·고착·급등

나쁜 데이터의 세 가지 얼굴

센서 데이터가 망가지는 방식은 크게 세 가지입니다. 셋은 원인도 탐지 방법도 후속 처리도 전부 다르므로 반드시 구분해야 합니다.

- ① 결측(missing) — 값이 아예 없음 어떤 시각의 값이 존재하지 않는 상태입니다. 원인은 통신 계층에만 있지 않습니다. 패킷 유실뿐 아니라 센서 하드웨어 결함, 급격한 전압 강하, 센서 헤드의 캘리브레이션 지연까지 복합적으로 결측을 만듭니다. 결측은 세 유형 중 그나마 다루기 쉽습니다. 없다는 사실 자체가 눈에 보이기 때문입니다.
- ② 고착(stuck / frozen) — 같은 값이 계속 나온 가장 위험한 유형입니다. 데이터는 빠짐없이 도착하고, 값도 정상 범위 안에 있으며, 화면의 그래프는 아름다운 수평선을 그립니다. 그런데 그 값은 30분 전에 멈춘 값입니다. 원인은 센서 소자 고장, 중간 장비가 마지막 값을 반복 전송, 캐시에 고정된 값 등입니다. USL/LSL 검사도, 급변 검사도 이걸 잡지 못합니다. 잡는 방법은 원문 1.6 절에서 말한 성질을 뒤집는 것입니다. 일정 구간 동안의 분산이 0에 가까우면 의심합니다. 살아 있는 센서는 반드시 떨기 때문입니다.
- ③ 급등(spike / outlier) — 물리적으로 불가능한 순간 변화 한 스캔 만에 값이 꺾충 뛰었다가 돌아옵니다. 전기적 간섭이나 순간적 접촉 불량일 때는 버려야 할 잡음이고, 실제 과압이나 서지일 때는 가장 중요한 신호입니다. 서지는 배관이나 전기 계통에 순간적으로 밀려드는 과도한 압력·전류를 말합니다. 밸브를 급히 닫을 때 배관을 때리는 수격(water hammer)이 대표적입니다. 버려야 할 잡음인지 진짜 사건인지를 값만 보고 구분할 방법은 없습니다. 그래서 급등은 “거르는 대상”이 아니라 “표시하고 남기는 대상”으로 다뤄야 하며, 판단은 다른 태그와 의 관계나 인간에게 맡깁니다.

다뤄서는 안 되는 방법 세 가지 결측을 0으로 채우지 마십시오. 0은 물리적으로 의미 있는 값입니다. 유량 0은 “펌프 정지”이고, 온도 0 °C는 “결빙”입니다. 없는 값을 0으로 채우면 평균이 무너지고, 기계학습 모델은 있지도 않은 사건을 배웁니다. 센티널 값을 그대로 저장하지 마십시오. 현장 장치는 통신 불량을 -9999나 32767 같은 특별한 값으로 표시하는 경우가 많습니다. 이 값이 필터링되지 않고 데이터베이스에 들어가면, 평균값이 수천 단위로 어긋납니다. 센티널은 파이프라인 입구에서 “값 없음”으로 번역돼야 합니다. 보간값으로 원본을 덮어쓰지 마십시오. 뒤장에서 배울 보간(interpolation)은 빈 곳을 계산으로 메우는 기법입니다. 그 결과를 원천 데이터에 그대로 써 버리면 실제 계측치와 추정치의 경계가 사라집니다. 사고가 나서 “그때 온도가 얼마였습니까”를 따질 때, 그 숫자가 실제로 댄 값인지 나중에 계산한 값인지 아무도 증명할 수 없게 됩니다. 감사 추적(Audit Trail)이 무너지는 것입니다. 감사 추적이란 어떤 값이 언제 어디서 어떤 손을 거쳐 지금 이 자리에 있는지를 나중에 되짚어 증명할 수 있게 남겨 둔 기록을 말합니다. 식품·의약·화학처럼 규제를 받는 산업에서는 법적 요구 사항입니다.

대신 필요한 것이 품질 코드(quality code)입니다. 값 옆에 “이 값이 어떤 상태인지”를 함께 씁니다. 실측인지, 직전값으로 채운 것인지, 선형 보간으로 채운 것인지를 구분합니다. 그러면 값을 읽지 않으면서 추정치를 추정치로 표시할 수 있고, 나중에 “추정치를 빼고 다시 계산”이 가능해집니다.

결측을 줄이는 층과 처리하는 층

결측이 생기기 전에 막는 장치가 먼저입니다. 브로커의 영속 세션은 구독자가 끊긴 동안의 메시지를 보관하고, 에지의 Store-and-Forward는 중앙으로 보내지 못한 데이터를 현장에 저장했다가 다시 보냅니다. 그래도 남은 공백을 품질 코드와 보간으로 다룹니다. 영속 세션은 3장, 실제 저장 후 전송 구성은 2장에 통합했습니다.

1.10 출처 — 숫자 하나에 붙어야 하는 이력

숫자의 신분증

출처(provenance)는 원래 미술품의 소장 이력을 뜻하는 말입니다. 데이터에서는 “이 값이 언제, 어디서, 무엇에 의해, 어떤 과정을 거쳐 여기 있는가”를 가리킵니다. 값만 저장하고 출처를 버리면 다음 질문에 답할 수 없습니다.

이 값은 언제의 값인가 — 센서가 켜 시각인가, 수집기가 읽은 시각인가, 데이터베이스가 저장한 시각인가 이 값은 어느 설비의 것인가 — 같은 TT-101이 여러 라인에 있을 수 있습니다 이 값은 실측인가 추정인가 이 값은 중복인가 — 같은 값이 두 번 도착한 것인가, 진짜로 같은 값이 두 번 나온 것인가 그래서 실용적인 최소 출처 정보는 다음과 같습니다.

최소 출처는 시각·공간 식별자·순번·품질·단위입니다. 시각은 센서 사건 시각인지 수집·처리 시각인지 명시해야 합니다. epoch는 시각을 세기 시작하는 기준점이며 여기서는 1970-01-01 00:00 UTC 이후의 나노초를 사용합니다. site·device·tag는 어느 공장·설비·계측점인지 구분합니다. 순번이나 타임스탬프는 재전송된 같은 관측을 알아보는 근거입니다. 같은 일을 여러 번 처리해도 결과가 한 번 처리한 것과 같아지는 성질을 멍등성이라고 합니다. 품질과 단위는 숫자의 의미와 신뢰 상태를 함께 보존합니다. 실제 계약의 필드는 2장의 정규화 설명에서 확인합니다.

여기에 더해, 수집 미들웨어는 값에는 실리지 않지만 반드시 관리해야 하는 정보를 따로 보관하는 것이 이상적입니다. 센서의 캘리브레이션 계수와 물리적 설치 좌표가 대표적입니다. 이것들은 디지털 트윈 메타데이터라고 불리며, “이 값이 왜 이만큼 틀릴 수 있는가”를 나중에 설명해 주는 근거가 됩니다 (다만 원문 1.7 절에서 밝혔듯 AR-100에는 구현돼 있지 않습니다).

출처를 지키는 가장 확실한 구조는 원본과 가공본을 물리적으로 분리하는 것입니다. 무손실 원본을 담는 경로와, 중복을 제거하고 빈 곳을 메운 정제본을 담는 경로를 나눕니다. 그러면 정제 로직을 고쳐도 원본은 그대로 남고, 언제든지 원본에서 다시 만들 수 있습니다.

1.11 AR-100의 계측·물리 모델

AR-100에서는, 계측 태그 12점 전체가 simulator/plant.yaml 한 파일에 정의돼 있고, 이 파일이 태그·레지스터 주소·단위·규격의 유일한 출처입니다. EdgeX 장치 프로파일 (edgex/profiles/ar100.profile.yaml), Flink가 읽는 규격 테이블 (flink/sql/tag_limits.csv), FUXA 화면 정의 (fuxa/project.json)는 전부 여기서 생성됩니다. 태그를 바꾸면 make regen-edgex && make regen-fuxa로 나머지를 다시 만듭니다. 아래 표의 홀딩 레지스터 주소·규격·잡음 σ 는 plant.yaml에서, 정상 운전값은 50분 수렴 후 실측값입니다.

태그	계측 대상	단위	홀딩 레지스터	LSL / USL	정상 운전값	잡음 σ (단위 값)
LT-101	원료탱크 레벨	%	HR 0-1	10.0 / 95.0	82.70 %	0.05 %
LT-102	반응기 레벨	%	HR 2-3	15.0 / 90.0	54.31 %	0.05 %
TT-101	반응기 온도	°C	HR 4-5	40.0 / 95.0	70.91 °C	0.06 °C
TT-102	재킷 온도	°C	HR 6-7	— / 125.0	91.97 °C	0.09 °C
PT-101	반응기 압력	barg	HR 8-9	— / 6.0	3.22 barg	0.012 barg
FT-101	공급유량	m ³ /h	HR 10-11	— / —	6.17 m ³ /h	0.04 m ³ /h
FT-102	배출 유량	m ³ /h	HR 12-13	— / —	6.27 m ³ /h	0.04 m ³ /h
IT-101	펌프 전류	A	HR 14-15	— / 14.0	6.26 A	0.03 A
IT-102	교반기 전류	A	HR 16-17	— / 9.6	6.40 A	0.025 A
VT-101	교반기 진동	mm/s	HR 18-19	— / 7.1	2.06 mm/s	0.06 mm/s
pH-101	반응기 pH	pH	HR 20-21	6.2 / 8.4	7.42 pH	0.006 pH
CT-101	전도도	mS/cm	HR 22-23	— / —	3.38 mS/cm	0.012 mS/cm

IT-102의 USL 9.6 A는 임의의 값이 아닙니다. plant.yaml의 agitator.rated_current_a: 8.0, 즉 교반기 모터 정격이 8.0 A이고 그 120 %가 9.6 A입니다. “정격의 120 %를 넘으면 과부하”라는 현장의 판정 기준이 그대로 규격이 된 것입니다.

주의 — 잡음 σ 열을 세로로 비교하지 마십시오. 위 표의 σ 는 각 태그의 단위로 적힌 값입니다. pH-101의 0.006은 pH 단위이고 TT-102의 0.09는 °C입니다. 단위가 다르므로 “0.006이 0.09보다 작다”는 문장은 아무 의미가 없습니다. 잡음의 크기는 센서 종류와 측정 원리에 따라 따로 정해지는 것이고, 서로 비교하고 싶다면 각 태그의 운전 범위로 나눈 상대값(예: TT-102는 $0.09/125 = 0.072\%$)으로 바꿔야 합니다.

주의. 태그 정의가 두 곳에 있으면 반드시 어긋납니다. 이 프로젝트에서도 설계 문서 ARCHITECTURE.md 48행 표에는 TT-102의 USL이 120으로 적혀 있지만, 실제 설정 파일 plant.yaml과 거기서 생성된 tag_limits.csv는 125.0입니다. 동작하는 값은 plant.yaml 쪽입니다. 단일 출처 원칙이 지켜야 하는 것이 바로 이런 어긋남입니다. 이 장에서도 설계 문서에만 있는 값과 코드가 실제로 만들어 내는 값을 그때그때 구분해 적었습니다.

AR-100 · 값을 난수로 흔들지 않습니다. simulator/plant.py가 질량수지 · 에너지수지 · 헤드스페이스 압력 세 식을 1차 오일러 적분으로 풉니다. 용어를 먼저 풀겠습니다.

질량수지(mass balance) — 들어온 만큼 늘고 나간 만큼 준다는 당연한 셈입니다. 탱크의 내용물은 공급 유량에서 배출 유량을 뺀 만큼 변합니다. 에너지수지(energy balance) — 같은 셈을 열에 대해 하는 것입니다. 히터가 넣은 열 - 벽으로 빠져나간 열 - 찬 원료가 식힌 열이 내용물의 온도를 올립니다. 1차 오일러 적분 — 미래 상태를 한 번에 계산하지 않고, 아주 짧은 시간(여기서는 1초) 동안은 변화율이 일정하다고 보고 “현재값 + 변화율 × 시간”을 반복해 나가는 가장 단순한 수치 계산법입니다. 매 스캔마다 이 덧셈을 한 번씩 하는 것이 시뮬레이션입니다.

물리 파라미터는 전부 plant.yaml에서 주입되며 코드에 박힌 상수는 없습니다.

```
// 질량수지 — 들어온 만큼 늘고 나간 만큼 준다
self.rx_vol += (q_in - q_out) * dt_s / 3600.0
// 에너지수지 — 히터로 넣고, 벽으로 잃고, 찬 원료가 식힌다
q_loss = heat_loss_kw_per_k * (self.temp_c - ambient)
q_feed = (q_in / 3600.0) * rho * cp * (ambient - self.temp_c)
self.temp_c += (q_heat - q_loss + q_feed) / (mass * cp) * dt_s
// 헤드스페이스 압력 — 레벨과 온도에 물리적으로 묶인다
head_frac = max(1.0 - level_frac, 0.05)
p_abs = charge * (temp_c + 273.15) / (ambient + 273.15) * (ref_head / head_frac)
p_vap = 0.0061094 * math.exp(17.625 * temp_c / (temp_c + 243.04)) // Magnus 식
```

세 번째 식이 이 시뮬레이터를 “진짜로 반응하는 공장”으로 만드는 핵심입니다. 헤드스페이스(headspace)는 반응기 안 액면 위의 빈 공간입니다. 질소로 charge_pressure_bar: 3.4만큼 미리 채워 두었습니다(절대압입니다 — 원문 1.3절). 액면이 올라가면 이 공간이 압축되므로 압력이 오르고, 온도가 오르면 기체가 팽창해 또 오르고, 여기에 Magnus 식(온도만 넣으면 그 온도에서 물이 증발하려는 압력을 돌려주는 경험식)으로 구한 수증기 분압이 더해집니다. 마지막에 대기압 1.013 bar를 빼서 barg로 내보냅니다. 그리고 이 분수에 섭씨를 그대로 넣으면 안 되는 이유는 원문 1.3절에서 이미 계산해 보였습니다.

이 결합 덕분에 하나를 건드리면 다른 것들이 연쇄적으로 움직입니다. 레벨이 오르면 압력이 오르고 → 압력이 오르면 펌프가 밀어내야 할 압력(토출 배압)이 커져 유량이 줄고 → 유량이 줄면 펌프 전류가 떨어집니다. 배압이란 펌프가 밀어 보내려는 쪽에서 거꾸로 되밀어 오는 압력을 말합니다. 코드에서는 $q_{in} *= \max(0.35, 1.0 - \text{press_now} / 12.0)$ 한 줄이 이 효과입니다.

AR-100 · 태그끼리 실제로 묶여 있다는 증거. 실측한 상관계수가 이를 뒷받침합니다. 상관 계수는 두 값이 함께 움직이는 정도를 -1에서 +1 사이의 한 숫자로 요약한 것입니다. 읽는 법은 이렇습니다. +1에 가까우면 한쪽이 오를 때 다른 쪽도 거의 예외 없이 같이 오릅니다. -1에 가까우면 한쪽이 오를 때 다른 쪽이 어김없이 내립니다. 0에 가까우면 서로 아무 관계가 없습니다. 부호는 방향, 절댓값은 관계의 또렷함입니다. 아래 +0.989는 “거의 한 몸처럼 같이 움직인다”로, -0.980은 “거의 한 몸처럼 반대로 움직인다”로 읽으면 됩니다.

태그 쌍	상관계수	물리적 의미
FT-101 ↔ IT-101	+0.989	유량이 늘면 펌프 부하가 는다
TT-101 ↔ pH-101	-0.980	온도가 오르면 pH가 내려간다
LT-102 ↔ PT-101	+0.936	레벨이 오르면 헤드스페이스가 압축돼 압력이 오른다
pH-101 ↔ CT-101	-0.934	pH가 오르면 전도도가 내려간다

태그 쌍	상관계수	물리적 의미
LT-102 ↔ IT-102	+0.764	레벨이 오르면 교반 부하가 는다

난수 시뮬레이터에는 이런 구조가 없습니다. 상관 구조가 없으면 다변량 이상 탐지를 가르칠 수도, 시험할 수도 없습니다. 다변량 이상 탐지란 태그를 하나씩 따로 보지 않고 여러 태그를 한꺼번에 보면서 “평소의 어울림”이 깨진 것을 잡아내는 방법입니다. 뒤 장에서 배울 오토인코더 (autoencoder)가 바로 이 표를 학습합니다. 오토인코더는 입력을 일부러 좁은 통로로 압축했다가 원래대로 복원해 보게 시키는 신경망이고, 평소의 어울림을 잘 배워 둔 모델은 정상 데이터는 거의 완벽히 복원하지만 어울림이 깨진 데이터는 복원에 실패합니다. 그 실패의 크기로 이상을 판정합니다. plant.yaml 의 drift 고장이 노리는 것이 바로 이 지점입니다 — pH 와 전도도를 동시에 같은 방향으로 밀어 올려 위 표의 -0.934 라는 관계를 깨뜨리되, 각 태그는 규격 안에 머무르게 합니다. 단일 임계치는 아무 반응도 하지 못합니다.

AR-100 · 물리 파라미터를 손으로 맞추지 않았습니다. 파라미터 스윙(값들을 격자로 늘어 놓고 전부 돌려 본 뒤 목표에 가장 가까운 조합을 고르는 방법)으로 정했습니다. 히터 용량·밸브 Cv· 최대 유량·보충 유량을 조합해 가며, 목표 운전점 (LT-102 55 %, TT-101 72 °C, PT-101 3.0 barg)과 의 제품 오차를 최소화했습니다. 여기서 Cv(밸브 계수)는 그 밸브를 완전히 연 상태에서 정해진 압력차를 걸었을 때 얼마나 많은 유체가 지나가는가를 나타내는 밸브의 크기 지표입니다. Cv 가 클수록 굵은 밸브입니다.

plant.yaml 에 들어 있는 확정값은 다음과 같습니다.

```
pump: { max_flow_m3h: 14.0, rated_current_a: 11.0, no_load_current_a: 2.4 }
valve: { cv: 10.0 }
heater: { max_power_kw: 700.0, kp: 45.0 }
feed_tank: { area_m2: 3.0, height_m: 4.0, refill_m3h: 8.0 }
```

주의 — 스윙 기록과 확정값이 한 군데 다릅니다. docs/journal/02-tier1-edge.md § 2.2 에 남은 스윙 결과는 refill=9.0 입니다. 하지만 실제로 동작하는 plant.yaml 은 refill_m3h: 8.0 입니다(스윙 격자가 [9.0, 12.0] 두 점뿐이었고, 최종 파일에는 8.0 으로 들어갔습니다). 나머지 세 값(히터 700kW, Cv 10.0, 최대 유량 14.0 m³/h)은 기록과 파일이 일치합니다. 단일 출처는 plant.yaml 이고, 기록 문서가 아닙니다. 이 절이 방금 가르친 원칙을 이 절 자신에게 적용하면 이렇게 됩니다.

이때 남은 차이 하나가 중요한 교육 지점입니다. 온도 설정치는 72 °C 인데 실제 온도는 70.91 °C 에서 멈춥니다. 히터가 P 제어만 쓰기 때문에 생기는 정상상태 오차(droop)입니다.

P 제어(비례 제어) — 지금의 오차(설정치 - 현재값)에 비례해서 출력을 내는 방식입니다. 코드로는 `duty = min(max(kp * err / 100.0, 0.0), 1.0)` 한 줄이며, plant.yaml 의 heater.kp: 45.0 이 그 비례 상수입니다. 오차가 크면 세게, 작으면 약하게 땀니다. 문제는 오차가 0 이 되면 출력 도 0 이 된다는 것입니다. 열이 계속 새고 있는 반응기에서 히터를 꺼 버리면 온도가 떨어지므로, 결국 오차가 조금 남아 있는 지점에서 균형을 잡습니다. 이 남은 오차가 droop 입니다. I 제어(적분 항) — 오차를 시간에 걸쳐 계속 더해 나가면서, 쌓인 양만큼 출력을 더 보태는 방식입니다. 작은 오차라도 오래 남아 있으면 합이 커지므로 출력이 계속 올라가고, 결국 오차를 0 까지 밀어붙입니다. P 와 I 를 함께 쓰는 것이 PI 제어이고, 실제 플랜트의 온도 루프는 거의 다 PI(또는 여기에 D 를 더한 PID)입니다.

AR-100 이 P 제어만 쓰는 것은 버그가 아니라 의도된 선택이며, 남은 오차는 물리적으로 올바른 거동입니다. 이를 “고치려” 들면 모델이 비현실적이 됩니다.

AR-100 · 잡음과 클램프. 계측 잡음도 태그마다 다른 σ 로 plant.yaml 의 noise 절에 정의돼 있고, `_measure()` 가 물리 계산값에 `random.gauss(0.0,  $\sigma$ )` 를 더한 뒤에야 레지스터에 씁니다. 물리 계산 단계와 계측 단계가 코드에서 분리돼 있다는 점이 중요합니다. 물리 진값은 `_pressure()` · `_vibration()` 같은 함수가 만들고, 지시값은 `_measure()` 를 통과한 뒤에야 존재합니다 — 원문 1.8 절의 구분이 코드 구조 그대로입니다.

원문 1.6 절에서 말한 음수 문제도 실제로 일어났습니다. 초기 구현에서 펌프를 정지했더니 유량이 -0.03 m³/h 로 찍혔습니다(docs/journal/02-tier1-edge.md § 2.5·§ 2.6). 물리적으로 불가능한 값입니다. 그래서 해당 태그들에 `clamp_lo: 0.0` 을 넣어 실제 트랜스미터의 측정 하한(LRV)처럼 잘라냈고, 수정 뒤 같은 시험에서는 0.07 m³/h, 0.03 m³/h 가 나옵니다. 잡음이 0 근처에서 음수 쪽으로 붙어 도 0 에서 멈춥니다. 온도 태그 TT-101 · TT-102 에는 이 클램프가 없습니다 — 온도는 음수가 될 수 있기 때문입니다.

여기에 더해 autopilot 이 있습니다. 설정치를 고정해 두면 정상 데이터가 늘 똑같아 학습할 상관 구조가 생기지 않습니다. 그래서 `autopilot.period_s: 240` 마다 목표를 다시 뽑고 `ramp_s: 45` 에 걸쳐 완만히 이동시킵니다. 범위는 펌프 속도 45~80 %, 밸브 개도 35~60 %, 온도 64~80 °C 입니다. 운전원이 화면에서 손으로 조작하면 그 설정치는 `hold_after_manual_s: 300` 동안 autopilot 에서 제외됩니다. 사람의 조작을 기계가 되돌려 버리는 일을 막는 장치입니다.

현장 안전 인터록

AR-100 · 고압 인터록. plant.yaml 의 physics.interlock 에 정의돼 있습니다. pressure_trip_barg: 6.5 를 넘으면 펌프를 강제로 트립시키고, pressure_reset_barg: 5.2 아래로 떨어져야 해제합니다. 트립과 복귀 사이에 1.3 bar 의 간격을 둔 것을 히스테리시스(hysteresis)라고 합니다. 이게 없으면 압력이 설정치 언저리에서 떨 때 펌프가 초당 여러 번 켜졌다 꺼지는 채터링(chattering)이 일어나 모터가 상합니다.

이 로직은 화면이 아니라 현장 쪽 코드에 있습니다. 이유는 세 가지입니다. 속도 — 화면에 조건을 걸면 판정이 네트워크를 왕복한 뒤에 내려옵니다. 현장 로직은 스캔 주기 안에서 끝납니다. 가용성 — 네트워크가 끊기거나 브라우저가 닫히거나 관제 서버가 죽어도 인터록은 계속 동작 해야 합니다. 화면에 있는 안전 로직은 화면이 없을 때 없습니다. 결정성 — 산업 제어가 요구하는 것은 인터록, 쓰기 확인(Echo Check), 명령 실행 감사 로깅 같은 결정론적 안전 메커니즘입니다. 읽기 전용 트렌드 대시보드는 이를 보장할 수 없습니다. 그래서 인터록 상태는 Coil 20 에 읽기 전용으로 노출됩니다. 화면은 “지금 인터록이 걸렸다”를 볼 수는 있어도 풀 수는 없습니다.

그리고 원문 1.8 절의 구분이 여기서 결정적으로 작동합니다. 처음 구현에서는 인터록이 물리 계산 결과 인 self._pressure(...) 를 봤습니다. 그랬더니 압력계 고장을 주입해도 트립이 걸리지 않았습니다. 실제 PLC 는 진값을 모르므로, 직전 스캔의 트랜스미터 지시값을 참조하도록 고쳤습니다.

```
// 실제 PLC 와 동일하게 직전 스캔의 '트랜스미터 지시값' 을 참조한다.
// 따라서 계기 고장(spike)으로도 트립이 발생한다.
press_now = self.measured_pressure or self._pressure(rx_level_frac, self.temp_c)
```

고친 뒤 실측 결과입니다.

```
=== 5. spike 고장 → 고압 인터록 트립 ===
PT-101=6.51 barg (USL 6.0) 인터록코일 20=True FT-101=0.00
→ 과압 인터록으로 펌프 트립 확인
```

계기 고장 하나로 생산이 멈추는 스푸리어스 트립이 눈앞에서 재현됩니다. 이것이 실제 플랜트에서 가장 자주 일어나는 사고 유형이며, 시뮬레이터가 이를 재현하지 못하면 훈련 도구로 쓸 수 없습니다.

주의. PT-101 의 USL 은 6.0 barg 인데 인터록 트립은 6.5 barg 입니다. 두 값을 혼동하면 안 됩니다.

6.0 을 넘으면 경보가 발생하고 사람이 대응할 시간이 생깁니다. 6.5 를 넘으면 기계가 사람을 기다리지 않고 펌프를 세웁니다. 두 값을 같게 잡으면 경보와 정지가 동시에 일어나 대응할 층이 사라집니다. 원문 1.5 절에서 말한 “규격 한계 ≠ 안전 트립 설정치”가 실제 설정 파일에서 이렇게 나타납니다.

경계값 읽기: 원문 158 쪽의 구현 코드는 압력 ≥ 6.5 barg 에서 트립하고, 압력 ≤ 5.2 barg 에서 해제합니다. 앞선 설명의 “넘으면 / 아래로”는 개념 설명이며, 경계값 자체의 동작은 이 코드 표기와 구분해 읽습니다. [원문 33, 158 쪽]

1.12 대안과 비교

같은 자리의 다른 도구들 — 데이터 원천을 만드는 방법 난수 발생기 — 값을 무작위로 혼듭니다. 만들기 가장 쉽고 파이프라인 배관이 뚫렸는지 확인하는 데는 충분합니다. 그러나 제어를 해도 값이 변하지 않고, 태그끼리 아무 관계가 없습니다. 임계치 검사까지는 시험 할 수 있어도 제어 루프와 다변량 이상 탐지는 시험할 수 없습니다. 녹화 재생(replay) — 실제 공장에서 받은 과거 데이터를 그대로 다시 틀어 줍니다. 값의 현실성은 최고입니다. 대신 재생이므로 개입할 수 없습니다. 밸브를 닫아도 파일에 적힌 유량이 그대로 나옵니다. 그리고 실제 공장 데이터를 구할 수 있어야 합니다. 물리 모델(이 교본의 선택) — 보존 법칙을 코드로 풀어 상태를 적분합니다. 제어 입력에 실제로 반응하고, 태그끼리 물리적으로 묶이며, 고장을 원하는 시점에 원하는 형태로 주입할 수 있습니다. 대신 운전점을 맞추는 작업이 필요합니다 — 파라미터를 잘못 잡으면 온도가 끓거나 탱크가 비어 버립니다. AR-100 은 파라미터 스왑으로 이 작업을 수치적으로 처리했습니다. 다만 원문 1.1 절에서 밝혔듯, 이 방식은 감지 소자와 전류 루프 아래쪽은 재현하지 않습니다. 실제 PLC + 시험 설비 — 가장 정확하지만 비싸고, 고장을 일부러 낼 수 없으며, 안전 문제 때문에 실험 범위가 좁습니다. 상용 공정 시뮬레이터 — 화학 공정을 정밀하게 푸는 전문 도구가 있습니다. 정확도는 높지만 라이선스 비용이 크고, 교육용으로 내부를 열어 보여주기가 어렵습니다.

본질적 차이는 하나로 정리됩니다. 제어 입력이 출력에 되먹임되는가. 되먹임이 없으면 SCADA 를 가르칠 수 없습니다. SCADA 의 절반이 제어이기 때문입니다.

같은 자리의 다른 도구들 — 태그를 이름 붙이는 방법 ISA-5.1 계열 규칙(이 교본의 선택) — TT-101 처럼 기능 문자 + 번호. 짧고, 도면과 그대로 맞고, 처음 보는 엔지니어도 읽을 수 있습니다. 단점은 계층 구조를 표현하지 못한다는 것입니다. 태그만 봐서는 어느 공장 어느 라인인지 알 수 없어, 사이트·장비 식별자를 따로 붙여야 합니다. AR-100 이 site

· device 를 값마다 함께 싣는 이유입니다. 자유 명명 — reactor_temp_sensor_main 처럼 서술적으로 씁니다. 읽기는 쉽지만 사람마다 다르게 적어 같은 대상에 이름이 여러 개 생깁니다. 도면과도 어긋납니다. OPC-UA 정보 모델 — 태그를 평평한 문자열이 아니라 객체 트리로 표현합니다. 반응기 객체 아래에 온도 속성이 있고, 그 속성에 단위·범위·품질이 딸려 있습니다. 표현력이 가장 높지만 모델을 설계하고 유지하는 비용이 듭니다. Sparkplug B — MQTT 위에서 쓰는 산업 규약으로, 토픽 구조에 그룹·에지노드·디바이스 계층을 강제하고 메트릭에 타입과 품질을 붙입니다. MQTT의 자유로움을 제한해 일관성을 얻는 방식입니다.

어느 쪽을 골라도 지켜야 할 규칙은 같습니다. 정의는 한 곳에만 두고 나머지는 생성한다. AR-100 이 plant.yaml 하나에서 EdgeX 프로파일·Flink 규격표·화면 정의를 모두 생성하는 이유입니다.

같은 자리의 다른 도구들 — 안전 로직을 어디에 둘 것인가 안전계장시스템(SIS, Safety Instrumented System) — 제어용 PLC와 물리적으로 분리된 별도 안전 PLC가 인증된 하드웨어·소프트웨어로 판정합니다. 진짜 위험이 걸린 공정의 표준 답입니다. 비쌉니다. 현장 PLC 내부 로직(이 교본의 선택) — 스캔 주기 안에서 결정론적으로 판정하고, 상태를 읽기 전용으로 만 바깥에 노출합니다. 네트워크·서버·브라우저가 모두 죽어도 살아 있습니다. SCADA 화면 스크립트 — 화면에 “압력이 6.5를 넘으면 펌프 정지” 스크립트를 걸어 둡니다. 만들기는 제일 쉽지만 화면이 닫히면 안전도 닫힙니다.

네트워크 지연만큼 판정이 늦고, 쓰기가 실제로 반영됐는지 확인(Echo Check)하거나 누가 언제 무슨 명령을 내렸는지 감사 로그를 남기는 결정론적 장치가 없습니다. 읽기 전용 트렌드 대시보드를 “완전한 SCADA”라고 부르면 안 되는 이유가 여기 있습니다. 스트림 처리 계층(Flink 등) — 수 밀리초 안에 판정할 수는 있지만, 판정 결과가 다시 네트워크를 타고 현장까지 내려가야 합니다. 그 사이 어느 구간이든 끊기면 안전이 성립하지 않습니다. 스트림 계층은 탐지·분석에 쓰고 안전 정지에는 쓰지 않습니다. 클라우드 룰 엔진 — 지연과 단절 위험이 가장 큼니다. 안전 용도로는 고려 대상이 아닙니다.

층을 나누는 원칙은 단순합니다. 사람을 기다릴 수 있으면 위쪽에, 기다릴 수 없으면 현장에. AR-100에서 USL6.0 경보는 스트림 계층이, 6.5 트립은 현장 로직이 맡는 이유입니다.

2 Modbus 와 IoT 미들웨어 — 신호를 말이 되게

원문 교본 40-79 쪽의 설명을 통합

이 장은 주소와 숫자로 나온 현장 신호를 태그·타입·시각·출처가 있는 데이터로 바꾸는 구간입니다. Modbus 의 한계, EdgeX 의 장비 모델, Telegraf 의 계약 변환, 단절 시 저장과 재전송을 순서대로 살펴봅니다.

본문에 계속 나올 말 하나를 미리 풀어 두겠습니다. 에지(edge, 가장자리)는 데이터가 태어나는 현장 끝단을 가리킵니다. 공장 바닥에 놓인 작은 산업용 컴퓨터, 즉 에지 게이트웨이가 그 자리에 있습니다. 반대말은 중앙 서버실이고, 이 교본은 현장 쪽을 아래, 중앙 쪽을 위로 그립니다.

2.1 Modbus — 1979 년에 맺은 약속 Modbus 는 병원으로 치면 1970 년대에 정해진 차트 양식입니다. 아주 단순해서 누구나 쓸 수 있었고, 그래서 40 년이 넘도록 모든 병동이 여전히 이 양식을 씁니다. 대신 칸이 너무 적습니다. 숫자를 적는 칸만 있고 환자 이름 칸도, 단위 칸도, 측정 시각 칸도 없습니다.

왜 Modbus 가 생겼는가

1968 년 미국 회사 모디콘(Modicon)이 세계 최초의 PLC 를 만들었습니다. PLC(Programmable Logic Controller, 프로그래머블 로직 컨트롤러)는 공장 설비를 제어하는 산업용 소형 컴퓨터입니다. 펌프를 돌리고, 밸브를 열고, 센서 값을 읽어 "압력이 한계를 넘으면 펌프를 멈춰라" 같은 논리를 실행합니다.

문제는 그 다음이었습니다. PLC 가 읽어들이는 값을 상위 컴퓨터로 보내려면 통신이 필요한데, 1970 년대에는 제조사마다 전용 통신 방식을 따로 만들었습니다. 케이블 규격도, 신호 순서도, 명령어도 전부 달랐습니다. A 사 PLC 와 B 사 감시 프로그램을 연결하려면 그 조합만을 위한 변환 장치를 따로 사야 했습니다.

모디콘은 1979 년 Modbus 라는 통신 규약을 만들고 그 명세를 공개했습니다. 누구나 무료로 읽고 구현할 수 있게 한 것입니다. 이것이 결정적이었습니다. 다른 제조사들이 자기 장비에 Modbus 를 엮기 시작했고, 사실상의 산업 표준이 되었습니다. 이 교본이 실측을 뜬 2026 년 9 월 기준으로 47 년째 현역입니다.

살아남은 이유는 오직 하나, 단순해서입니다. 전체 명세가 수십 쪽이고, 마이크로컨트롤러 한 조각으로 구현할 수 있습니다. 반대로 말하면, 단순한 만큼 못 하는 일이 아주 많습니다. 이 장의 후반부가 바로 그 빈칸을 누가 메우는가에 관한 이야기입니다.

어떻게 생겼는가

Modbus 는 요청-응답 방식입니다. 한쪽이 묻고 한쪽이 답합니다. 묻는 쪽을 전통적으로 마스터(master), 답하는 쪽을 슬레이브(slave)라고 부릅니다. 최근 명세는 각각 클라이언트(client)·서버(server) 라고 고쳐 부르지만 현장 문서에는 아직 마스터·슬레이브가 압도적으로 많습니다. 같은 말입니다.

여기서 초보가 가장 먼저 걸려 넘어지는 지점이 있습니다. 슬레이브 쪽이 먼저 말하는 법은 없습니다. 온도가 갑자기 100 도로 치솟아도 PLC 가 알아서 알려 주지 않습니다. 마스터가 물어봐야만 답합니다. 그래서 Modbus 수집은 항상 폴링(polling), 즉 "1 초마다 계속 물어보기"가 됩니다.

뒤에 나올 MQTT 와 대비되는 지점이 바로 여기인데, 흔히 쓰는 "Modbus 는 pull, MQTT 는 push"라는 요약은 절반만 맞습니다. 정확한 차이는 누가 먼저 말을 꺼낼 수 있는가입니다. MQTT 에서는 장비 쪽이 언제든지 스스로 메시지를 보낼 수 있고, 언제 보낼지는 보내는 쪽이 정합니다. "값이 바뀌었을 때만 보낸다"는 MQTT 의 규칙이 아니라 발행하는 프로그램의 선택입니다. 실제로 이 교본의 우회 경로 시뮬레이터도 값이 바뀌든 말든 1 초마다 발행합니다. Modbus 에서는 그 선택지 자체가 없습니다.

Modbus 는 전달 방식에 따라 세 가지 모습으로 나타납니다.

Modbus-RTU — 시리얼 선(주로 RS-485 라는 두 가닥 전선)으로 바이너리를 주고받습니다. 공장에서 가장 흔한 형태입니다. 한 선에 여러 슬레이브를 매달고 슬레이브 ID 로 구분합니다. Modbus-ASCII — 같은 내용을 사람이 읽을 수 있는 문자로 보냅니다. 느려서 거의 안 씁니다. Modbus/TCP — 일반 이더넷(랜선) 위에 얹은 것입니다. 502 번 포트를 씁니다. 이 교본이 다루는 시스템이 쓰는 방식입니다.

요청 한 건의 알맹이는 딱 두 조각입니다. 함수 코드(function code) 한 바이트와, 그 뒤에 붙는 주소·개수입니다. 함수 코드가 "무엇을 어떻게 할지"를 정합니다.

함수 코드	이름	하는 일	한 요청의 최대 개수
01	Read Coils	비트 여러 개를 읽는다	2000 개
02	Read Discrete Inputs	읽기 전용 비트를 읽는다	2000 개
03	Read Holding Registers	16 비트 값 여러 개를 읽는다	125 칸
04	Read Input Registers	읽기 전용 16 비트 값을 읽는다	125 칸
05	Write Single Coil	비트 하나를 켜거나 끈다	1 개
06	Write Single Register	16 비트 값 하나를 쓴다	1 칸
15 (0x0F)	Write Multiple Coils	비트 여러 개를 한꺼번에 쓴다	1968 개
16 (0x10)	Write Multiple Registers	16 비트 값 여러 개를 한꺼번에 쓴다	123 칸

오른쪽 열을 그냥 지나치지 마십시오. 이 상한은 명세가 정한 것이고, 실제 장비를 붙이면 거의 반드시 걸립니다. 계측점이 200 점인 장비에서 홀딩 레지스터 200 칸을 한 번에 읽으려 하면 실패합니다. 125 칸씩 두 번 나눠 읽어야 합니다. 상한이 125·123 처럼 애매한 것은 한 응답이 256 바이트짜리 프레임 안 에 들어가야 하기 때문입니다.

슬레이브 ID 도 규칙이 있습니다. 쓸 수 있는 번호는 1~247 입니다. 0 번은 브로드캐스트 예약이라 쓰면 안 됩니다. 0 번으로 보낸 쓰기 명령은 선에 매달린 모든 슬레이브가 동시에 수행하고 아무도 응답 하지 않습니다. 248~255 는 명세가 남겨 둔 번호입니다.

4 가지 메모리 영역

Modbus 슬레이브의 속은 네 개의 서로 다른 값 보관함으로 나뉘어 있습니다. 파일 네 개가 나란히 있다고 생각하면 됩니다. 나누는 기준은 두 가지뿐입니다. 한 칸이 1 비트인가 16 비트인가, 그리고 마스터가 쓸 수 있는가 읽기만 되는가입니다. 2×2, 그래서 넷입니다.



원문 도해 · Modbus 네 가지 메모리 영역 (원문 43 쪽)

이름의 유래를 알면 외우기 쉽습니다. Coil(코일)은 릴레이의 전자석 코일에서 왔습니다. 전류를 흘리면 붙고 끊으면 떨어지는, 켜고 끄는 출력입니다. Discrete Input(디스크리트 입력)의 discrete 는 "불연속", 즉 켜짐/꺼짐 둘 중 하나라는 뜻입니다. Register(레지스터)는 16 비트짜리 값 한 칸입니다.

Input Register 와 Holding Register 의 이름은 솔직히 뜻을 알려 주지 않습니다. 원래 의도는 "Input Register = 바깥에서 들어온 측정값(읽기만)", "Holding Register = 장비가 들고 있는 설정값(읽고 쓰기)"이었습니다. 그런데 현장에서는 이 구분이 거의 지켜지지 않습니다. 대부분의 장비가 계측값까지 Holding Register 에 올려 둡니다. 읽기 전용 영역을 따로 관리할 이유가 실무적으로 별로 없고, 마스터 쪽에서도 함수 코드 하나(03)로 다 읽는 편이 편하기 때문입니다.

주의. "Input Register 니까 센서값이 여기 있겠지"라고 넘겨짚지 마십시오. 그 장비의 레지스터 맵 (register map) 문서를 읽어야만 알 수 있습니다. 레지스터 맵은 제조사가 주는, "몇 번 주소에 무엇이 어떤 타입으로 들어 있는지" 적힌 표입니다. 이 표 없이 Modbus 장비를 붙이는 것은 불가능합니다.

예외 응답(Exception Response)

Modbus 가 아무 말도 못 하는 병어리는 아닙니다. 거절할 줄은 압니다. 이것을 모르면 실제 장비를 붙였을 때 돌아온 응답을 해석하지 못하고, 다음 절에서 다룰 "조용히 틀리는" 사고의 범위도 잘못 알게 됩니다.

슬레이브가 요청을 처리할 수 없으면 정상 응답 대신 예외 응답을 돌려줍니다. 모양은 아주 단순합니다. 응답의 함수 코드 자리에 요청받은 함수 코드에 0x80(십진수 128)을 더한 값을 넣고, 그 뒤에 예외 코드 한 바이트를 붙입니다. 홀딩 레지스터 읽기 (03)가 거절당하면 0x83 이 돌아옵니다. 다시 말해 함수 코드 자리의 최상위 비트가 1로 서 있으면 "이건 답이 아니라 거절이다"라는 신호입니다.

예외 코드	이름	실제로는 이런 뜻
01	Illegal Function	이 장비는 그 함수 코드를 구현하지 않았다. 코일이 없는 장비에 05를 쓰면 이것이 온다
02	Illegal Data Address	요청한 주소가 내 맵 바깥이다. 가장 많이 보게 되는 코드
03	Illegal Data Value	주소는 맞는데 값이나 개수가 허용 범위를 벗어났다. 한 번에 200칸을 읽으려 할 때도 이것이 온다
04	Server Device Failure	내부에서 복구 못 할 오류가 났다
05 · 06	Acknowledge · Server Busy	받긴 했는데 오래 걸린다 / 지금 바쁘니 나중에 다시 물어라

예외 응답은 프로토콜이 표현할 수 있는 몇 안 되는 "정상이 아님"의 신호입니다. 다만 그 대상은 요청 자체의 잘못이지 값의 잘못이 아닙니다. 센서가 죽어서 값이 없다는 사실은 여기에 실을 수 없습니다. 그것을 어떻게 전달할지는 이 장 뒤쪽에서 다시 다룹니다.

0-based 와 1-based, 조용히 틀리는 함정

여기가 Modbus 입문자가 가장 많이, 그리고 가장 오래 해매는 지점입니다.

Modbus 에는 주소를 세는 방식이 두 가지 있습니다.

전선 위의 주소(protocol address, PDU address) — 실제 통신 패킷에 담기는 숫자입니다. 0 부터 셉니다. 첫 홀딩 레지스터가 0 입니다. 사람이 쓰는 주소(data model address, 전통적 표기) — 영역 번호를 앞에 붙이고 1 부터 셉니다. 첫 코일은 00001, 첫 디스크리트 입력은 10001, 첫 입력 레지스터는 30001, 첫 홀딩 레지스터는 40001 입니다.

즉 문서에 40001 이라고 적힌 값은 전선 위에서 0 번입니다. 40101 은 100 번입니다. 영역 접두어를 떼고 1 을 빼야 합니다.

왜 이게 위험한가. 앞 절에서 본 대로 존재하지 않는 주소를 물으면 예외 코드 02 가 돌아옵니다. 그래서 주소를 크게 틀리면 바로 들립니다. 무서운 것은 한 칸 밀린 경우입니다. 한 칸 옆은 대개 그 장비가 실제로 가진 유효한 주소이므로, 슬레이브는 거절할 이유가 없습니다. 그 자리에 있는 16 비트를 아무 일도 없었다는 듯이 돌려줍니다. 그 값이 온도인지 압력인지, 의미 있는 값인지 빈 칸인지는 판단하지 않습니다.

파일 경로를 틀리면 "파일 없음" 오류가 납니다. 레지스터 주소를 맵 안에서 틀리면 다른 숫자가 조용히 옵니다. 그래서 증상이 이렇게 나타납니다. 시스템은 멀쩡히 돌아가고, 대시보드에는 그럴듯한 숫자가 찍히고, 아무도 이상을 눈치채지 못합니다. 몇 주 뒤 "펌프 속도가 왜 계속 45%로 보이지?" 하는 의문이 생기고, 그제서야 옆 레지스터를 읽고 있었다는 사실이 드러납니다.

16 비트 한 칸에 소수를 담는 법, 그리고 엔디안

레지스터 한 칸은 16 비트입니다. 담을 수 있는 것은 0~65535(부호 없음) 또는 -32768~32767(부호 있음) 범위의 정수뿐입니다. 소수점이 없습니다.

그런데 온도는 72.43 도입니다. 어떻게 할까요? 두 가지 관행이 있습니다.

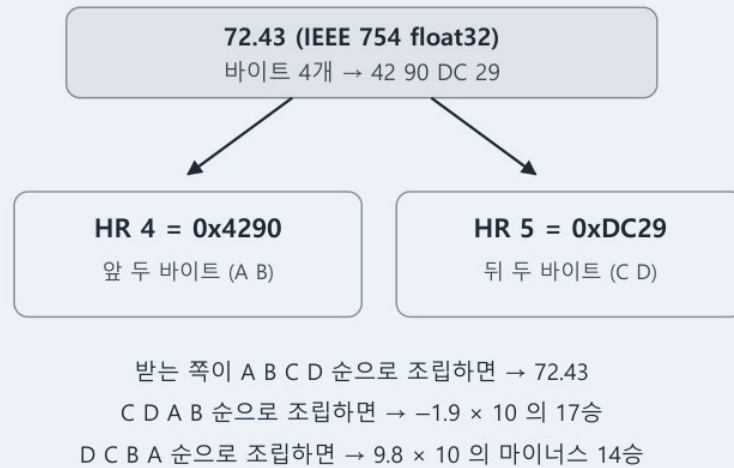
첫째, 정수로 곱해 보내기. 72.43 을 10 배 해서 724 로 보내고, 받는 쪽이 10 으로 나눕니다. 소수 한 자리면 충분한 설정치에 흔히 씁니다. 간단하지만 배율을 양쪽이 미리 합의해야 하고, 그 합의가 어디에도 안 적혀 있으면 나중에 아무도 모릅니다.

둘째, 레지스터 두 칸을 이어 붙이기. 컴퓨터가 소수를 표현하는 표준 방식인 IEEE 754 float32(32 비트 부동소수점)를 쓰려면 32 비트가 필요합니다. 레지스터 두 칸(16+16)을 하나로 묶어 쓰는 것입니다. 오늘날 계속값은 대부분 이 방식입니다.

여기서 새 함정이 생깁니다. 32 비트는 바이트 네 개이고, 이것을 두 칸에 나눠 담아야 합니다. 명세가 정한 것과 정하지 않은 것을 정확히 구분해야 합니다.

명세가 정한 것 — 한 칸 안의 바이트 순서. Modbus는 레지스터 한 칸을 전송할 때 상위 바이트를 먼저 보내도록 규정합니다. 이것을 빅엔디언(big-endian, 큰 쪽이 앞)이라고 합니다. 사람이 숫자를 읽는 순서와 같습니다. 명세가 비워둔 것 — 두 칸 사이의 순서. 32 비트를 두 칸에 나눌 때 앞쪽 절반을 낮은 번호 레지스터에 넣을지 높은 번호에 넣을지는 아무도 정해 주지 않았습니니다. 이것을 워드 순서(word order)라고 하고, 제조사마다 다릅니다.

엔디언(endianness)은 원래 "여러 바이트로 된 값을 어떤 순서로 늘어놓는가"를 가리키는 말입니다. Modbus 현장에서는 이 말이 바이트 순서와 워드 순서를 뭉뚱그려 쓰이는데, 둘의 성격이 다르다는 점은 기억해 두십시오.



같은 전선 위 바이트인데 조립 순서만 다르면 전혀 다른 값이 나옵니다. 위 수치는 IEEE 754 규칙대로 직접 계산한 값입니다.

원문 도해 · float32 두 워드와 조립 순서 (원문 46 쪽)

바이트 네 개를 A·B·C·D 라고 부를 때, 현장에서 실제로 마주치는 조합은 네 가지입니다.

부르는 이름	순서	명세와의 관계	설명
Big-endian (ABCD)	A B / C D	명세 준수	사람이 읽는 순서 그대로
Word-swap (CDAB)	C D / A B	명세가 안 정한 영역	칸 안은 규정대로인데 두 칸의 순서만 뒤집음. 가장 흔한 사고 원인
Byte-swap (BADC)	B A / D C	명세 위반	칸 안에서 바이트를 뒤집음. 규정을 어긴 구현
Little-endian (DCBA)	D C / B A	명세 위반	완전 역순

진단법은 단순합니다. 온도가 $-3.8e-29$ 나 $-1.9e17$ 처럼 물리적으로 말이 안 되는 값으로 나오면 거의 확실히 조립 순서 문제입니다. 값이 아주 틀린 것은 오히려 다행입니다. 자릿수가 비슷하게 그럴듯한 값이 나올 때가 훨씬 무섭습니다.

AR-100의 Modbus 접속과 값 형식

plant-simulator는 컨테이너 내부 502 번 포트, 호스트 PORT_SIM_MODBUS=27002로 Modbus/TCP를 노출합니다. edgex/devices/ar100.devices.yaml의 UnitID는 1입니다. 계측 레지스터는 1장 전체 표처럼 HR 0·2·4···22에 float32 두 워드씩 배치합니다. simulator/sim.py의 f32_regs()는 struct.pack(">f")와 unpack(">HH")로 ABCD 순서를 만듭니다. 제어는 코일과 정수형 설정치로 나뉩니다. SP(setpoint)는 측정값이 아니라 사람이 지정하는 목표값입니다.

주소	영역	권한	내용
Coil 0	Coil	R/W	P-101 공급펌프 기동
Coil 1	Coil	R/W	M-101 교반기 기동
Coil 2	Coil	R/W	HX-101 재킷히터 Enable

주소	영역	권한	내용
Coil 10	Coil	R/W	알람 Acknowledge
Coil 20	Coil	R	고압 인터록 발동 상태 (읽기 전용 약속)
HR 100	Holding	R/W	P-101 속도 설정치 [%], 범위 0~100, 기본 60
HR 101	Holding	R/W	CV-101 밸브 개도 설정치 [%], 범위 0~100, 기본 45
HR 102	Holding	R/W	반응기 온도 설정치 [$^{\circ}\text{C} \times 10$], 범위 200~1000, 기본 720
HR 200	Holding	R	스캔 시퀀스 카운터 (부호 없는 32 비트, 2 칸)

HR 102 가 앞에서 설명한 "10 배 정수" 관행입니다. 온도 설정치 72.0°C 를 720 이라는 정수로 씁니다. 그리고 이 배율 합의는 문서에 묻히지 않고 EdgeX 프로파일에 명시되어 있습니다. `edgex/profiles/ar100.profile.yaml` 의 TempSpX10 항목입니다.

```
- name: TempSpX10
  attributes:
    primaryTable: HOLDING_REGISTERS
    startingAddress: 102
  properties:
    valueType: Int16
    readWrite: RW
    minimum: 200
    maximum: 1000
    scale: 0.1
```

valueType 에 쓰이는 이름도 풀어 두겠습니다. Int16 은 부호 있는 16 비트 정수(-32768~32767), Uint32 는 앞의 U 가 unsigned 를 뜻해 부호 없는 32 비트 정수(0~4294967295), Float32 는 32 비트 실수입니다. 시퀀스 카운터가 Uint32 인 이유는 계속 증가하기만 하는 값이라 음수가 필요 없고, 16 비트로는 65535 에서 금방 되돌아오기 때문입니다.

scale: 0.1 한 줄이 EdgeX 가 720 을 읽어 72.0 으로 되돌리게 합니다. 합의가 코드가 아니라 설정에 적혀 있다는 점이 중요합니다.

주의 — readonly 는 프로젝트의 약속이지 Modbus 의 강제가 아닙니다

Coil 20 의 권한 칸에 R 이라고 적혀 있지만, Modbus 자체는 이것을 막아 주지 않습니다. 코일은 정의상 마스터가 쓸 수 있는 영역이고, 누가 함수 코드 05 로 Coil 20 에 1 을 쓰면 슬레이브는 그대로 받습니다. 예외 응답도 오지 않습니다.

plant.yaml 의 readonly: true 는 두 가지 일만 합니다. 첫째, `edgex/gen_profile.py` 가 이 표시를 보고 EdgeX 프로파일에서 그 리소스를 readWrite: R 로 만들어, EdgeX 를 통한 쓰기를 막습니다. 둘째, 시뮬레이터가 매 스캔마다 인터록의 실제 상태를 이 코일에 다시 덮어씁니다(`sim.py` 의 `_write_readings`). 그래서 누가 몰래 써 넣어도 1 초 안에 원래 값으로 돌아옵니다.

정리하면, 읽기 전용은 프로토콜이 아니라 상위 계층과 운영 규칙이 지키는 것입니다. "Modbus 가 권한을 관리해 준다"고 배우면 이 장이 내내 경고하는 사고를 그대로 맞게 됩니다.

주의 — 이 시스템 안에 0-based 와 1-based 가 동시에 존재합니다. 같은 레지스터가 설정 파일마다 다른 번호로 적혀 있고, 둘 다 맞습니다.

EdgeX 의 device-modbus 는 0-based 를 받습니다. 그래서 프로파일에 startingAddress: 100 으로 적습니다. FUXA(관제 화면)의 Modbus 드라이버는 1-based 를 받습니다. 내부에서 `parseInt(address) - 1` 을 하기 때문입니다. 그래서 `fuxa/build_project.py` 의 생성 함수가 일부러 1 을 더합니다.

```
def mb(tag_id, name, area, addr0, dtype, desc):
    """addr0 는 0-based 실제 Modbus 주소. FUXA 는 1-based 이므로 +1 해서 넣는다."""
    mb_tags[tag_id] = { ..., "memaddress": area, "address": str(addr0 + 1), ... }
```

FUXA 는 영역까지 문자열로 구분합니다. 코일은 "000000", 디스크리트 입력은 "100000", 입력 레지스터는 "300000", 홀딩 레지스터는 "400000" 이라고 리터럴로 적어야 합니다. 리터럴이란 계산 해서 나오는 값이 아니라 글자 그대로 그렇게 적힌 값을 뜻합니다. 코일에 "0" 이라고 쓰면 Cannot read properties of undefined (reading 'Items') 로 디바이스 적재가 실패합니다. 이 사실은 문서에 없어서 FUXA 소스를 읽어야 확인했습니다(docs/GOTCHAS.md 18 항).

여기서 실제로 위험한 조합을 보십시오. HR 100 은 펌프 속도(기본 60), HR 101 은 밸브 개도(기본 45)입니다. 둘 다 0~100 퍼센트입니다. 1-based 를 0-based 로 착각해 한 칸 밀려 읽으면 펌프 속도 자리에 45 가 들어옵니다. 두 주소

모두 유효한 맵 아니라 예외 응답도 오지 않습니다. 완벽하게 그럴듯한 값입니다. 오류도, 경고도, 이상한 그래프도 없습니다. 이것이 "조용히 틀린 값"의 정확한 모습입니다.

AR-100 · 이 주소 체계가 맞다는 것을 실제로 확인했습니다. Modbus 로 읽은 값과 시뮬레이터가 REST API 로 알려주는 내부 값을 태그별로 맞춰 봤습니다.

```
=== 1. Modbus float32 읽기 vs REST 교차검증 ===
LT-101 modbus= 70.652 rest= 70.652
TT-101 modbus= 52.133 rest= 52.133
... → 전 태그 일치
```

이 표의 숫자를 정상 운전값으로 오해하지 마십시오. 이 시험은 기동 직후에 돌린 것이라 공정이 아직 데워지는 중입니다. 50 분 수렴 후의 정상 운전점은 TT-101 이 70.91 °C, LT-101 이 82.70 %입니다(docs/journal/09-metrics.md § 9.9). 교차검증이 확인하는 것은 값의 크기가 아니라 두 경로가 같은 숫자를 내놓는가이므로, 과도구간의 값으로도 목적은 충분히 달성됩니다.

주소·타입·조립 순서 중 하나라도 틀렸다면 이 표의 두 열이 일치할 수 없습니다. 교차검증은 이렇게 독립된 두 경로로 같은 값을 뽑아 비교하는 것입니다. Modbus 쪽만 봐서는 맞는지 알 방법이 없습니다.

쓰기 방향도 확인했습니다. 코일 0 에 0 을 쓰자 물리량이 반응했습니다. 아래는 앞서 설명한 하한 클램프를 반영한 뒤의 값입니다(docs/journal/02-tier1-edge.md § 2.8).

```
=== Southbound 제어: 펌프 정지 (coil 0 = 0) ===
FT-101 공급유량 8.29 → 0.07 m3/h
IT-101 펌프전류 7.87 → 0.06 A
→ 제어 반영 확인
```

클램프를 넣기 전 같은 시험에서는 유량이 -0.03 m3/h 로 나왔습니다. 음수 유량은 있을 수 없는 값 이므로, 그 자체가 "계측 모델이 덜 만들어졌다"는 신호였습니다.

주의 — Modbus 에는 "값이 없음"을 표현할 방법이 없습니다. 앞에서 본 예외 응답은 요청이 잘못됐다는 신호이지 값이 없다는 신호가 아닙니다. 주소가 맞으면 레지스터는 언제나 16 비트짜리 무언가를 돌려줍니다. 센서가 죽어도, 통신이 끊겨도, 0 이든 직전 값이든 무언가가 옵니다. NULL 이 없습니다.

프로토콜이 표현하지 못하는 의미를 전달하는 방법은 현장에서 셋뿐입니다. 센티널 값(약속된 특수 값), 별도의 품질 레지스터(옆 칸에 "이 값 믿어도 됨/안 됨"을 담는 방식), 그리고 앞서 본 예외 응답 입니다. 예외 응답은 요청 단위라 "12 개 중 3 번 태그만 고장"을 말할 수 없고, 품질 레지스터는 레지스터 맵을 두 배로 늘립니다.

이 시스템은 센티널을 씁니다. simulator/plant.py 의 BAD_QUALITY = -999999.0 입니다. 물리적으로 절대 나올 수 없는 값을 골라 "이건 값이 아니라 표식"으로 쓰는 것입니다.

센티널은 임시방편이지 해법이 아닙니다. 이 표식을 진짜 공백으로 바꿔 주는 일은 상위 계층 (Telegraf 브리지)이 맡습니다. 잠시 뒤 원문 2·2 절에서 그 코드를 봅니다.

같은 자리의 다른 도구들 OPC-UA(OPC Unified Architecture) — 2006 년에 1.0 명세가 공개되고 2010 년대에 IEC 62541 로 국제 표준 이 된 규약입니다. 주소 대신 이름과 계층 구조로 값을 가리키고, 타입·단위·타임스탬프·품질을 값과 함께 실어 보냅니다. 인증과 암호화가 명세에 들어 있습니다. Modbus 의 빈칸을 전부 메운 대신, 구현이 훨씬 무겁고 장비 가격도 올라갑니다. 이 교본이 만든 시스템은 OPC-UA 를 쓰지 않습니다. 구현 경로는 Modbus 에서 EdgeX 로 가는 하나뿐이고, OPC-UA 는 비교 대상으로만 등장합니다(docs/journal/11-teaching-guide.md § 11.6 의 "다루지 않는 것" 표). EtherNet/IP — 로크웰(Allen-Bradley) 계열 PLC 의 사실상 표준. 이더넷 위에서 돌지만 Modbus/TCP 와는 완전히 다른 규약입니다. PROFINET / PROFIBUS — 지멘스 계열. 유럽 공장에서 지배적입니다. BACnet — 빌딩 자동화(공조·조명·엘리베이터) 전용. 공장보다는 건물에서 씁니다. CAN / CANopen — 차량과 이동 장비. 짧은 메시지를 아주 빠르게 주고받는 데 특화. MQTT Sparkplug B — MQTT 위에 산업용 의미 체계를 얹은 규격. 프로토콜이라기보다 "MQTT 를 이렇게 쓰자"는 약속입니다.

설계 보고서가 지적하는 현실이 바로 이것입니다. "산업 현장은 센서 제조사 및 설비 연식에 따라 Modbus-RTU, Modbus-TCP, OPC-UA, BACnet, CAN 등 이종 프로토콜이 난립해 있다." 한 공장 안에서 이 다섯 가지가 동시에 돌고 있는 것이 정상입니다.

Modbus 와 OPC-UA 는 본질이 어떻게 다른가

둘을 "구형 대 신형"으로 이해하면 절반만 맞습니다. 정확한 차이는 의미를 누가 들고 있는가입니다.

Modbus 에서 "레지스터 4 번이 반응기 온도이고 단위는 섭씨"라는 지식은 전선 위에 없습니다. 그 지식은 종이 문서나 설정 파일에 따로 있고, 사람이 양쪽을 맞춰 줘야 합니다. 틀려도 아무도 모릅니다.

OPC-UA 에서는 그 지식이 값과 함께 전선 위를 흘러갑니다. 클라이언트가 서버에 접속해 "내가 가진 것들을 보여 달라"고 하면 이름·타입·단위가 붙은 목록이 돌아옵니다. 이것을 디스커버리(discovery, 자동 발견)라고 합니다.

비교 항목	Modbus	OPC-UA
값을 가리키는 방법	숫자 주소 (레지스터 번호)	이름과 계층 경로 (노드)
타입 정보	없음 — 사람이 문서로 합의	값에 포함
단위	없음	값에 포함
타임스탬프	없음 — 읽은 쪽이 찍음	서버가 찍은 시각이 포함
품질 표시	없음 — 예외 응답은 요청 단위 오류만	Good / Bad / Uncertain 포함
보안	없음 — 인증도 암호화도 없음	인증서 기반 인증·암호화가 명세에 포함
값 전달 방식	마스터가 계속 물어봄 (폴링)	구독하면 변할 때 보내 줌 (subscription)
구현 부담	아주 가벼움 — 8 비트 마이크로컨트롤러도 가능	무거움 — 스택 라이브러리가 필요
장비 보급	압도적. 40 년치 설비가 전부 지원	2010 년대 이후 신규 장비 중심

그래서 결론은 "OPC-UA 로 바꾸자"가 아닙니다. 공장에는 1995 년에 설치한 Modbus 전용 계량기와 2024 년에 들인 OPC-UA 로봇이 같이 있습니다. 둘 다 받아서 하나로 만들어 주는 계층이 필요하다 — 이것이 다음 절의 주제입니다.

2.2 정규화 — 숫자에 의미·단위·시각·출처를 붙이는 일

정규화(normalization)의 정확한 뜻

먼저 혼동부터 정리하겠습니다. 이 단어는 분야마다 전혀 다른 뜻으로 쓰입니다.

데이터베이스의 정규화 — 표를 쪼개 중복을 없애는 설계 기법(1NF, 2NF...). 이 장의 정규화가 아닙니다. 통계·머신러닝의 정규화 — 값을 0~1 범위로 줄이는 스케일링. 이것도 아닙니다. IIoT 에서의 정규화 — 서로 다른 프로토콜에서 들어온 값들을 하나의 공통 형식과 공통 의미 체계 로 맞추는 것. 이것입니다.

설계 보고서는 이렇게 표현합니다. "IoT 미들웨어(예: EdgeX Foundry 의 Device Service)는 이러한 산업용 필드 버스 신호를 단일화된 JSON 또는 Protocol Buffers 형태의 시맨틱 데이터 모델로 정규화하는 작업을 수행한다."

시맨틱(semantic)은 "의미의"라는 뜻입니다. 시맨틱 데이터 모델 = 숫자에 의미가 붙어 있는 형식. Protocol Buffers 는 구글이 만든 이진 데이터 형식으로, JSON 보다 작고 빠르지만 사람이 눈으로 읽을 수 없습니다.

구체적으로 무엇을 붙이는 것인지가 핵심입니다. 헛벗은 숫자 72.43 에 최소한 이 다섯 가지를 붙여야 기록이 됩니다.

붙일 것	왜 필요한가	없으면
의미 — 어떤 측정점인가	이 숫자가 온도인지 압력인지	그래프를 그릴 수도 없다
단위 — 무슨 단위인가	섭씨인지 화씨인지, bar 인지 kPa 인지	임계치 비교가 틀린다
시각 — 언제 잤 값인가	시계열은 시각이 없으면 존재하지 않는다	순서도, 추세도, 상관도 계산 불가
출처 — 어느 공장 어느 설비 인가	같은 태그명이 라인마다 있다	다른 라인 값과 섞인다
품질 — 실측인가 추정인가	실제로 잤 값과 보간한 값의 구분	사고 조사 때 무엇이 사실인지 모른다

보간은 없는 구간을 계산으로 채우는 일입니다. 실제 관측과 추정은 quality 로 구분하고 원본을 덮어쓰지 않습니다. 계산 방법과 원본·정제본 분리는 2 권 5 장에서 설명합니다.

왜 이 일을 아래쪽에서 해야 하는가. 정규화를 안 하면 그 부담이 위로 전부 전가됩니다. Kafka 는 바이트를 나를 뿐이니 상관없지만, Flink 가 "이 메시지는 Modbus 에서 온 거니까 4 번 레지스터는 온도로 해석하고, 저건 OPC-UA 에서 온 거니까 노드 경로를 파싱하고..."를 해야 합니다. 저장소도, 대시보드 도, 머신러닝 코드도 전부 프로토콜별 분기를 갖게 됩니다. 장비 한 대를 새로 들일 때마다 상위 계층 전체를 고쳐야 합니다.

반대로 진입점에서 한 번 정규화해 두면, 위쪽 모든 계층은 "이 시스템에는 한 가지 형식의 메시지만 흐른다"고 가정하고 만들어집니다. 설계 보고서가 말하는 관심사 분리(Separation of Concerns)가 이것입니다.

AR-100에서는, 정규화는 두 단계에 걸쳐 일어납니다.

1 단계 — EdgeX가 Modbus 바이너리를 Event JSON으로, device-modbus 서비스가 프로파일 YAML을 보고 레지스터를 해석해 이런 것을 내놓습니다. 아래는 실제 시스템이 산출한 값입니다.

```
{
  "apiVersion": "v3",
  "deviceName": "reactor-line-01",
  "profileName": "AR100-Reactor-Line",
  "sourceName": "AllSensors",
  "origin": 1789545856852575500,
  "readings": [
    { "resourceName": "CT-101", "valueType": "Float32",
      "units": "mS/cm", "value": "3.1290388e+00" },
    { "resourceName": "pH-101", "valueType": "Float32",
      "units": "pH", "value": "7.6120467e+00" },
    ...
  ]
}
```

비교해 보십시오. 전선 위에는 0x4048 0x4A5C 같은 16비트 덩어리 두 개뿐이었습니다. 여기에는 태그명(CT-101), 타입(Float32), 단위(mS/cm), 설비명(reactor-line-01), 나노초 시각(origin)이 붙어 있습니다.

2 단계 — Telegraf 브리지가 전 구간 공통 계약으로, EdgeX Event는 EdgeX의 형식이지만 이 시스템의 형식이 아닙니다. Kafka에 들어가는 순간의 형식을 데이터 계약이라 부르고, 이렇게 고정했습니다.

```
{
  "ts": 1789546028493305900,
  "site": "AR-100",
  "device": "reactor-line-01",
  "tag": "FT-102",
  "value": 6.286102,
  "quality": "GOOD"
}
```

계약의 경계 · seq와 단위

원문 36쪽의 설계 스키마에는 seq가 있지만, 이 EdgeX 경로의 최종 JSON과 Reading.java에는 없습니다. 시뮬레이터는 HR 200에 Uint32 순번을 만들고 lite 경로의 MQTT 발행에는 seq를 넣습니다. 다만 원문 106쪽의 bridge-lite.conf 최종 변환식도 여섯 필드만 내보내므로, “lite 경로는 Kafka까지 seq를 유지한다”는 36쪽 표와 일치하지 않습니다. 첨부 원문만으로 Kafka 도착 레코드의 seq 보존 여부를 확정할 수 없어 두 서술의 차이를 남깁니다. [원문 36, 58, 106쪽]

단위도 EdgeX Event의 units에는 들어 있지만 위 여섯 필드 계약에는 별도 항목이 없습니다. 태그 정의와 규격 테이블의 단위를 함께 읽어야 합니다. “의미를 보존해야 한다”는 설계 원칙과 “모든 정보가 매 메시지 필드로 실린다”는 주장은 다릅니다. [원문 25, 58쪽]

Telegraf는 데이터를 어디선가 받아 모양을 바꿔 다른 곳으로 보내는 도구입니다. 이 변환은 telegraf/bridge-edgex.conf라는 설정 파일 하나로 이뤄집니다. 프로그램을 짜지 않았다는 뜻입니다. 설정 파일의 문법은 TOML(Tom's Obvious Minimal Language)이라는 형식으로, 키 = 값과 대괄호로 묶은 절로 이루어진 읽기 쉬운 설정용 문법입니다. 설정은 네 덩어리입니다.

json_v2 — Telegraf의 JSON 파서 이름입니다. 파서란 글자 뭉치를 읽어 구조로 만들어 주는 부품입니다. 여기서는 readings 배열을 리딩 하나씩으로 펼치고 origin을 나노초 시각으로 삼도록 지정합니다. processors.rename — 이름표를 바꾸는 부품입니다. resourceName을 tag로, deviceName을 device로 고쳐 계약에 맞춥니다.

processors.starlark — Starlark는 파이썬을 아주 얇은 작은 스크립트 언어입니다(구글이 빌드 설정용으로 만들었습니다). 설정 파일 안에 짧은 함수를 적어 넣을 수 있어서, 여기서 센서가 아닌 리소스(SEQ, 제어 코일·설정치)를 걸러내고 site와 quality를 붙입니다. json_transformation — 내보낼 JSON의 모양을 계약대로 다시 그리는 규칙입니다.

앞 절에서 예고한 센티널 처리가 바로 세 번째 부품 안에 있습니다.

```

BAD_QUALITY = -999998.0
def apply(metric):
    tag = metric.tags.get("tag")
    if tag == None or tag not in SENSOR_TAGS:
        return None # 제어 태그·SEQ 는 텔레메트리에서 제외
    v = metric.fields.get("value")
    if v == None or v <= BAD_QUALITY:
        return None # 통신 불량 → 발행하지 않음
    metric.tags["site"] = "AR-100"
    metric.tags["quality"] = "GOOD"
    return metric

```

return None 은 "이 값은 내보내지 말라"는 뜻입니다. 여기서 버려야 Kafka 위에 진짜 공백이 생기고, 5 장의 보간이 의미를 갖습니다. 기준선을 시뮬레이터의 -999999.0 보다 살짝 위인 -999998.0 으로 잡아 부등호 비교로 걸러 냅니다. 한 가지 함정이 있었습니다. EdgeX 는 값을 문자열로 씁니다("3.1290388e+00"). 숫자로 강제 변환 해야 합니다.

```

[inputs.mqtt_consumer.json_v2.object.fields]
value = "float" # EdgeX 는 값을 문자열로 씁니다 → 강제 변환

```

AR-100 · 이벤트는 1 초에 1 건인데 Kafka 메시지는 12 건입니다. 이 두 숫자가 다른 이유를 짚어야 뒤 장이 이어집니다.

EdgeX 는 12 개 계속값을 한 이벤트로 묶어 내보냅니다. 같은 스캔에서 읽은 12 점이 흩어지지 않게 하려는 것입니다. 그런데 Telegraf 의 json_v2 파서가 readings 배열을 펼치는 순간, 그 한 이벤트 는 태그별 12 개 메시지로 갈라집니다. 그래서 실측이 이렇게 나옵니다(docs/journal/09-metrics.md § 9.3).

```

센서 유입      12 건/초 (Kafka 오프셋 차분, 252 건/20 초)
EdgeX autoEvent 1 이벤트/초 (12 리딩 묶음)

```

갈라진 12 개 메시지는 각각 Kafka 로 들어갑니다. Kafka 는 같은 종류의 메시지를 모아 두는 통을 토픽이라 부르고, 한 토픽을 여러 줄로 쪼개 놓은 것을 파티션이라 부릅니다(자세한 것은 4 장에서 다룹니다). 12 개 메시지가 어느 줄로 갈지는 tag 값이 정합니다.

```

[[outputs.kafka]]
topic = "sensor.telemetry.raw"
routing_tag = "tag" # 파티션 키 = 태그 → 태그 단위 순서 보장

```

그래서 TT-101 끼리의 순서는 보장되지만 TT-101 과 PT-101 사이의 순서는 보장되지 않습니다. 태그별로 유일하게 식별되고 태그별로 줄을 서는 구조입니다. 이 성질이 4 장의 Kafka 파티션과 5 장의 시간 처리에서 다시 문제가 됩니다.

같은 자리의 다른 도구들 — 정규화를 어디서 할 것인가 에지 미들웨어에서(EdgeX 방식) — 발생지에 가장 가깝습니다. 프로토콜 지식이 현장에만 머물고 위로 새지 않습니다. 대신 현장 컴퓨터에 컨테이너가 늘고 관리 대상이 늘어납니다. 브로커의 규칙 엔진에서(EMQX·ThingsBoard 방식) — 메시지가 지나는 길목에서 바꿉니다. 컴포넌트가 줄어 가볍지만, Modbus 같은 필드버스는 브로커까지 못 옵니다. 이미 MQTT 로 올라온 메시지만 손볼 수 있습니다. 스트림 처리 계층에서(Flink 방식) — 유연성이 가장 큼니다. 그러나 프로토콜별 분기가 중앙 코드로 올라 와, 장비를 바꿀 때마다 Flink 잡을 고쳐야 합니다. 가장 피해야 할 배치입니다. 애플리케이션에서 — 대시보드나 분석 코드가 직접 해석. 소비자가 늘어날수록 같은 해석 로직이 복제되고 점점 어긋납니다.

이 시스템은 둘을 섞었습니다. 프로토콜 해석(Modbus → 의미 있는 JSON)은 EdgeX 가, 공통 계약으로의 마지막 맞춤은 Telegraf 가 합니다. 그 덕에 EdgeX 를 건너뛰는 우회 경로(make lite)를 써도 Kafka 진입 시점의 형식이 완전히 같습니다. 상위 계층은 어느 경로로 들어온 값인지 알 필요가 없습니다.

2·3 IoT 미들웨어가 하는 일 다섯 가지 간호사를 "숫자를 복도로 나르는 사람"이라고 부르면 그 일을 완전히 오해한 것입니다. 간호사는 차트를 적고, 누가 이 환자를 볼 권한이 있는지 확인하고, 이 환자의 과거 기록과 알레르기를 알고, 전산이 멈추면 종이에 적어 뒀다가 복구되면 순서대로 입력하고, 의사의 처방을 환자에게 거꾸로 전달합니다. IoT 미들웨어가 하는 일이 정확히 이 다섯입니다.

왜 미들웨어라는 계층이 생겼는가

설계 보고서의 문장을 그대로 인용하겠습니다. "IoT 미들웨어는 단순한 패킷 포워더가 아니며, 물리적 산업 필드 계층과 IT 엔터프라이즈 시스템 간의 구조적 불일치를 해소하는 추상화 계층으로 작동한다."

패킷 포워더는 받은 것을 그대로 옆으로 넘기는 중계기입니다. 만약 미들웨어가 그것뿐이라면 없애는 편이 낫습니다. 컴포넌트 하나가 줄고 지연도 줄기 때문입니다. 설계 보고서가 굳이 "아니다"라고 못 박은 이유가 여기 있습니다.

구조적 불일치가 무엇인지 구체적으로 보겠습니다.

항목	필드 계층(공장 바닥)	IT 계층(서버실)
통신 모델	마스터가 물어봐야 답함 (폴링)	발생하면 알아서 보냄 (이벤트)
데이터 형식	바이너리 레지스터, 의미 없음	JSON, 스키마와 의미가 있음
주소 체계	숫자 번지	이름·경로·토픽
네트워크	끊기고 느리고 좁음	안정적이고 넓음
보안 모델	사실상 없음 (물리 격리에 의존)	인증서·토큰이 전제
장비 수명	20~30 년	3~5 년

이 두 세계를 직접 연결하면 불일치가 양쪽으로 번집니다. 미들웨어는 둘 사이에 서서 양쪽 말을 다 하는 계층입니다. 다섯 가지 일을 합니다.

다섯 가지 일

① 프로토콜 정규화. 원문 2·2 절에서 다룬 그것입니다. Modbus·OPC-UA·BACnet·CAN 을 전부 받아 하나의 형식으로 내놓습니다. 새 프로토콜은 디바이스 서비스(device service)를 하나 더 붙이는 것으로 끝나고, 상위 계층은 그 사실조차 모릅니다.

② 디바이스 등록·인증. 어떤 장비가 이 시스템에 속하는지, 지금 살아 있는지, 값을 보낼 자격이 있는지를 관리합니다. 장비가 들어오고 나가는 라이프사이클(lifecycle, 등록 → 운용 → 퇴역)을 다룹니다.

여기서 설계 보고서가 언급하는 mTLS 를 풀어야 합니다. TLS(Transport Layer Security)는 웹사이트 주소 앞의 자물쇠 표시를 만드는 그 기술입니다. 통신을 암호화하고, 서버가 자기가 진짜임을 증명합니다. 은행 사이트가 가짜가 아님을 확인해 주는 것이죠. 그런데 이때 손님 쪽은 증명하지 않습니다.

토픽 체계도 장비 등록 정보에서 일관되게 만들 수 있습니다. 브로커는 문자열을 배달 주소로 보지만, 미들웨어는 site·device 같은 장비 정보와 토픽 규칙의 관계를 관리합니다. 원문 94 쪽의 이 추가 설명을 정규화·장비 관리와 함께 읽습니다.

mTLS(mutual TLS, 상호 TLS)는 양쪽 다 증명서를 내미는 방식입니다. 서버도 자기를 증명하고, 접속하는 장비도 "나는 등록된 3 번 게이트웨이다"를 인증서로 증명합니다. 공장에서 이게 필요한 이유는 명백합니다. 아이디·비밀번호는 사람이 입력하지만 센서는 사람이 없고, 수천 대에 같은 비밀번호를 넣으면 한 대만 털려도 전부 털리기 때문입니다.

③ 디지털 트윈 메타데이터. 디지털 트윈(digital twin)은 물리적 설비의 정보를 컴퓨터 안에 똑같이 만들어 둔 것입니다. 여기서 말하는 것은 3D 모형 같은 것이 아니라, 값을 해석하는 데 필요한 부대 정보입니다.

설계 보고서가 예로 든 두 가지가 정확합니다. 캘리브레이션 계수(calibration coefficient, 보정 계수)는 "이 센서는 기준기보다 0.3 도 높게 읽으니 빼야 한다" 같은 개별 보정값입니다. 같은 모델 센서라도 개체마다 다르고, 주기적으로 기준기와 비교해 다시 잡습니다. 물리적 설치 좌표는 "이 온도계는 3 층 반응기 상단, 교반기에서 1.2 미터"입니다. 나중에 "왜 이 센서만 값이 높지?"를 따질 때 결정적입니다.

④ Store-and-Forward. 뒤에서 따로 다룹니다.

⑤ 사우스바운드 제어 경로. 방향을 가리키는 두 단어를 먼저 정리하겠습니다.

노스바운드(northbound, 북쪽으로) — 현장에서 위로 올라가는 방향. 센서 값이 가는 길. 사우스바운드(southbound, 남쪽으로) — 위에서 현장으로 내려가는 방향. 명령이 가는 길.

그림에서 현장을 아래에 그리는 관습에서 나온 말입니다.

설계 보고서는 이 대목을 특히 강하게 씁니다. "결정적으로, 센서나 PLC 레지스터에 특정 제어 값을 주입하는 사우스바운드 양방향 제어 경로를 수립하고 유지하는 주체는 Kafka 가 아닌 IoT 미들웨어다."

왜 Kafka 가 못 하는지는 4 장에서 자세히 보겠지만, 한 문장으로는 이렇습니다. Kafka 는 로그이지 통로가 아닙니다. 메시지를 순서대로 쌓아 두는 것이 Kafka 의 일이고, 누군가에게 "이걸 지금 실행하고 결과를 알려 달라"고 요구하는 일은 Kafka 의 모델에 없습니다. 밸브를 60%로 열라는 명령은 도착했는가, 실행됐는가, 실제 개도가 60 이 됐는가까지 확인해야 하는 대화입니다.

주의 — 지금 돌리는 이 실습본에는 ②의 인증·암호화가 꺼져 있습니다

앞에서 mTLS 를 길게 설명했지만, AR-100 데모 스택은 전 구간 평문입니다. 설정 파일에 그렇게 적혀 있습니다.

```
x-edgex-env: &edgex-env
  EDGEX_SECURITY_SECRET_STORE: "false"      # docker-compose.edgex.yml

listeners.ssl.default { enable = false }    # emqx/emqx.conf - TLS 리스너 꺼짐
authorization { no_match = allow }          # 규칙에 안 걸리면 전부 허용
```

EdgeX의 비밀 저장소(Secret Store)를 끄면 서비스 간 토큰 검증이 사라지고, EMQX는 TLS 리스너를 닫아 둔 채 인증 없이 누구나 붙을 수 있습니다. 교재의 docs/journal/11-teaching-guide.md §11.6도 "보안(TLS, 인증, EdgeX Secret Store) — 데모는 평문. 실 운영 필수"라고 명시합니다.

이유는 교육용이기 때문입니다. 인증서를 발급하고 배포하는 절차가 들어오면 파이프라인의 동작을 보기 전에 인증서 문제부터 풀어야 합니다. 대신 대가를 정확히 알아 두십시오. 이 스택은 격리된 개인 컴퓨터에서만 돌려야 하고, 여기서 배운 구성을 그대로 공장 망에 올리면 안 됩니다.

EdgeX Foundry는 어떻게 생겼는가

EdgeX Foundry는 리눅스 재단(Linux Foundation) 산하 LF Edge 프로젝트의 오픈소스 에지 플랫폼입니다. 설계 보고서의 평가는 "상위 시각화보다는 현장 에지 컴퓨터 내에서의 사우스바운드 프로토콜 통합에 특화되어 있다"입니다.

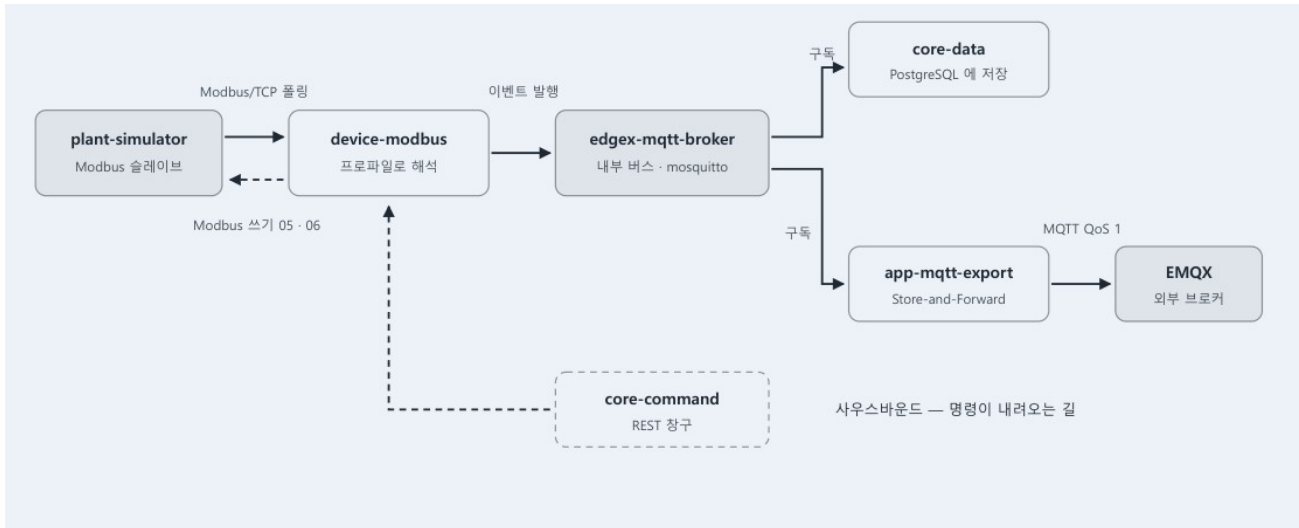
구조는 마이크로서비스입니다. 하나의 큰 프로그램이 아니라, 각자 한 가지 일만 하는 작은 서비스 여러 개가 서로 통신합니다. AR-100이 실제로 띄우는 것은 다음과 같습니다.

층	서비스	하는 일
디바이스 서비스	device-modbus	프로토콜 하나를 담당. 장비와 직접 말하고, 읽은 것을 내부 버스에 발행한다
코어 서비스	core-data	내부 버스를 구독해 이벤트를 저장(영속화)한다
	core-metadata	장비 정의·프로파일을 보관 (디지털 트윈 메타데이터)
	core-command	사우스바운드 명령의 창구. REST로 받아 디바이스 서비스에 넘긴다
	core-keeper	설정과 서비스 등록 정보 보관 (4.0에서 Consul을 대체)
애플리케이션 서비스	app-mqtt-export	내부 버스를 구독해 바깥(MQTT·HTTP·클라우드)으로 내보낸다
받침 구성 요소	mqtt-broker	서비스끼리 주고받는 내부 메시지버스. 이미지는 eclipse-mosquitto
	database	4.0에서는 PostgreSQL. 이벤트와 Store-and-Forward 큐가 여기 앉는다
	common-config-bootstrapper	다른 서비스보다 먼저 한 번 실행되어 공통 설정을 밀어 넣고 끝나는 프로그램. 부트스트래퍼라 부른다

여기서 초보가 반드시 오해하는 지점이 있습니다. 그림으로 그리면 device-modbus → core-data → app-mqtt-export 처럼 일렬로 보이지만, 실제로는 그렇지 않습니다. 디바이스 서비스는 이벤트를 내부 메시지버스에 발행하고, core-data와 app-mqtt-export는 그 버스를 각각 구독합니다. 둘은 나란히 선 소비자이지 앞뒤로 선 중계기가 아닙니다. AR-100의 설정에서도 모든 서비스가 같은 버스 주소를 받습니다.

```
ALL_SERVICES_MESSAGEBUS_HOST: edgex-mqtt-broker # 전 서비스 공통 주입
```

이 차이는 말장난이 아닙니다. 일렬 구조라면 core-data가 멈추는 순간 바깥으로 나가는 데이터도 멈춰야 합니다. 병렬 구조이므로 그렇지 않고, 뒤에 나오는 Store-and-Forward 단독 시험도 이 구조 위에서만 성립합니다.



원문 도해 · EdgeX 내부 버스의 병렬 구독 구조와 명령 경로 (원문 65 쪽)

디바이스 프로파일(device profile)이 이 구조의 심장입니다. "이 모델의 장비는 어떤 계측점을 갖고, 각 각 몇 번 레지스터에 어떤 타입으로 들어 있고, 단위는 무엇인가"를 적은 YAML 파일입니다. 이 파일이 있기에 코드를 짜지 않고 장비를 붙일 수 있습니다. 새 장비가 들어오면 프로파일을 쓰는 것으로 끝납니다.

AR-100 · EdgeX 4.0.0 을 씁니다(.env 의 EDGEX_VERSION=4.0.0).

프로파일은 손으로 쓰지 않습니다. 태그 정의를 두 곳에 두면 반드시 어긋나기 때문에, simulator/plant.yaml 을 유일한 출처로 삼고 edgex/gen_profile.py 가 프로파일과 디바이스 정의를 생성합니다.

```
$ cd edgex && python3 gen_profile.py
생성 완료: deviceResources=21 autoEvent=1000ms
```

21 개는 계측 12 + 시퀀스 카운터 1 + 코일 5 + 설정치 3 입니다. 이 리소스들이 세 개의 deviceCommand 로 묶입니다.

커맨드	권한	구성원	용도
AllSensors	R	계측 태그 12 개	자동 수집이 참조하는 소스
Control	RW	코일 4 + 설정치 3	사우스바운드 제어
Status	R	SEQ , Interlock	읽기 전용 상태

코일이 5 개인데 Control 에는 4 개만 들어간 것에 주목하십시오. 앞에서 본 Coil 20(인터록 상태)이 읽기 전용이라 빠졌습니다. EdgeX 는 커맨드의 권한과 소속 리소스의 권한이 어긋나면 프로파일을 통째로 거부합니다. 실제로 처음에는 Control (RW)에 Interlock (R)을 넣었다가 이런 오류로 기동 이 막혔습니다.

```
device command's ReadWrite permission 'RW' doesn't align the device resource
```

그래서 읽기 전용 리소스만 모아 Status (R) 커맨드를 따로 만들었습니다(docs/journal/02-tier1-edge.md § 2.7.3). Modbus 자체는 Coil 20 쓰기를 막지 않지만, 상위 계층인 EdgeX 는 이렇게 막습니다.

아무도 폴링 루프를 코딩하지 않았습니디. 디바이스 정의에 이 세 줄이 들어 있을 뿐입니다.

```
autoEvents:
- interval: 1000ms
  onChange: false
  sourceName: AllSensors
```

오해하기 쉬운데, autoEvents 는 폴링을 없애는 설정이 아닙니다. 반대로 device-modbus 가 1 초마다 슬레이브를 폴링하도록 시키는 설정입니다. Modbus 에서 폴링을 없애는 방법은 없습니다. 사라진 것은 폴링이 아니라 폴링 루프를 직접 짜는 일입니다.

onChange: false 도 읽을 줄 알아야 합니다. 이 값을 true 로 두면 "직전과 값이 달라졌을 때만 이벤트를 만들라"는 뜻이 됩니다. 데이터 양은 줄지만 정상 구간의 기록이 듬성듬성해져 시계열 분석 과 머신러닝 학습이 어려워집니다. 그래서 값이 그대로여도 매 초 찍도록 false 로 두었습니다.

반출은 app-mqtt-export 가 맡습니다. docker-compose.edgex.yml 에 전부 환경변수로 적혀 있습니다.

```

WRITABLE_PIPELINE_EXECUTIONORDER: "Transform, MQTTExport"
WRITABLE_PIPELINE_FUNCTIONS_MQTTEXPORT_PARAMETERS_BROKERADDRESS: "tcp://emqx:1883"
WRITABLE_PIPELINE_FUNCTIONS_MQTTEXPORT_PARAMETERS_TOPIC: "edgex/telemetry"
WRITABLE_PIPELINE_FUNCTIONS_MQTTEXPORT_PARAMETERS_QOS: "1"
WRITABLE_PIPELINE_FUNCTIONS_MQTTEXPORT_PARAMETERS_AUTORECONNECT: "true"
WRITABLE_PIPELINE_FUNCTIONS_MQTTEXPORT_PARAMETERS_PERSISTONERROR: "true"

```

"JSON 으로 바꾼 뒤 MQTT 로 보내내라, 브로커는 EMQX, 토픽은 edgex/telemetry , QoS 는 1, 끊기면 자동 재접속, 실패한 건 버리지 말고 보관하라." 파이프라인 자체가 설정입니다.

실측 비용은 작습니다. 정상 운전(12 건/초) 중 컨테이너별 메모리는 이렇습니다(docs/journal/09-metrics.md § 9.2).

컨테이너	메모리	컨테이너	메모리
edgex-postgres	54 MiB	edgex-core-keeper	15 MiB
edgex-core-data	18 MiB	edgex-core-metadata	11 MiB
edgex-app-mqtt-export	18 MiB	edgex-core-command	10 MiB
edgex-device-modbus	18 MiB	edgex-ui	6 MiB
edgex-mqtt-broker	1.3 MiB	9 개 합계	약 150 MiB

같은 환경에서 flink-taskmanager 하나가 2.19 GiB, kafka 가 2.17 GiB 입니다. 컨테이너 개수는 EdgeX 쪽이 압도적으로 많은데 메모리는 15 분의 1 입니다. "컨테이너 수 = 무거움"이 아니라는 실측 근거입니다. EdgeX 서비스가 Go 로 작성돼 개별 메모리 사용량이 작기 때문입니다.

주의 — 이 시스템에는 "1 초"가 두 군데 따로 적혀 있습니다

.env 의 SCAN_INTERVAL_MS=1000 — 시뮬레이터가 물리 모델을 한 스텝 풀고 레지스터를 갱신하는 주기입니다. sim.py 가 이 환경변수가 있으면 plant.yaml 의 값을 덮어씁니다. 디바이스 정의의 interval: 1000ms — EdgeX 가 그 레지스터를 읽어 가는 주기입니다. 이 값은 gen_profile.py 가 plant.yaml 의 scan_interval_ms 에서 만들어 냅니다.

둘은 서로 모르는 사이입니다. .env 만 500 으로 고치면 시뮬레이터는 초당 두 번 갱신하는데 EdgeX 는 여전히 초당 한 번만 읽어 가고, 중간 값은 아무도 못 본 채 사라집니다. 반대로 .env 를 2000 으로 늘리면 EdgeX 가 바뀌지 않은 같은 값을 두 번 읽어 갑니다.

이 둘이 지금 맞춰져 있기 때문에 앞의 실측이 성립합니다. 12 태그 × 초당 1 회 = 12 건/초입니다. 주기를 바꿀 일이 있으면 plant.yaml 을 고치고 make regen-edgex 로 프로파일을 다시 만들어 양쪽 을 함께 옮겨야 합니다.

Store-and-Forward

직역하면 저장하고(store) 나중에 보낸다(forward)입니다.

왜 필요한가. 설계 보고서의 진단은 이렇습니다. "무선 센서 네트워크나 광역 공장망은 통신 간섭, 전원 노이즈, 스위치 리부팅 등으로 인한 간헐적 네트워크 단절에 취약하다." 사무실 와이파이가 잠깐 끊기면 페이지를 다시 누르면 됩니다. 공장은 다릅니다. 그 30 초 동안 반응기 안에서 일어난 일은 영원히 사라집니다. 다시 켤 방법이 없습니다.

게다가 그 30 초가 하필 사고가 난 30 초일 수 있습니다. 사고 조사에서 가장 중요한 구간이 비어 있는 것입니다.

어떻게 생겼는가. 에지 게이트웨이가 상위로 보내지 못한 데이터를 디스크에 먼저 기록합니다. 설계 보고서는 이때 트랜잭션 단위로 쓰라고 합니다. 쓰는 도중에 전원이 나가도 기록이 반쪽만 남는 일이 없게, 한 건을 통째로 기록하거나 아예 안 하거나 둘 중 하나로 만들라는 뜻입니다. 설계 보고서는 저장소로 SQLite, RocksDB, 플래시 메모리 파일시스템을 듭니다. 연결이 복구되면 로컬 큐에 쌓인 것을 타임스탬프 순서대로 순차 재전송합니다. 순서가 중요합니다. 뒤영켜 도착하면 시계열 분석과 상태 기반 연산이 전부 망가집니다.

초보가 반드시 헷갈리는 지점 — 이건 총이 세 개입니다. 무손실 장치가 하나가 아니라 셋이고, 각각 막는 고장이 다릅니다.

층	누가	무엇을 막는가	한계
브로커 세션 영속	EMQX	구독자가 잠깐 끊긴 사이의 메시지 유실	브로커가 살아 있어야 함. 큐 길이 한계
Store-and-Forward	에지 미들웨어	브로커까지 가는 길이 끊긴 구간	에지 디스크 용량이 한계

층	누가	무엇을 막는가	한계
Kafka 보존	Kafka	하위 소비자(Flink·DB)가 멈춘 구간	Kafka 에 도착한 뒤의 이야기

첫 줄의 "브로커 세션 영속"이 무엇인지 풀어야 합니다. MQTT 클라이언트는 접속할 때 "내가 끊겼다 돌아오면 내 구독 목록과 밀린 메시지를 지워도 되는가"를 브로커에 알려 줍니다. 지우지 말라고 하는 것이 세션 영속이고, 옛 표기로는 `clean_session=False` 입니다. AR-100의 우회 경로 시뮬레이터가 바로 이렇게 접속합니다.

```
c = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2,
    client_id=f"sim-{self.cfg['device']}",
    clean_session=False,      # 세션 영속 (PDF p.4 권고)
)
```

받는 쪽인 브로커에도 짝이 되는 설정이 필요합니다. 메시지를 얼마나 오래, 몇 건까지 들고 있을지입니다.

```
mqtt {
    session_expiry_interval = 2h      ## 끊긴 클라이언트의 세션을 2 시간 보존
    max_queue_len = 100000           ## 그동안 쌓아 둘 메시지 상한
}                                   ## emqx/emqx.conf
```

그러나 설계 보고서가 지적하는 한계가 여기 있습니다. "광역 네트워크가 완전히 차단되면 브로커의 세션 버퍼는 곧 한계에 도달하므로, 현장 게이트웨이는 인입되는 센서 프레임을 로컬에 트랜잭션 단위로 선기록해야 한다." 브로커가 아무리 잘 보관해도, 브로커까지 가는 길 자체가 끊기면 브로커는 애초에 그 메시지를 받은 적이 없습니다. 그 구간을 메우는 것이 두 번째 층, Store-and-Forward 입니다.

AR-100 · Store-and-Forward 설정 한 줄이 왜 안 먹었는가

켜는 것 자체는 간단합니다.

```
WRITABLE_STOREANDFORWARD_ENABLED: "true"
```

그런데 재시도 주기가 바뀌지 않았습니다. 로그가 이렇게 나왔습니다.

```
Starting StoreAndForward Retry Loop with 5m0s RetryInterval and 10 max retries.
      ^^^^                                ^^
      설정한 5s / 1000 이 아님
```

문서를 다시 읽는 대신 설정 저장소를 직접 조회했습니다. EdgeX 4.0 은 설정을 core-keeper 에 보관합니다.

```
docker run --rm --network iiot curlimages/curl -s \
    "http://edgex-core-keeper:59890/api/v3/kvs/key/edgex?keyOnly=true"
```

키가 두 군데에 나뉘어 있었습니다.

```
edgex/v4/app-mqtt-export/Writable/StoreAndForward/Enabled      ← 서비스별
edgex/v4/core-common-config-bootstrapper/app-services/Writable/
    StoreAndForward/RetryInterval                              ← 공통
edgex/v4/core-common-config-bootstrapper/app-services/Writable/
    StoreAndForward/MaxRetryCount                              ← 공통
```

Enabled 는 서비스 설정, 재시도 값은 공통 설정이었습니다. 서비스에 없는 키를 덮어쓰려 하면 EdgeX 는 조용히 건너뛵니다. 오류도 경고도 없습니다. 그래서 앞서 소개한 부트스트래퍼 쪽에 넣었습니다.

```
edgex-common-config:
  environment:
    APP_SERVICES_WRITABLE_STOREANDFORWARD_RETRYINTERVAL: "5s"
    APP_SERVICES_WRITABLE_STOREANDFORWARD_MAXRETRYCOUNT: "1000"

Starting StoreAndForward Retry Loop with 5s RetryInterval and 1000 max retries.
```

여기서 배울 것은 EdgeX 가 아닙니다. 설정이 안 먹을 때 문서를 다시 읽는 것보다 설정 저장소를 직접 들여다보는 것이 빠르다는 사실입니다. EdgeX 는 keeper, 쿠버네티스는 `kubectl get -o yaml`, 스프링은 `/actuator/env` 입니다.

AR-100 · 이 시스템의 Store-and-Forward 큐는 어디에 있는가

설계 보고서는 저장소로 SQLite·RocksDB·플래시를 들지만, 그것은 일반론입니다. 이 시스템에서는 다릅니다. 설정 파일의 주석이 그대로 답입니다.

```
# — Store-and-Forward: EMQX 단일 시 이벤트를 DB 에 적재 후 순차 재전송 —
# (PDF p.4 "에지 레벨 로컬 역단절 메커니즘" 구현)
WRITABLE_STOREANDFORWARD_ENABLED: "true" # docker-compose.edgex.yml
```

여기서 말하는 "DB"는 EdgeX 가 쓰는 자기 데이터베이스이고, EdgeX 4.0 의 DB 는 PostgreSQL 입니다. 컴포즈에 edgex-postgres 라는 이름으로 떠 있고(image: postgres:16.3-alpine3.20), 모든 서비스가 ALL_SERVICES_DATABASE_HOST: edgex-postgres 로 그곳을 가리킵니다. 앞의 메모리 표에서 EdgeX 구성 요소 중 edgex-postgres 만 54 MiB 로 유독 큰 이유이기도 합니다.

낡은 주석 하나가 남아 있습니다. simulator/plant.yaml 맨 아래에는 "EdgeX app-service 의 Store-and-Forward 가 Redis 에 적재 후 재전송하는지 검증합니다"라고 적혀 있습니다. Redis 는 EdgeX 3.x 의 DB 이고 4.0 에서 PostgreSQL 로 바뀌었으니, 이 줄은 버전이 올라가면서 갱신되지 못한 흔적입니다. 설정 파일의 주석은 코드가 아니므로 틀려도 아무것도 깨지지 않습니다. 그래서 오래 살아남습니다.

단절 시험은 EMQX 를 멈추는 방식으로 합니다(make fault-netdown). 시뮬레이터가 아니라 컨테이너 수준에서 30 초 동안 정지시켰다가 되살립니다.

```
fault-netdown:
  @echo "EMQX 정지 (30 초)..."; docker compose pause emqx
  @sleep 30; docker compose unpause emqx
```

이 시험이 성립하려면 조건이 하나 있습니다. EdgeX 의 내부 메시징버스가 EMQX 와 다른 컨테이너 여야 합니다. 그래서 이 시스템은 내부 버스용으로 eclipse-mosquitto 를 따로 띄웁니다. 컴포즈 파일 머리말에 이유가 적혀 있습니다.

```
# 내부 버스용 mosquitto 를 EMQX 와 분리해 두어야 EMQX 를 정지시켰을 때
# Store-and-Forward 만 단독으로 검증됩니다.
```

둘을 한 브로커로 합쳤다면 EMQX 를 멈추는 순간 EdgeX 내부 통신까지 죽어, 데이터가 애초에 생산되지 않습니다. 그러면 "끊긴 동안 쌓아 뒀다가 다시 보냈다"를 증명할 수 없습니다. 무엇을 시험 하려는지가 컨테이너를 어떻게 쪼개는가까지 결정한 사례입니다.

시험 범위 · make fault-netdown 은 EMQX 를 30 초 멈추는 수동 시연입니다. 저장 대기 항목이 늘고 복구 뒤 0 으로 돌아가는지 확인합니다. 원문은 이 시험이 scripts/verify.py 의 자동 검증 23 항목에는 없다고 명시합니다. "시연 절차가 있음"과 "자동 검사에 포함됨"을 구분합니다. [원문 72, 100 쪽]

주의 — EdgeX 4.0 은 3.x 와 구조가 다릅니다. 3.x 자료를 그대로 따르면 기동조차 되지 않습니다

구성 요소	EdgeX 3.x	EdgeX 4.0
서비스 레지스트리	Consul	core-keeper (EdgeX 자체 구현)
데이터베이스	Redis	PostgreSQL
내부 메시징버스	Redis Streams	MQTT (mosquitto)

레지스트리(registry)는 "어떤 서비스가 어디에 떠 있는지"를 적어 두는 전화번호부입니다. Consul 은 하시코프의 유명한 레지스트리 제품이고, 인터넷의 EdgeX 자료 대부분이 이것을 전제합니다.

확인 방법은 추측이 아니라 공식 구성 파일을 받아 보는 것입니다.

```
curl -O https://raw.githubusercontent.com/edgexfoundry/edgex-compose/v4.0.0/docker-compose-no-sect grep -E "[a-z]+:" docker-compose-no-sect.yml
```

```
core-command:
core-common-config-bootstrapper:
core-data:
core-keeper:      ← Consul 이 아님
core-metadata:
database:         ← postgres (Redis 아님)
mqtt-broker:      ← eclipse-mosquitto (내부 버스)
```

같은 이유로 걸린 함정이 셋 더 있었습니다.

-o 플래그 없이는 환경변수가 무시됩니다. EdgeX 4.0 이 설정을 keeper 에 영속화하기 때문에, 최초 1 회 기동에만 환경변수가 반영되고 이후에는 저장된 값을 씁니다. .env 를 고쳐도 안 바뀌는 현상의 원인입니다. command: ["--registry", "-cp=keeper.http://edgex-core-keeper:59890", "-o"] 로 매번 다시 밀어 넣습니다.

service_completed_successfully 를 쓰면 영구 대기합니다. 부트스트래퍼가 "exiting" 로그를 남기고도 프로세스가

종료되지 않아, compose 가 무한정 기다립니다. 공식 구성이 service_started 를 쓰는 데는 이유가 있습니다 — EdgeX 서비스들은 설정이 올라올 때까지 스스로 재시도하도록 설계돼 있습니다. deviceCommand 의 권한은 구성원과 일치해야 합니다. 앞에서 본 Interlock 분리가 이 규칙 때문입니다.

2·4 미들웨어를 둘 것인가 뺄 것인가 — 설계 보고서 결론 1 병원에서 간호사를 빼고 모니터를 공장 중앙 서버에 연결하면 어떻게 될까요. 환자가 전부 같은 기종 모니터를 쓰고, 기록만 남기면 되고, 아무도 처방을 내리지 않는 병동이라면 그래도 됩니다. 실제로 그게 더 빠르고 싸입니다. 그런데 처방을 내려야 하는 순간, 간호사가 없다는 사실이 갑자기 치명적이 됩니다.

설계 보고서는 왜 "IoT 미들웨어 존치"를 결론 1에 뒀는가

설계 보고서의 다섯 결론 중 첫 번째는 이것입니다.

"IoT 미들웨어의 존치: 센서 네트워크와 Kafka 사이에 EdgeX Foundry 나 고성능 브로커(EMQX)를 반드시 유지하여 프로토콜 정규화와 에지 Store-and-Forward 유실 방지 체계를 수립해야 한다."

"반드시"라는 말이 붙은 이유는, 빠자는 주장이 충분히 그럴듯하기 때문입니다. 컴포넌트가 줄면 자연 이 줄고, 장애 지점이 줄고, 운영 인력이 줄고, 비용이 줄입니다. 아키텍처 검토에서 늘 나오는 제안입니다.

설계 보고서가 존치를 권고하는 근거는 미들웨어가 없으면 아예 불가능해지는 것이 세 가지 있기 때문입니다.

프로토콜 정규화 — Modbus 장비는 Kafka 로 스스로 올라갈 수 없습니다. 누군가 풀링하고 해석해야 합니다. 에지 Store-and-Forward — 중앙이 끊긴 구간을 메우는 일은 중앙이 할 수 없습니다. 정의상 현장 에 있어야 합니다.

사우스바운드 제어 경로 — 설계 보고서의 표현대로, 제어 값을 주입하는 주체는 "Kafka 가 아닌 IoT 미들웨어"입니다.

그러면 직결이 유리한 조건은 무엇인가. 설계 보고서는 이 조건도 분명히 적었습니다.

"현장 센서들이 이미 균일한 스펙을 갖추고 표준화된 MQTT 토픽으로 데이터를 자율 발행하고 있으며, 시스템의 요구사항이 단방향 데이터 수집 및 로깅에 국한되어 있다면, IoT 미들웨어를 거치지 않고 MQTT 브로커에서 Kafka 로 직결하는 아키텍처가 유리하다."

세 조건이 전부 동시에 성립해야 한다는 점이 중요합니다. 하나라도 어긋나면 미들웨어가 필요해집니다.

조건	직결이 유리	미들웨어가 필요
장비 구성	같은 기종, 같은 펌웨어	연식·제조사가 뒤섞임
프로토콜	전부 MQTT 를 스스로 말함	Modbus·OPC-UA·BACnet 혼재
메시지 형식	이미 공통 스키마로 발행	장비마다 제각각
방향	올려보내기만 (수집·로깅)	내려보내기 필요 (제어)
인가	고정 — 한 번 설정하면 그대로	동적 — 운전원·상황별로 권한이 바뀜
망 안정성	중앙과 안정적으로 연결	간헐적 단절이 예상됨
처리량	수십만 TPS 이상	게이트웨이당 수백~수천

마지막 행의 TPS 는 transactions per second, 초당 처리 건수입니다. 참고로 AR-100 의 실측 유입은 12 건/초이므로, 이 표에서 직결이 유리한 쪽 근처에도 가지 않습니다.

설계 보고서는 마지막 못을 이렇게 박습니다. "설비 제어 명령을 하향 전파해야 하거나 장비의 인가 권한을 동적으로 제어해야 하는 SCADA 본연의 기능을 수행해야 한다면, IoT 미들웨어를 진입점에 배치하고 정제된 이벤트 스트림만을 Kafka 로 토출시키는 다계층 구조를 유지하는 것이 아키텍처적 결함을 방지하는 유일한 방안이다."

SCADA(Supervisory Control And Data Acquisition)의 이름 자체에 Control 이 들어 있습니다. 감시만 하는 것은 SCADA 가 아닙니다. 이 문장이 뒤에 나올 Grafana 비판(감시는 되지만 제어가 안 된다)과 정확히 같은 논리입니다.

AR-100 · 두 경로를 둘 다 만들었습니다

이 시스템은 미들웨어 존치와 직결 중 하나를 고르지 않고, 두 가지 기동 방식을 제공합니다.

	make up (EdgeX 경로)	make lite (우회 경로)
수집 경로	시뮬레이터 → EdgeX → EMQX → telegraf-bridge → Kafka	시뮬레이터 → EMQX → telegraf-bridge-lite → Kafka
컨테이너 수	29 개	19 개

	make up (EdgeX 경로)	make lite (우회 경로)
브리지 설정	telegraf/bridge-edgex.conf	telegraf/bridge-lite.conf
Kafka 진입 스키마	동일 — ts / site / device / tag / value / quality	
실증되는 것	프로토콜 정규화, Store-and-Forward, 사우스바운드	직결에 가까운 단순함과 가벼움

우회 경로에도 Telegraf가 남아 있는 것이 이상해 보일 수 있습니다. 그럴 만한 이유가 있습니다. 설계 보고서가 그린 원래 그림은 EMQX가 직접 Kafka로 넣는 것이었는데, 잠시 뒤 볼 이유로 오픈소스 EMQX에는 그 기능이 없습니다. 그래서 여기서 말하는 "직결"은 EdgeX라는 미들웨어 계층을 건너뛰는 뜻이지 컴포넌트가 0개라는 뜻이 아닙니다. 다만 bridge-lite.conf는 변환을 거의 하지 않습니다. 시뮬레이터가 이미 최종 계약 형태로 발행하기 때문입니다.

직발행 경로는 시뮬레이터 안의 이 코드가 담당합니다. 앞에서 본 세션 영속이 여기 걸려 있습니다.

```
c = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2,
    client_id=f"sim-{self.cfg['device']}",
    clean_session=False,      # 세션 영속 (PDF p.4 참고)
)
```

두 경로의 Kafka 진입 형식이 같기 때문에, Tier 3 이후(Flink·InfluxDB·Grafana·FUXA)는 어느 쪽으로 기동했는지 전혀 모릅니다. 이것이 정규화를 진입점에 두는 것의 실제 효용입니다. 경로를 통째로 갈아 끼워도 위쪽이 무사합니다.

설계 기록(docs/journal/08-decisions.md의 ADR-003)은 이 선택의 이유를 이렇게 남겼습니다. "EdgeX만" 쓰면 부트스트랩 실패 시 전체가 불가능해지고, "lite만" 쓰면 프로토콜 정규화와 Store-and-Forward를 실증할 수 없어 설계 보고서 결론 1이 반영되지 않습니다.

같은 자리의 다른 도구들 — 미들웨어 자리에 무엇을 놓을 것인가 EMQX — 고성능 분산 MQTT 브로커. 규칙 엔진으로 메시지를 걸러내고 바꿀 수 있습니다. 설계 보고서는 "수십만 TPS의 데이터를 Kafka 토픽으로 즉각 분배할 수 있다"고 평가하는데, 앞에서 본 대로 그 Kafka 분배는 상용 판 기능입니다. 그리고 이미 MQTT로 올라온 것만 다릅니다. Modbus 장비를 붙일 수 없습니다. ThingsBoard — 디바이스 관리·전송 엔드포인트·드래그 앤 드롭 룰 엔진·대시보드를 한 제품에 담은 종합 IoT 플랫폼입니다. 슬라이더·토글 같은 양방향 RPC(Remote Procedure Call, 원격 프로시저 호출 — 다른 컴퓨터의 기능을 내 것처럼 불러 쓰고 결과를 돌려받는 방식) 위젯으로 제어까지 됩니다. 설계 보고서의 유보는 명확합니다. "플랫폼의 모놀리식/마이크로서비스 구조 자체가 무겁고, 초당 수십만 건 이상의 초고빈도 스트림이 유입될 경우 플랫폼 내부 메시지 큐의 오버헤드가 발생할 수 있다." 또 프로토콜 수용 범위가 MQTT·CoAP(Constrained Application Protocol, 작은 기기용으로 HTTP를 경량화한 규약)·HTTP·LwM2M(Lightweight M2M, 기기 원격 관리·펌웨어 갱신용 경량 규격) 중심이라 산업용 필드버스도 약합니다. Node-RED — 블록을 선으로 잇는 방식의 흐름 편집기. 시작이 빠르고 프로토콜 노드가 많습니다. 반면 디바이스 등록·인증·메타데이터 관리 같은 라이프사이클 기능이 없어, 장비 수가 늘면 흐름도가 감당이 안 됩니다. Apache StreamPipes — 센서 어댑터·윈도우 변환·이상 탐지 블록을 웹 편집기에서 조립하는 노코드 분석 플랫폼입니다. 분석 쪽에 무게가 실려 있어 현장 제어 기능이 없습니다. 직결(미들웨어 없음) — 센서가 직접 MQTT를 말하고 브로커에서 곧장 Kafka로. 위의 EMQX 항목이 말하는 그 구성이고, 오픈소스로 하려면 브리지가 하나 필요합니다.

설계 보고서의 비교표는 이렇습니다. 원문의 항목과 판단 기준을 유지했습니다.

비교 항목	ThingsBoard (Community)	EdgeX Foundry	Apache StreamPipes	이 시스템의 스택 (Flink·Kafka·FUXA)
핵심 포지셔닝	중앙 집중형 통합 디바이스 관리	에지 레벨 산업 프로토콜 정규화	노코드 산업용 스트림 데이터 분석	엔터프라이즈급 대규모 분산 파이프라인
데이터 수집 백본	내장 큐 또는 외부 Kafka 플러그인	내부 eKuiper/ZeroMQ 또는 MQTT	내장 Kafka/Pulsar 엔진 기반	순수 Kafka KRaft 분산 클러스터
산업 프로토콜 수용 성	MQTT·CoAP·HTTP·LwM2M 중심	Modbus·OPC-UA·BACnet 등 극대화	전용 커넥터를 통한 PLC/MQTT 수집	IoT 미들웨어 (EdgeX/EMQX) 어댑터 연동
스트림 분석 및 ML	노드 기반 비주얼 룰 엔진	eKuiper 연계 경량 SQL 스트림 처리	사전 정의된 스트림 분석 블록	Flink 기반 분산 상태 연산 및 ONNX 실시간 서빙

비교 항목	ThingsBoard (Community)	EdgeX Foundry	Apache StreamPipes	이 시스템의 스택 (Flink-Kafka-FUXA)
SCADA 제어 적합성	양방향 RPC 제어 위젯 완비	REST 기반 Command 마이크로서비스	분석 위주 (현장 제어 기능 부재)	FUXA 결합을 통한 정밀 P&ID 양방향 제어
분산 확장성 및 성능	중간~높음 (컨테이너 클러스터 배포)	단일 에지 게이트웨이 리소스에 종속	중간 (독립 클러스터링 지원)	극대화 (Kafka 파티션 및 Flink 슬롯 선형 확장)

표에 처음 나온 말 둘을 풀어 두겠습니다. P&ID는 Piping and Instrumentation Diagram, 우리말로 배관계장도입니다. 배관·밸브·펌프·계기를 약속된 기호로 그린 공정 도면이고, 운전원이 보는 관제 화면이 대개 이 도면 모양을 그대로 씁니다. eKuiper는 EdgeX 진영의 초경량 SQL 룰 엔진으로, 에지에서 간단한 필터링과 집계를 합니다. AR-100은 그 자리에 Flink를 썼기 때문에 eKuiper를 쓰지 않습니다.

마지막 행이 EdgeX의 성격을 말해 줍니다. EdgeX는 중앙에서 수만 대를 감당하는 물건이 아닙니다. 게이트웨이 한 대에 붙여 그 라인의 장비들을 정규화하는 것이 제자리이고, 규모는 게이트웨이 대수로 늘립니다.

3 MQTT와 브로커 — 방송하듯 보내기

원문 교본 79~108 쪽의 설명을 통합

발행자는 브로커에 한 번 보내고, 구독자는 관심 있는 토픽을 받습니다. 이 결합 해제가 MQTT의 역할입니다. 토픽·QoS·세션은 배달 범위를 정하지만, 지나간 이력을 다시 읽는 일은 다음 권의 Kafka가 맡습니다.

왜 센서 데이터에는 웹의 방식이 맞지 않는가

우리가 웹에서 쓰는 통신 방식은 요청/응답(request/response)입니다. 브라우저가 서버에게 "이 페이지 주세요" 하고 요청을 보내면 서버가 응답을 돌려줍니다. 이 규약의 이름이 HTTP(HyperText Transfer Protocol, 하이퍼텍스트 전송 규약)입니다. 특징은 세 가지입니다. 첫째, 말을 거는 쪽이 항상 클라이언트입니다. 서버는 먼저 말하지 못합니다. 둘째, 한 번의 요청과 한 번의 응답이 짝을 이루고 끝납니다. 셋째, 누가 누구에게 말하는지 주소로 꼭 집어 정해야 합니다.

센서 데이터를 이 방식으로 모으려면 값이 필요로 하는 쪽이 계속 물어봐야 합니다. 이것을 폴링(polling, 주기적 조회)이라고 합니다. 폴링에는 네 가지 문제가 있습니다.

- (1) 물어보는 주기와 값이 변하는 주기가 어긋납니다. 1초마다 물어보면 그 사이에 일어난 순간 과압은 놓치고, 0.01초마다 물어보면 값이 안 변했는데도 계속 묻습니다. 센서는 자기가 값을 낸 그 순간을 가장 잘 압니다. 말할 타이밍을 아는 쪽은 듣는 쪽이 아니라 말하는 쪽입니다.
- (2) 연결 수가 곱셈으로 늘어납니다. 센서가 N대, 그 값을 필요로 하는 소비자가 M개면 관계가 $N \times M$ 개 생깁니다. 센서 1,000대에 소비자가 5개면 5,000개의 연결 관계를 누군가 관리해야 합니다. 소비자 하나 더 붙으려면 센서 1,000대의 설정을 전부 고쳐야 합니다. 이것을 N대 M 문제라고 부릅니다.
- (3) 연결을 맺는 일 자체가 비쌉니다. TCP(Transmission Control Protocol, 전송 제어 규약) 연결 하나를 맺으려면 3-way handshake(세 단계 악수)를 거칩니다. 클라이언트가 SYN을 보내고, 서버가 SYN-ACK로 답하고, 클라이언트가 ACK를 보내는 패킷 세 개짜리 절차입니다. 왕복(round trip, 보냈다가 답이 돌아오기까지)으로 세 번 데이터를 실어 보내기 전까지 약 한 번입니다. 여기에 암호화를 쓰면 왕복이 더 붙습니다. 암호화 규약의 이름은 TLS(Transport Layer Security, 전송 계층 보안)로, 웹 주소의 https가 바로 이것입니다. 주고받는 바이트를 암호로 덮어 중간에서 읽지 못하게 하고, 상대가 사칭이 아닌지 인증서로 확인합니다. HTTP 요청은 기본적으로 매번 이 값을 치르거나, 연결을 유지하더라도 요청마다 수백 바이트짜리 텍스트 헤더(Host: , User-Agent: , Accept: ...)를 다시 실어 보냅니다. 값 하나가 8바이트인데 포장이 400바이트인 셈입니다.
- (4) 배터리가 닳습니다. 무선 센서에서 전력을 가장 많이 쓰는 동작은 계산이 아니라 무선 송신기를 켜는 일입니다. 연결을 새로 맺을 때마다 송신기가 오래 켜집니다.

발행/구독(publish/subscribe, 흔히 pub/sub)은 이 네 가지를 한꺼번에 뒤집는 구조입니다. 등장인물이 셋입니다. 값을 내놓는 발행자(publisher), 값을 받고 싶은 구독자(subscriber), 그리고 둘 사이에 서서 배달을 맡는 브로커(broker)입니다. 발행자는 브로커에게 "이 주소로 이 값"이라고 던지고 끝냅니다. 누가 받는지, 몇 명이 받는지, 지금 받는 사람이 있기는 한지 알지 못하고 알 필요도 없습니다. 구독자는 브로커에게 "이 주소로 오는 것은 나에게 주세요"라고 신청해 둡니다. 누가 보내는지는 모릅니다.

이렇게 되면 발행자와 구독자는 세 가지 면에서 서로 묶이지 않습니다(이것을 결합 해제, decoupling 이라고 합니다).

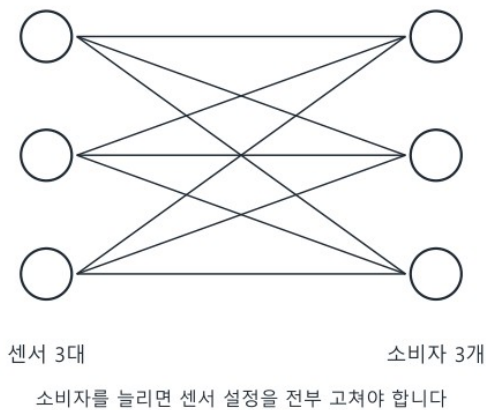
- 공간 — 서로의 IP 주소를 모릅니다. 브로커 주소만 압니다.
- 시간 — 동시에 접속해 있지 않아도 됩니다. 구독자가 잠시 꺼져 있어도 브로커가 받아 둘 수 있습니다.
- 동기 — 발행자는 응답을 기다리지 않습니다. 던지고 곧바로 다음 일을 합니다.

연결 수도 곱셈에서 덧셈으로 바뀝니다. 센서 N 대와 소비자 M 개는 각자 브로커와 연결 하나씩만 가지므로 $N+M$ 개입니다. 소비자를 하나 더 붙이는 일은 브로커에 구독 신청 한 줄을 더하는 일이지, 센서를 건드리는 일이 아닙니다. 이것이 MQTT 입니다. 1999 년 송유관 원격 감시를 위해 만들어진, 태생부터 "연결이 느리고 자주 끊기는 현장에서 계속값을 올린다" 는 목적의 규약입니다.

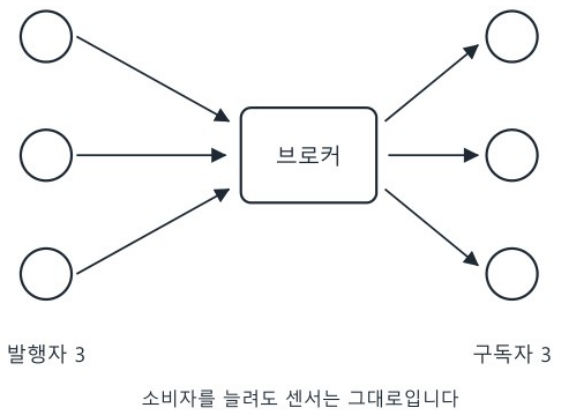
주의 — 이름을 잘못 읽으면 개념까지 틀립니다. MQTT 를 "Message Queuing Telemetry Transport(메시지 큐 방식 원격계측 전송)" 의 약자라고 적은 자료가 많지만, 이것은 나중에 거꾸로 만들어 붙인 풀이입니다. 원래 이름은 IBM 의 메시징 제품 MQSeries 에서 따온 MQ Telemetry Transport 이고, 표준 문서인 MQTT 5.0 명세는 아예 "MQTT 는 더 이상 약자가 아니다" 라고 못박습니다. 이 구분이 중요한 이유는 다음 절에서 바로 나옵니다 — MQTT 브로커는 메시지를 쌓아 두는 큐(queue)가 아닙니다. "메시지 큐 방식" 이라는 풀이는 정확히 그 오해를 심습니다.

요청/응답 — 관계 $3 \times 3 = 9$ 발행/구독 — 연결 $3 + 3 = 6$

요청/응답 — 관계 $3 \times 3 = 9$



발행/구독 — 연결 $3 + 3 = 6$



원문 도해 · 직결의 연결 관계와 발행/구독의 연결 관계 (원문 81 쪽)

주의. 발행/구독이 요청/응답보다 "좋은" 방식인 것은 아닙니다. 답을 받아야 하는 일에는 여전히 요청/응답이 맞습니다. 밸브를 40%로 열라고 지시했으면 정말 40%가 되었는지 확인(에코 체크)을 받아야 하고, 안 되었으면 누가 언제 실패했는지 남아야 합니다. 발행하고 잇는 구조로는 이 확인이 성립하지 않습니다. 그래서 AR-100 에서도 올라가는 데이터는 MQTT, 내려가는 제어는 Modbus/TCP 직접 쓰기로 길을 나눕니다(제어 경로는 뒤 장에서 다룹니다).

3.2 브로커가 실제로 하는 일

브로커는 배달원이자 창고가 아닙니다

브로커는 하나의 서버 프로세스입니다. 기본 포트 1883 번(TLS 로 암호화하면 8883 번)에서 TCP 연결을 기다립니다. 클라이언트(발행자든 구독자든 MQTT 로 붙는 쪽은 전부 클라이언트입니다)가 붙으면 다음 순서가 일어납니다.

- (1) 접속 — 클라이언트가 CONNECT 패킷을 보냅니다. 여기에는 클라이언트 식별자(Client ID), 사용자/비밀번호, 세션을 이어받을지 여부, 유언(Last Will), keepalive 주기가 들어갑니다. 브로커는 CONNACK 로 답합니다.
- (2) 구독 등록 — 구독자가 SUBSCRIBE 로 관심 있는 토픽(주소)을 신청하면, 브로커는 구독 테이블에 "이 클라이언트는 이 패턴을 원한다" 를 기록하고 SUBACK 으로 답합니다.
- (3) 배달 — 발행자가 PUBLISH 를 보내면 브로커는 그 토픽에 맞는 구독자를 구독 테이블에서 찾아 같은 내용을 각자에게 PUBLISH 로 내보냅니다. 이 매칭이 브로커가 하는 계산의 거의 전부입니다.

(4) 연결 유지 확인 — 할 말이 없어도 클라이언트는 keepalive 주기마다 PINGREQ 를 보내고 브로커는 PINGRESP 로 답합니다. 이 두 패킷은 각각 2 바이트입니다. 브로커는 keepalive 의 1.5 배 시간 동안 그 클라이언트로부터 아무 패킷도 받지 못하면 연결이 끊어진 것으로 판정합니다.

브로커는 데이터베이스가 아닙니다. 브로커는 배달 하고 나면 그 메시지를 버립니다. 어제 값을 다시 달라고 할 수 없고, "지난 1 시간치를 처음부터 다시 읽겠다" 도 안 됩니다. 브로커가 메시지를 붙들고 있는 경우는 딱 두 가지, 구독자가 잠시 끊겨서 오프라인 큐에 쌓아 둘 때와 보관 메시지(retained)로 토픽당 마지막 한 건을 들고 있을 때뿐입니다. 둘 다 "이력" 이 아닙니다.

그래서 파이프라인에는 브로커 뒤에 Kafka 가 따로 섭니다. Kafka 는 받은 메시지를 디스크에 로그로 남기고, 소비자마다 "어디까지 읽었는지" 를 따로 관리하며, 머칠치를 처음부터 다시 읽게 해 줍니다. 브로커는 지금 이 순간의 흐름을 다루고 Kafka 는 지나간 흐름 전체를 다룹니다. 역할이 겹치는 것이 아니라 다른 것입니다. 이 구분은 다음 장의 주제입니다.

메시지 한 건은 무엇으로 이루어지는가

이 장의 나머지가 전부 이 네 단어 위에 서 있습니다. 여기서 한 번에 풀어 두겠습니다.

토픽(topic) — 메시지의 주소입니다. iiot/AR-100/reactor-line-01/TT-101 같은 문자열 하나입니다. 브로커가 배달처를 고르는 유일한 근거입니다. 자세한 것은 3.3 에서 다룹니다.

페이로드(payload) — 메시지의 내용물입니다. 포장(토픽-헤더)을 뺀 알맹이, 실제로 실어 나르려는 본문 그 자체입니다. 택배에 비유하면 송장이 토픽이고 상자 속 물건이 페이로드입니다. MQTT 규약의 관점에서 페이로드는 그냥 바이트 덩어리입니다. 규약은 그 안에 JSON 글자가 들었는지, 사진이 들었는지, 숫자 하나가 들었는지 정하지도 않고 들여다보지도 않습니다. 길이가 0 인 페이로드, 즉 빈 내용물도 정상적인 메시지입니다(3.5 의 보관 메시지 삭제에서 이 성질을 씁니다). 이 교본에서 "페이로드에 시퀀스 번호를 넣는다", "페이로드에 타임스탬프를 넣는다" 는 말은 전부 그 바이트 덩어리를 JSON 으로 짓되 그 안에 이런 항목을 함께 담는다는 뜻입니다.

플래그(flag) — 켜짐/꺼짐 두 상태만 갖는 1 비트짜리 표시입니다. 숫자도 문자도 아니고 "예/아니오" 하나입니다. MQTT 에는 이런 표시가 여럿 있습니다. RETAIN 플래그(이 메시지를 브로커가 한 건 붙들고 있을 것인가), DUP 플래그(이것은 재전송인가), Clean Session 플래그(접속하면서 이전 상태를 지울 것인가). "플래그를 켜다" 는 그 비트를 1 로 둔다는 뜻입니다.

패킷 식별자(Packet Identifier) — QoS 1·2 메시지에 붙는 1 부터 65535 까지의 일련번호입니다. 왜 필요한가 하면, 확인 응답이 돌아왔을 때 어느 메시지에 대한 확인인지 알아야 하기 때문입니다. 보내는 쪽은 PUBLISH 에 번호 42 를 붙여 보내고, 받는 쪽은 PUBACK 에 42 를 실어 답합니다. 그래야 동시에 여러 건을 날려 두고도 짝을 맞출 수 있습니다. QoS 0 메시지는 확인이 없으므로 이 번호도 붙지 않습니다.

3.3 토픽과 와일드카드

주소를 문자열로 짓는다는 것

토픽의 형식은 슬래시로 구분한 문자열 하나가 전부입니다.

iiot/AR-100/reactor-line-01/TT-101 슬래시로 나뉜 각 조각을 단계(level)라고 부릅니다. 위 토픽은 4 단계입니다. 왼쪽이 넓고 오른쪽이 좁습니다. 사이트(공장) 안에 설비가 있고 설비 안에 계측점이 있습니다. 파일 경로와 모양은 같지만 성격은 다릅니다. 무엇보다 토픽은 미리 만들어 둘 필요가 없습니다. 데이터베이스라면 테이블을 먼저 정의해야 하지만, MQTT 에서는 발행자가 그 문자열로 한 번 보내는 순간 그 토픽이 존재합니다. 아무도 구독하지 않으면 그냥 버려지고, 그것으로 끝입니다. 이 가벼움이 장점이자 함정입니다. 오타를 내 면 오류가 나지 않고 조용히 아무 데도 도착하지 않습니다.

구독할 때는 정확한 토픽 대신 와일드카드(wildcard, 대체 문자)를 쓸 수 있습니다. 두 개뿐입니다.

- + — 한 단계를 아무 값으로나 채웁니다. iiot/+ /TT-101 은 어느 사이트, 어느 설비의 것이든 TT-101 만 받습니다.
- # — 그 아래 전부를 받습니다. 반드시 맨 끝에만 올 수 있습니다. iiot/# 은 iiot 로 시작하는 모든 것입니다.

여기에 초보가 반드시 걸려 넘어지는 예외가 둘 있습니다.

예외 1 — # 는 자기 위 단계 자신도 매칭합니다. iiot/# 은 iiot/AR-100/... 만 받는 것이 아니라 iiot 라는 토픽 그 자체로 발행된 메시지도 받습니다. # 를 "아래" 만 가리키는 기호로 읽으면 이 한 건을 계속 놓칩니다.

예외 2 — 와일드카드는 \$ 로 시작하는 토픽을 잡지 못합니다. 브로커는 자기 상태(접속 수, 처리량 따위)를 \$SYS/ 로 시작하는 시스템 토픽에 스스로 발행합니다. 규약은 # 나 + 로 시작하는 구독 필터가 \$ 로 시작하는 토픽에 매칭되지

않도록 못박고 있습니다. 그래서 # 를 구독해도 브로커의 내부 통계가 데이터에 섞여 들어오지 않습니다. 시스템 토픽을 보고 싶으면 \$SYS/# 처럼 \$ 를 직접 적어야 합니다.

그리고 반드시 기억할 규칙 하나. 와일드카드는 구독에만 쓸 수 있고 발행에는 쓸 수 없습니다. 보내는 쪽은 언제나 주소를 정확히 적어야 합니다. 한 번에 여러 곳에 보내는 것은 브로커의 일이지 발행자의 권한이 아닙니다.

토픽 설계는 나중에 바꾸기 어렵기 때문에 처음에 정해야 합니다. 원칙은 두 가지입니다. 첫째, 왼쪽에 서 오른쪽으로 좁아지게 짓습니다. 그래야 iiot/AR-100/# 처럼 중간에서 잘라 구독할 수 있습니다. 둘째, 토픽에는 주소를 넣고 값을 넣지 않습니다. iiot/AR-100/온도높음 같은 토픽은 판단을 주소에 새겨 넣은 것이라, 판단 기준이 바뀌면 주소 체계가 통째로 깨집니다.

구독 필터	대상 토픽	결과	이유
iiot/#	iiot/AR-100/reactor-line-01/TT-101	받습니다	# 는 그 아래 모든 단계
iiot/#	iiot	받습니다	# 는 상위 단계 자신도 매칭 (예외 1)
iiot/+/+/+	iiot/AR-100/reactor-line-01/TT-101	받습니다	정확히 4 단계이고 각 단계가 채워짐
iiot/+/+/TT-101	iiot/AR-100/reactor-line-01/TT-101	받습니다	사이트·설비는 무엇이든, 태그만 고정
iiot/AR-100/#	iiot/AR-100/reactor-line-01/TT-101	받습니다	이 사이트의 모든 하위
iiot/+/+	iiot/AR-100/reactor-line-01/TT-101	못 받습니다	+ 는 정확히 한 단계. 3 단계 토픽에만 맞음
iiot/AR-100/reactor-line-01/TT-10	iiot/AR-100/reactor-line-01/TT-101	못 받습니다	부분 문자열 매칭이 아님. 단계 단위 완전 일치
#	\$SYS/broker/clients/connected	못 받습니다	와일드카드는 \$ 로 시작하는 토픽을 잡지 않음 (예외 2)
\$SYS/#	\$SYS/broker/clients/connected	받습니다	\$ 를 직접 적었으므로 매칭
#/TT-101	—	쓸 수 없습니다	# 는 맨 끝에만 올 수 있음

3.4 QoS — 얼마나 확실하게 배달할 것인가

세 가지 배달 등급과 그 값어치

QoS(Quality of Service, 서비스 품질)는 "이 메시지를 얼마나 확실하게 배달할 것인가" 를 정하는 숫자입니다. 0, 1, 2 세 단계이고, 발행할 때 메시지마다 붙입니다. 이름은 거창하지만 실제로는 몇 번 왕복하며 확인 도장을 찍느냐의 차이입니다.

QoS 0 — 최대 1 회(at most once). PUBLISH 를 한 번 던지고 끝입니다. 확인이 없습니다. 네트워크가 삼켜 버리면 그대로 사라지고 아무도 모릅니다. 대신 가장 빠르고 가장 쌉니다. "다음 값이 1 초 뒤에 또 오니 한 건쯤 없어도 그만" 인 값에 씁니다.

QoS 1 — 최소 1 회(at least once). 발행자가 패킷 식별자를 붙인 PUBLISH 를 보내고 상대의 PUBACK (확인)을 기다립니다. 정해진 시간 안에 오지 않으면 같은 메시지를 같은 식별자로 다시 보냅니다. 이때 DUP (duplicate, 재전송) 플래그가 켜집니다. 그래서 같은 값이 두 번 도착할 수 있습니다. 첫 번째 전송이 잘 도착했는데 확인 도장만 길에서 사라진 경우가 그렇습니다. 보낸 쪽은 그 사실을 알 방법이 없으므로 다시 보냅니다.

QoS 2 — 정확히 1 회(exactly once). 네 번의 패킷이 오갑니다. PUBLISH → PUBREC (받았음) → PUBREL (그럼 처리하세요) → PUBCOMP (처리 끝). 두 번의 왕복입니다. 양쪽이 "이 패킷 식별자는 처리 중" 이라는 상태를 들고 있어야 하고, 그 상태는 재접속을 넘어 살아남아야 합니다. 중복도 유실도 없는 대신 왕복이 두 배, 브로커가 기억할 것도 두 배입니다.

여기서 반드시 알아야 할 세 가지가 있습니다.

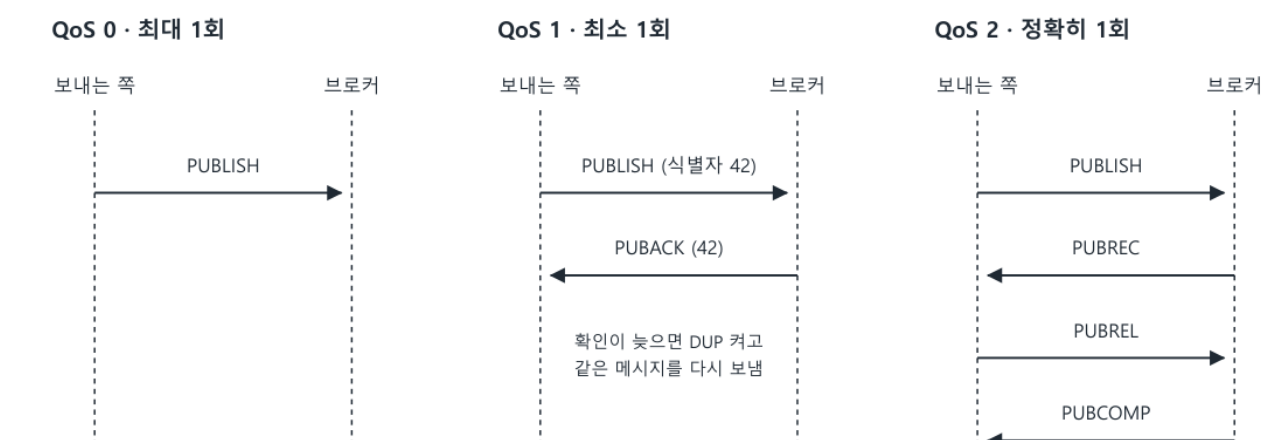
첫째, QoS 는 한 구간(홉)에만 적용됩니다. 발행자→브로커 구간의 QoS 와 브로커→구독자 구간의 QoS 는 별개입니다. 발행자가 QoS 2 로 보냈어도 구독자가 QoS 0 으로 구독했다면 구독자에게 가는 길 은 QoS 0 입니다. 즉 "끝에서 끝까지 정확히 한 번" 은 MQTT 가 보장해 주지 않습니다. 이것을 착각해 서 QoS 2 만 켜 두고 안심하는 경우가 매우 흔합니다.

둘째, 구독 시 지정하는 QoS 는 상한입니다. 브로커는 발행 QoS 와 구독 QoS 중 낮은 쪽으로 배달합니다. 올려 주지는 않습니다.

셋째, "유실 없음" 에는 단서가 붙습니다. QoS 1·2 가 유실을 막는다는 말은 한 구간 안에서, 보내는 쪽이 확인을 받을 때까지 재전송을 계속하는 동안에만 성립합니다. 브로커가 구독자에게 보내려고 쌓아 둔 큐에는 길이 상한이 있고, 넘치면 버립니다(3.5). 브로커 자신이 죽어 메모리의 큐가 날아갈 수도 있습니다. QoS 는 전송 구간의 재시도 규칙이지 전체 파이프라인의 무손실 보증서가 아닙니다.

그래서 산업 현장은 거의 언제나 QoS 1 을 씁니다. 이 교본의 바탕이 된 원문 자료도 같은 결론입니다. "네트워크 오버헤드가 과도한 QoS 2(정확히 1 회)보다는 고유 시퀀스 번호와 타임스탬프를 페이로드 에 포함한 QoS 1(최소 1 회 전달) 모드를 채택하고, 중복 수신 문제는 스트림 처리 계층의 멱등성 (Idempotency) 로직으로 해소하는 것이 표준적이다."

멱등성(Idempotency)이란 같은 일을 여러 번 해도 결과가 한 번 한 것과 같은 성질입니다. 메시지의 페이로드 안에 "이건 몇 번째 값" 이라는 표(시퀀스 번호)와 "언제 전 값" 이라는 표(타임스탬프)를 함께 담아 두면, 받는 쪽이 이미 본 표를 걸러 낼 수 있습니다. 중복 제거를 통신 규약에 맡기지 않고 뒷단으로 미루는 것 — 이것이 현장의 표준 해법입니다. 다만 이 해법에는 대가가 있고, 그 대가는 3.9 에서 실제 코드로 보겠습니다.



원문 도해 · QoS 0·1·2 의 한 구간 절차 (원문 87 쪽)

QoS	오가는 패킷	왕복	유실	중복	기억할 상태	쓰는 자리
0	PUBLISH	0	있음	없음	없음	초당 수십 건 나오는 참고용 값
1	PUBLISH → PUBACK	1	없음 (한 구간 안에서, 재전송이 계속되는 동안)	있음	패킷 식별자 1 개	산업 계측값의 사실상 표준
2	PUBLISH → PUBREC → PUBREL → PUBCOMP	2	없음 (같은 단서)	없음	식별자 + 진행 단계	과금·정산처럼 중복 이 곧 손해인 값

3.5 세션 · 밀림 · 보관 메시지 · 유언

영속 세션이 보장하는 것과 보장하지 않는 것

공장 무선망은 끊깁니다. 통신 간섭, 전원 잡음, 스위치 재부팅 같은 이유로 몇 초에서 몇 분씩 사라집니다. 이때 브로커가 "끊긴 동안 온 메시지" 를 어떻게 할지를 정하는 것이 세션(session)입니다.

세션은 브로커가 클라이언트 식별자(Client ID)별로 들고 있는 상태 묶음입니다. 내용은 셋입니다.

- 그 클라이언트의 구독 목록
- 아직 전달하지 못한 QoS 1·2 메시지(오프라인 큐)
- 진행 중이던 패킷 식별자들클라이언트가 접속할 때 이 상태를 버릴지 이어받을지 고릅니다. MQTT 3.1.1 에서는 CONNECT 의 Clean Session 플래그입니다. 켜면(true) 접속할 때마다 새 세션이라 끊긴 동안 온 것은 전부 버려지고, 끄면(false) 영속 세션(Persistent Session)이 됩니다. MQTT v5 에서는 이것이 Clean Start(시작 할 때 지울지)와 Session Expiry Interval(끊긴 뒤 몇 초까지 보관할지) 두 개로 쪼개졌습니다. 3.1.1 의 "무기한 아니면 즉시 삭제" 라는 거친 선택지가 초 단위 숫자로 바뀐 것입니다.

세 요소 중 무엇이 실제로 쓰이는지는 그 클라이언트가 무엇을 하는가에 달려 있습니다. 여기가 초보가 가장 많이 헛갈리는 자리입니다.

- 구독하는 클라이언트에게는 세 가지가 모두 의미가 있습니다. 재접속할 때 구독을 다시 신청하지 않아도 되고, 끊겨 있던 동안 온 QoS 1:2 메시지를 오프라인 큐에서 받습니다.
- 발행만 하는 클라이언트에게는 구독 목록도 비어 있고 받을 오프라인 큐도 없습니다. 남는 것은 진행 중이던 패킷 식별자 하나뿐입니다. 즉 QoS 1로 보내 놓고 PUBACK 을 못 받은 채 연결이 끊긴 메시지를, 재접속 뒤에 같은 식별자로 이어서 재전송할 수 있게 해 주는 것이 전부입니다. 작지만 무의미하지는 않습니다. 다만 "영속 세션을 켜오니 발행 쪽도 유실이 없다" 는 말은 성립하지 않습니다. 그 값은 애초에 브로커에 도착하지 못했고, 브로커의 세션은 도착하지 못한 값에 대해 아무것도 모릅니다.
- 영속 세션이 보장하지 않는 것을 마저 정리하면 이렇습니다.
- 무한히 보관하지 않습니다. 큐 길이 상한과 만료 시간이 있고, 넘치면 버립니다.
- QoS 0 은 보통 쌓아 주지 않습니다. 브로커 설정에서 끄고 켤 수 있는데, 대개 꺼 둡니다.
- 브로커 자신이 죽으면 메모리에 있던 큐는 사라질 수 있습니다.
- 클라이언트 식별자를 매번 바꾸면 세션은 아무 소용이 없습니다. 세션은 식별자로 찾아오기 때문에, 접속할 때마다 임의의 식별자를 만들어 쓰면 브로커는 매번 새 클라이언트로 봅니다.

그래서 더 근본적인 대책은 브로커가 아니라 에지 게이트웨이 쪽의 저장 후 전송(Store-and-Forward)입니다. 원문 자료의 표현대로 "광역 네트워크가 완전히 차단되면 브로커의 세션 버퍼는 곧 한계에 도달하므로, 현장 게이트웨이는 인입되는 센서 프레임을 로컬 SQLite, RocksDB, 또는 플래시 메모리 파일 시스템에 트랜잭션 단위로 선기록(Store) 해야" 하고, 연결이 재수립되면 "로컬 큐에 보관 된 이력 데이터를 타임스탬프 순서대로 순차 재전송(Forward)" 합니다. 브로커의 세션 영속은 짧은 끊김을 막고, 에지의 저장 후 전송은 긴 단절을 막습니다. 둘은 대체재가 아니라 두 겹의 안전장치입니다. AR-100 에서 이 두 겹이 각각 어디에 구현돼 있는지는 3.9 에서 파일과 값으로 확인합니다.

브로커가 구독자를 못 따라갈 때(밀림)

지금까지는 구독자가 끊긴 경우였습니다. 더 흔한 문제는 구독자가 살아는 있는데 느린 경우입니다. 초당 1 만 건이 들어오는데 구독자가 초당 3 천 건밖에 처리하지 못하면, 차이만큼 브로커 안에 쌓입니다. 이 현상을 밀림(backpressure, 역압)이라고 합니다.

브로커에는 이 밀림을 다루는 두 개의 관문이 있습니다.

(1) 인플라이트 한도(in-flight limit). 브로커가 한 구독자에게 확인(PUBACK/PUBREC)을 못 받은 채 동시에 내보내 둘 수 있는 QoS 1:2 메시지의 최대 개수입니다. 이 창이 꽉 차면 브로커는 그 구독자에게 더 내보내지 않고 기다립니다. 확인이 하나 돌아올 때마다 자리가 하나 납니다. 느린 구독자가 브로커를 끝없이 밀어붙이지 못하게 막는 밸브입니다.

(2) 메시지 큐(queue). 인플라이트 창이 찬 동안, 그리고 구독자가 끊긴 동안 밀린 메시지가 쌓이는 곳입니다. 여기에도 길이 상한이 있습니다.

그러면 무엇이 먼저 버려지는가. 순서가 있습니다.

· QoS 0 메시지가 가장 먼저 버려집니다. 큐에 QoS 0 을 담을지 여부가 설정으로 정해지는데, 담지 않도록 해 두면 인플라이트 창이 찬 순간 즉시 폐기됩니다. QoS 0 은 원래 "확인하지 않는다" 는 약속이므로 이것이 규약 위반은 아닙니다.

· QoS 1:2 는 큐에 쌓이고, 큐가 상한을 넘으면 가장 오래된 것부터 버려집니다. 가장 새 값을 살리는 선택입니다. 공정 계측에서는 대개 맞는 선택이지만, "빠짐없이 다 받아야 한다" 는 요구가 있다면 이 시점에 이미 그 요구는 깨져 있습니다.

그래서 브로커 설정에서 큐 길이와 인플라이트 한도를 보는 일은 "우리는 얼마만큼의 밀림까지 버티기로 했는가" 를 숫자로 선언하는 일입니다. 무한대라는 선택지는 없습니다. 무한대로 두면 버려지는 대신 브로커의 메모리가 터집니다.

보관 메시지(Retained)와 유언(Last Will)

보관 메시지(Retained message)는 브로커가 토픽마다 마지막 한 건을 붙들고 있다가, 새로 구독하는 클라이언트에게 즉시 한 번 건네주는 기능입니다. 발행할 때 RETAIN 플래그를 켜면 됩니다. 관제 화면 을 열자마자 빈칸이 아니라 마지막 값이 보이게 하고 싶을 때 씁니다. 지우려면 같은 토픽에 길이 0 짜리 빈 페이로드를 보관 플래그를 켜서 보냅니다.

함정은 그 값이 언제 것인지 알 수 없다는 점입니다. 설비가 세 시간 전에 죽었어도 화면에는 세 시간 전 값이 현재값처럼 떠 있습니다. 그래서 보관 메시지를 쓸 때는 페이로드에 타임스탬프를 반드시 넣고, 받는 쪽이 "너무 오래된 값" 을 구분하게 만들어야 합니다.

유언(Last Will and Testament, LWT)은 클라이언트가 접속할 때 CONNECT 에 실어 브로커에게 미리 맡겨 두는 메시지입니다. "내 연결이 비정상상으로 끊기면 이 토픽에 이 내용을 대신 발행해 주세요."

3.2 에서 본 규칙 그대로, 브로커가 keepalive 의 1.5 배 시간 동안 그 클라이언트에게서 아무 패킷도 받지 못해 연결이 끊어졌다고 판정하면 그 유언을 발행합니다. 그러면 관제 쪽이 "3 번 게이트웨이가 죽었다" 를 즉시 압니다.

여기서도 오해가 하나 있습니다. 유언은 클라이언트가 죽어서 나가는 것이 아니라 브로커가 연결이 끊겼다고 판정해서 나갑니다. 그리고 클라이언트가 DISCONNECT 패킷을 보내고 암전히 나가면 유언은 발행되지 않습니다. "정상 종료" 와 "사고" 를 구분하는 장치입니다.

3.6 MQTT v5 가 더한 것

3.1.1 로도 되는데 v5 가 왜 나왔나

MQTT 3.1.1(2014 년 표준화)은 지금도 가장 널리 쓰입니다. 규약이 작고 구현이 쉽기 때문입니다. 하지만 장비가 수만 대로 늘어나자 세 가지가 아쉬워졌습니다. 왜 실패했는지 알 수 없고(3.1.1 의 오류 응답은 사실상 "거절됨" 한 줄입니다), 메시지에 부가 정보를 붙일 자리가 없고(전부 페이로드 안에 옥여넣어야 합니다), 여러 소비자가 한 토픽을 나눠 받을 방법이 없습니다(구독자는 전부 같은 것을 받습니다).

MQTT v5(2019 년)는 이 셋을 포함해 여러 개를 채웠습니다. 중요한 점은 둘이 한 브로커에서 공존한다는 것입니다. v5 를 지원하는 브로커는 3.1.1 클라이언트도 그대로 받습니다. 어느 버전으로 붙을지는 클라이언트가 CONNECT 에서 정합니다. 그래서 현장에는 v5 브로커에 3.1.1 클라이언트가 붙어 있는 구성이 아주 흔합니다.

주제	MQTT 3.1.1	MQTT v5 가 더한 것
실패 사유	CONNACK 에 코드 몇 개	Reason Code — 거의 모든 응답 패킷에 사유가 실림
세션 보관	Clean Session 0/1 (무기한 또는 즉시 삭제)	Clean Start + Session Expiry Interval(초 단위)
메시지 수명	없음	Message Expiry Interval — 오래된 값은 배달 전에 폐기
토픽 반복	매번 전체 문자열 전송	Topic Alias — 긴 토픽을 숫자로 대신
부가 정보	페이로드 안에 직접	User Property — 카·값 쌍을 헤더에 첨부
요청-응답	규약 없음(관례로 처리)	Response Topic + Correlation Data — 답을 어디로 보낼지와 어느 요청에 대한 답인지를 규약이 실어 줌
부하 분산	구독자 전원이 같은 메시지 수신	공유 구독 \$share/그룹이름/토픽필터 — 같은 그룹 이름으로 구독한 여러 클라이언트가 한 메시지를 한 명만 받아 일을 나눔. 브로커가 그룹 안에서 돌아가며 배달합니다
흐름 제어	브로커 설정에만 의존	Receive Maximum — "나는 확인 안 한 QoS 1·2 를 동시에 이만큼까지만 받겠다" 를 접속할 때 상대방에게 통보. 3.5 의 인플라이트 한도를 클라이언트가 스스로 정하는 장치
구독 옵션	QoS 만 지정	No Local(내가 보낸 건 나에게 되돌려주지 말 것), Retain As Published(브로커가 전달할 때 원래 발행자가 켜 RETAIN 표시를 그대로 유지할지), Retain Handling(구독하는 순간 보관 메시지를 보내 줄지 — 항상 / 새 구독일 때만 / 아예 안 보냄 중 선택)
서버 주도종료	그냥 끊김	서버가 DISCONNECT + 사유를 보내고 끊음

3.7 헤더 2 바이트라는 말의 정확한 뜻

무엇이 2 바이트이고 무엇은 아닌가

MQTT 를 소개할 때 "고정 헤더가 2 바이트뿐인 경량 규약" 이라는 말이 항상 나옵니다. 정확히는 이렇습니다. 모든 MQTT 패킷은 고정 헤더(fixed header)로 시작하는데, 그 첫 1 바이트가 패킷 종류 4 비트 + 플래그 4 비트입니다. 그 뒤에 "남은 길이(Remaining Length)" 가 1~4 바이트 가변으로 붙습니다. 그래서 최소 2 바이트입니다.

여기서 흔히 과하게 일반화하는 대목을 바로잡아야 합니다. 그 플래그 4 비트 안에 DUP(재전송) 1 비트, QoS 2 비트, RETAIN 1 비트가 들어가는 것은 PUBLISH 패킷일 때뿐입니다. 다른 패킷 종류에서 그 4 비트는 예약값이라 규약이 정한 고정 비트를 그대로 넣어야 하고 다른 뜻이 없습니다 (PUBREL · SUBSCRIBE · UNSUBSCRIBE 는 0010 , 나머지는 0000). 즉 PINGREQ 나 PUBACK 에는 QoS 를 적을 자리가 없습니다. 적을 이유도 없습니다 — 배달 등급은 메시지의 성질이지 확인 응답의 성질이 아니기 때문입니다.

비교 대상은 HTTP 입니다. HTTP 헤더는 사람이 읽는 텍스트라 요청 한 건에 보통 수백 바이트가 붙고, 그 내용이 매 요청마다 거의 그대로 반복됩니다. MQTT 는 이것을 비트 단위 이진값으로 눌러 놓았습니다. 연결을 한 번 맺어 계속 쓰기 때문에 반복되는 정보를 다시 보낼 이유도 없습니다. 살아 있음을 알리는 PINGREQ / PINGRESP 도 각각 2 바이트입니다.

그런데 여기서 반드시 오해를 풀어야 합니다. 가벼운 것은 포장이지 내용물이 아닙니다. 3.2 에서 정의 한 대로 MQTT 는 페이로드를 들여다보지 않습니다. 그 안에 무엇이 들었든 바이트 덩어리로 취급합니다. 그러니 JSON 처럼 사람이 읽는 형식을 담으면 전체 크기는 페이로드가 지배합니다. AR-100 의 시뮬레이터가 내보내는 한 건(simulator/sim.py 165~173 행의 구조에, docs/journal/03-tier2-backbone.md § 3.5 의 실측 샘플 값을 넣은 것)을 바이트 단위로 세면 이렇습니다.

```
PUBLISH (QoS 1)  토픽: iiot/AR-100/reactor-line-01/FT-102
고정 헤더  1 바이트 패킷 종류(PUBLISH) + DUP / QoS / RETAIN 플래그
              2 바이트 남은 길이 = 180 (128 이상이라 가변 인코딩이 2 바이트)
가변 헤더  2 바이트 토픽 길이 (= 34)
              34 바이트 토픽 문자열
              2 바이트 패킷 식별자 (QoS 0 이면 이 2 바이트가 없음)
페이로드  142 바이트 {"ts": 1789546028493305900, "site": "AR-100",
                      "device": "reactor-line-01", "tag": "FT-102",
                      "value": 6.2861, "quality": "GOOD", "seq": 184213}

남은 길이 = 2 + 34 + 2 + 142 = 180 바이트
합계      = 1 + 2 + 180 = 183 바이트
이 중 프로토콜 몫 = 183 - 142 = 41 바이트 (22.4 %)
```

이 예시에서 토픽의 마지막 단계(FT-102)와 페이로드의 "tag" 값이 같다는 점을 눈여겨보십시오. 3.3 에서 토픽의 마지막 단계를 계측점으로 쓰기로 정했으므로, 같은 정보가 주소에도 내용물에도 들어 있는 것입니다. 이것은 낭비가 아니라 의도입니다. 토픽만 보고 걸러 낼 수도 있어야 하고(브로커가 하는 일), 페이로드만 보고도 어느 태그인지 알 수 있어야 하기 때문입니다(Kafka 로 넘어간 뒤 브로커가 없는 곳에서하는 일). 둘이 다르다면 그때부터는 어느 쪽을 믿어야 할지 알 수 없어집니다.

주의. 위 계산에서 보듯 진짜 아껴야 할 곳은 페이로드입니다. 대역폭이 정말 부족한 회선이라면 JSON 대신 이진 형식을 쓰거나 토픽을 짧게 짓는 편이 헤더를 줄이는 것보다 훨씬 효과가 큼니다. 대표적인 이진 형식이 Protocol Buffers(구글이 만든 직렬화 형식, 흔히 protobuf)입니다. JSON 이 "value": 6.2861 처럼 항목 이름을 글자로 매번 씬 데 반해, protobuf 는 .proto 라는 구조 정의 파일에 "1 번 항목은 value, 실수형" 이라고 미리 약속해 두고 전송할 때는 번호와 이진 값만 씁니다. 그래서 훨씬 작고 빠릅니다. 대가는 사람이 그냥 열어볼 수 없다는 것과 보내는 쪽과 받는 쪽이 같은 .proto 를 갖고 있어야 한다는 것입니다. MQTT 의 경량성은 연결을 유지한 채 자잘한 메시지를 아주 많이 주고받을 때 값어치가 나옵니다. 큰 파일 하나를 보내는 일에는 아무 이점이 없습니다.

3.9 AR-100

AR-100 은 EMQX 5.8.6 OSS 를 사용합니다(.env 의 EMQX_VERSION=5.8.6). 설정 파일 emqx/emqx.conf 는 컨테이너 /opt/emqx/etc/emqx.conf 에 읽기 전용으로 연결됩니다. 호스트 MQTT 포트는 27083(내부 1883), 관리 화면은 27183(내부 18083)입니다. 다른 개발 서비스와 충돌을 피하려고 호스트 포트를 27000 번대로 모았습니다.

설정	값	이게 무슨 뜻인가
max_inflight	128	3.5의 인플라이트 한도. 브로커가 한 클라이언트에게 PUBACK(또는 PUBREC)을 아직 못 받은 채 동시에 내보내 둘 수 있는 QoS 1:2 메시지의 개수입니다. 확인을 하나 받을 때마다 자리가 하나 납니다. 1로 두면 한 건 보내고 확인을 받아야 다음 건을 보내므로 왕복 시간만큼 처리량이 묶이고, 크게 두면 느린 구독자에게도 밀어 넣다가 큐가 터집니다. 128은 그 사이의 일반적인 값입니다
max_mqueue_len	100000	메시지 큐 길이 상한. 구독자가 끊겨 있거나 인플라이트 창이 찼을 때 밀린 메시지를 최대 10만 건까지 쌓아 둡니다. 넘치면 오래된 것부터 버립니다. "10만 건까지의 밀린 버티기로 한다"는 선언입니다
mqueue_store_qos0	false	QoS 0 메시지는 그 큐에 담지 않습니다. 지금 못 보내면 그대로 버립니다. 3.5에서 본 "무엇이 먼저 버려지는가"의 답이 이 한 줄입니다
max_packet_size	1MB	패킷 한 건의 최대 크기. 이보다 큰 것을 보내려 하면 브로커가 거부합니다. 3.7에서 전 한 건이 183바이트였으니 약 5,700배의 여유입니다. 센서 값에는 과분하고, 이미지나 파일을 MQTT로 밀어 넣는 것을 막는 상한 역할을 합니다
session_expiry_interval	2h	끊긴 클라이언트의 세션을 2시간 보관합니다. docs/journal/10-config-reference.md에 적힌 그대로 "2시간은 데모용이며, 실제로는 에지 게이트웨이의 최대 예상 단절 시간으로 잡습니다."
max_qos_allowed	2	QoS 2까지 허용. 허용일 뿐이고 실제로 쓰는 곳은 없습니다(아래 참조)
retain_available	true	보관 메시지 기능을 켜 둡니다. 역시 쓰는 곳은 없습니다
max_connections	100000	이 리스너에 동시에 붙을 수 있는 클라이언트 수. 이 예제에는 한참 과분한 값이고, 실제 공장 규모(에지 게이트웨이 수천 대)를 가정한 설정입니다
listeners.quic.default	enable = false	QUIC는 UDP 위에 만든 비교적 새로운 전송 규약으로, TCP+TLS가 쓰는 연결 수립 왕복을 줄이고 단말의 네트워크가 바뀌어도(와이파이→LTE) 연결이 이어집니다. EMQX는 MQTT를 QUIC 위에 얹는 것을 지원하지 않지만 이 프로젝트는 끔니다
추가 설정		원문 값·의미
node.name / cluster.discovery_strategy		emqx@127.0.0.1 / manual. 실제 가입 노드는 하나인 교육용 구성
cookie / data_dir		iiot-scada-secret / /opt/emqx/data. 데모 값과 데이터 저장 위치
TCP / WebSocket		0.0.0.0:1883 / 0.0.0.0:8083
SSL / WSS / QUIC		비활성. TLS 암호화가 적용되지 않음
authorization		no_match=allow, deny_action=ignore. ACL 규칙 파일 없음

주의 — 원문 자료가 전제한 Tier 2와 원문의 저장소의 Tier 2는 두 군데가 다릅니다. 원문 자료의 최종 권장 아키텍처는 "엔터프라이즈 수집 및 백본 계층(Tier 2)에서는 클러스터링된 EMQX 브로커가 수천 대의 에지 게이트웨이로부터 암호화된(TLS) MQTT 스트림을 중단 수신한다"라고 적혀 있습니다. 위 파일은 둘 다 아닙니다.

(1) 클러스터가 아닙니다. node.name = "emqx@127.0.0.1"은 이 노드가 자기 자신만 아는 주소로 떠 있다는 뜻이고, cluster { discovery_strategy = manual }은 다른 노드를 자동으로 찾지 않는다는 뜻입니다(불이려면 사람이 명령으로 합류시켜야 합니다). 실제로 합류시킨 노드도 없으므로 단일 노드입니다. 그래서 이 브로커가 죽으면 수집이 멈춥니다. 클러스터로 묶었다면 구독 정보가 노드 사이에 공유되어 한 대가 빠져도 나머지가 받아 냈을 것입니다.

(2) 암호화가 꺼져 있습니다. listeners.ssl.default { enable = false } — TLS 리스너(8883)를 끄고 평문 1883만 엽니다. 공장 밖 회선을 타는 구성이라면 그대로 둘 수 없는 상태입니다.

이 차이는 실수가 아니라 예제의 범위입니다. 한 대의 개발 PC에서 docker compose up 한 번으로 전 구간을 보이는 것이 목표이므로, 노드를 여러 대 띄우거나 인증서를 발급하는 절차를 넣지 않았습니다. 다만 "PDF 대로 구현했다"고 말할 수 없는 지점이므로 여기에 적어 둡니다. 3.10에서 보게 될 Kafka Sink 문제와 성격이 같습니다 — 문서가 전제한 구성과 실제로 서 있는 구성이 다를 때는 그 차이를 덮지 말고 드러내야 합니다.

AR-100에서는, 토픽 체계가 두 갈래입니다. EdgeX를 거치지 않는 lite 경로에서는 시뮬레이터가 태그마다 한 건씩 `iiot/AR-100/reactor-line-01/TT-101` 형태로 발행합니다(`simulator/sim.py` 172행, 접두사 `iiot`는 `docker-compose.yml`의 `MQTT_TOPIC_PREFIX`). 브리지는 이것을 `iiot/+/+`로 한 번에 받습니다 — 사이트·설비·태그를 각각

한 단계씩 비운 필터입니다. EdgeX 경로에서는 반대로 단일 토픽 `edgex/telemetry` 하나만 씁니다(`docker-compose.edgex.yml` 의 `...MQTTEXPORT_PARAMETERS_TOPIC`). EdgeX 가 12 개 계측값을 이벤트 하나의 `readings[]` 배열로 묶어 내보내기 때문에 태그별로 토픽을 쪼갤 수가 없습니다. 같은 공정인데 상류 구성이 달라지면 토픽 설계가 통째로 달라진다는 것을 원문의 저장소가 두 경로로 실제로 보여 줍니다.

AR-100에서는, QoS 가 전 구간 1 입니다. 브로커는 `max_qos_allowed = 2` 로 2 까지 허용하지만 아무도 2 를 쓰지 않습니다. 시뮬레이터 발행: `self.mqtt.publish(f"{MQTT_TOPIC_PREFIX}/{site}/{dev}/{tag}", payload, qos=1)` (`sim.py` 172 행) EdgeX 반출: `WRITABLE_PIPELINE_FUNCTIONS_MQTTEXPORT_PARAMETERS_QOS: "1"` Telegraf 브리지 수신: `qos = 1` (`bridge-lite.conf` , `bridge-edgex.conf` 모두) 알람 역방향 재발행: `qos = 1` (`alert-republish.conf`) FUXA 구독: `{ "topic": "scada/alerts/#", "qos": 1, "type": "json" }` (`fuxa/project.json`)

그러면 순서가 뒤바뀌는 일이 실제로 있는가. MQTT 의 순서 보장 범위를 정확히 알아야 합니다. 규약이 보장하는 것은 같은 클라이언트가 같은 연결 위에서 같은 토픽으로 같은 QoS 로 보낸 메시지들 사이의 순서뿐입니다. 이 울타리를 벗어나면 보장이 없습니다. 서로 다른 토픽 사이의 순서는 보장되지 않고, 연결이 끊겼다 붙으면 그 경계에서 어긋날 수 있으며, 재전송이 섞이면(먼저 보낸 것을 다시 보내는 동안 다음 것이 이미 나간 경우) 받는 쪽이 보는 순서가 보낸 순서와 달라질 수 있습니다. AR-100 처럼 태그마다 토픽이 다른 구성에서는 애초에 태그 사이의 순서를 MQTT 가 보장 하지 않으며, 그래서 파티션 키를 태그로 잡아 태그 안에서만 순서를 지키는 3.10 의 설계가 따릅니다.

AR-100에서는, 세션은 양쪽 모두 영속입니다. 다만 양쪽이 얻는 것이 다릅니다. 발행 쪽 — `simulator/sim.py` 104 행의 `clean_session=False`, # 세션 영속 (PDF p.4 권고) , `keepalive` 는 106 행에서 30 초입니다. 그런데 이 클라이언트는 구독을 하나도 하지 않습니다. 3.5 에서 정리한 대로, 구독 목록이 비어 있고 받을 오프라인 큐도 없으므로 영속 세션으로 남는 것은 진행 중이던 패킷 식별자뿐입니다. 즉 QoS 1 로 던져 놓고 PUBACK 을 못 받은 채 끊긴 메시지를 재접속 뒤 같은 식별자로 이어져 재전송할 수 있다는 것이 이 설정의 실제 효과입니다. 이것은 "에지가 끊겨도 값이 안 없어진다" 는 뜻이 아닙니다. 그 보장은 아래의 저장 후 전송이 담당합니다. 구독 쪽 — Telegraf 의 `persistent_session = true` 이고 클라이언트 식별자를 `telegraf-bridge-lite` / `telegraf-bridge-edgex` 로 고정해 두었습니다. 여기서는 세 요소가 모두 살아 있습니다. 브리지가 잠시 죽어도 구독이 남아 있고, 그동안 도착한 QoS 1 메시지가 오프라인 큐에 쌓였다가 재접속 때 내려옵니다. 식별자를 고정한 것이 핵심입니다 — 붙을 때마다 임의의 식별자를 쓰면 브로커는 매번 새 클라이언트로 보고 쌓아 둔 것을 주지 않습니다.

브로커 쪽 보관 기간은 `session_expiry_interval = 2h` 입니다.

AR-100에서는, 보관 메시지와 유언은 쓰지 않습니다. `emqx.conf` 에 `retain_available = true` 로 기능은 켜 두었지만, 발행하는 코드 어디에도 RETAIN 플래그가 없고 유언(Will)을 등록하는 곳도 없습니다. 그래서 MQTT 로만 값을 받는 구독자가 새로 붙으면 다음 발행이 올 때까지 아무것도 받지 못합니다. 1 초에 한 번씩 값이 오는 공정이라 체감되지 않을 뿐이고, 발행 주기가 분 단위인 설비였다면 보관 메시지가 필요했을 것입니다.

실측 메모리는 EMQX 269 MiB, Mosquitto 1.3 MiB 입니다(`docs/journal/09-metrics.md` §9.2). 이 두 숫자는 말은 부하가 달라서 성능 비교가 아니라 규모감으로만 읽어야 합니다.

AR-100에서는, 실측 유입량은 초당 12 건(12 태그 × 1 Hz)입니다. Kafka 오프셋 차분으로 20 초를 재 서 252 건이 나왔습니다. 이 규모에서 EMQX 의 CPU 는 0.3%입니다. 3.5 에서 본 밀림은 이 부하에서 전혀 일어나지 않습니다 — 인플라이트 128 과 큐 10 만 건은 초당 12 건 앞에서 손도 대지 않은 여유 입니다. 브로커는 이 구간에서 병목이 아닙니다.

주의 — 인증·인가는 데모 수준입니다. `emqx.conf` 의 `authorization { no_match = allow, deny_action = ignore }` 는 규칙에 걸리지 않는 요청은 전부 허용한다는 뜻입니다. 그리고 그 파일 에 규칙이 한 줄도 없습니다. 여기서 말하는 규칙을 ACL(Access Control List, 접근 제어 목록)이라고 부릅니다 — "이 클라이언트는 이 토픽에 발행할 수 있다 / 이 토픽을 구독할 수 있다" 를 한 줄씩 적어 둔 허용·거부 목록입니다. 통합 아키텍처 문서의 표에는 EMQX 설정 수단이 "QoS1 / 세션 영속 / ACL" 이라고 적혀 있지만 현재 파일은 거기까지 가 있지 않고, ACL 파일 자체가 없습니다 (`emqx/` 안에는 `emqx.conf` 하나뿐입니다). 앞서 본 TLS 미적용과 합치면, 이 브로커는 네트워크에 닿을 수 있는 누구에게나 어떤 토픽이든 열려 있습니다. 실제 공장에 놓을 때는 TLS(가능하면 mTLS)를 켜고 클라이언트별로 발행·구독 가능한 토픽을 좁혀야 합니다. 브로커는 기본값으로는 열려 있는 장치입니다.

3.10 대안과 비교

같은 자리의 다른 도구들 Eclipse Mosquitto — C 로 쓰인 단일 프로세스 브로커. 몇 MB 메모리로 뜨고 설정 파일도 짧습니다. 클러스터링이 없어 한 대가 감당할 수 있는 만큼이 한계입니다. 에지 게이트웨이 안이나 한 장비 안의 버스로는 최적입니다. 원문의 저장소도 EdgeX 내부 버스로 쓰고 있습니다. HiveMQ Community Edition — Java 기반. 확장

API(extension)가 잘 정리돼 있습니다. 클러스터 같은 운영 기능은 상용 판에 있습니다. NanoMQ — 에지 전용 초경량 브로커. 자원이 아주 적은 게이트웨이 보드에 올립니다. VerneMQ — EMQX와 마찬가지로 Erlang 계열. 클러스터링 중심. RabbitMQ + MQTT 플러그인 — 본체는 AMQP(Advanced Message Queuing Protocol, 고급 메시지 큐 규약) 브로커입니다. AMQP는 MQTT와 달리 큐·익스체인지·라우팅 키 같은 개념을 규약 자체가 갖고 있고, 메시지를 큐에 쌓아 두었다가 작업자가 꺼내 가는 업무 처리에 맞춰 설계됐습니다. 여기에 MQTT를 곁들이는 구성이라, 이미 RabbitMQ를 쓰는 조직이 선택합니다. 대신 MQTT 고유 기능의 대응이 브로커 전용 제품보다 알을 수 있습니다. 브로커를 빼고 Kafka로 직결 — 센서가 Kafka에 직접 붙는 방식. 원문 자료는 이 선택지를 조건부로만 인정합니다(3.8 인용 참조): 센서 스펙이 균일하고 토픽이 이미 표준화돼 있고 단방향 수집만 하면 유리하다고 봅니다. 반대로 제어를 내려보내야 하면 안 된다고 못박습니다. HTTP POST / REST — 가장 익숙하고 가장 비쌉니다. 원문 자료의 계층표에서도 IoT 수집 미들웨어의 통신 방식으로 MQTT v5.0, CoAP, REST가 나란히 적혀 있습니다. 수집 주기가 분 단위로 느리고 장비가 적으면 여전히 합리적입니다. CoAP — UDP(User Datagram Protocol) 위에서 도는 초경량 요청/응답 규약입니다. UDP는 TCP와 달리 연결을 맺지 않고 도착 확인도 하지 않는 전송 규약이라, 패킷 하나를 던지면 그것으로 끝입니다(유실·순서 뒤바뀔 위험이 있어서 해야 합니다). 그래서 연결 유지 비용이 없고 배터리 장비에 맞습니다. 다만 발행/구독이 아니라 요청/응답 계열이라는 점이 MQTT와 본질적으로 다릅니다. OPC-UA PubSub — 산업 표준 쪽에서 나온 발행/구독. 데이터 모델(타입·단위·구조)을 규약이 직접 갖고 있다는 점이 MQTT와 근본적으로 다릅니다. MQTT는 페이로드를 모르지만 OPC-UA는 압니다. ThingsBoard·EdgeX Foundry — 이들은 브로커의 대안이 아니라 다른 층입니다. 자체 MQTT 수신 지점을 갖고 있어 브로커처럼 보이지만, 3.8에서 정리한 장비 모델과 제어 경로를 갖는 미들웨어입니다.

EMQX 5.8 OSS 대 Mosquitto

항목	EMQX 5.8 OSS	Eclipse Mosquitto
구현 기반	Erlang/OTP — 경량 프로세스 수십만 개를 다루도록 만들어진 런타임	C 단일 프로세스
여러 대로 묶기	클러스터 — 여러 노드가 구독 정보를 공유해 한 브로커처럼 동작 (cluster { discovery_strategy = ... })	없음. 브리지로 메시지를 넘겨줄 수는 있으나 한 브로커처럼 보이지는 않음
규칙 엔진	있음 — SQL 문법으로 메시지를 거르고 다른 곳으로 보냄	없음
관리 화면	내장 대시보드(18083). 접속 목록·구독·처리량을 눈으로 확인	없음. 로그와 명령줄 도구
지표 노출	Prometheus 형식 내장 (/api/v5/prometheus/stats)	별도 익스포터 필요
MQTT v5	지원	지원
설정 형식	HOCON — Human-Optimized Config Object Notation. JSON을 사람이 쓰기 편하게 고친 설정 형식으로, 중괄호로 계층을 만들고 따옴표·침표를 생략할 수 있으며 주석을 쓸 수 있습니다. 3.9의 mqtt { ... } 가 그 모양입니다	키 값 한 줄씩. 계층이 없어 단순하지만 항목이 많아지면 관계가 안 보입니다
실측 메모리 (원문의 저장소)	269 MiB	1.3 MiB (EdgeX 내부 버스 역할)
어울리는 자리	공장·전사 단위 수집 지점. 장비 수천~수만 대	게이트웨이 한 대 안, 또는 장비 내부 버스

정리하면 고르는 기준은 성능이 아니라 운영 형태입니다. 브로커 한 대가 죽어도 서비스가 이어져야 하고, 지금 몇 대가 붙어 있는지 화면으로 봐야 하고, 메시지를 브로커 안에서 걸러 다른 시스템으로 보내야 한다면 EMQX 쪽입니다. 게이트웨이 한 대 안에서 프로세스끼리 값을 주고받는 용도라면 Mosquitto가 더 맞습니다. 원문의 저장소가 둘 다 쓰고 있는 것이 그 답입니다.

EMQX 규칙 엔진 — 브로커 안에서 거르기

브로커가 메시지를 읽고 판단하게 만드는 장치

앞에서 브로커는 페이로드를 모른다고 했습니다. 규칙 엔진(Rule Engine)은 그 원칙에 구멍을 내는 기능입니다. 지나가는 메시지를 SQL 비슷한 문장으로 골라내고, 걸린 것에 동작을 붙입니다.

SELECT payload.value as v, topic FROM "iiot/#" WHERE payload.value > 6.0FROM 에 토픽 필터를 쓰고, WHERE 로 페이로드 안의 값을 조건에 겁니다. 걸린 메시지에는 다른 토픽 으로 다시 발행하거나, 외부 HTTP 서버를 부르거나, 데이터 저장소로 흘려보내는 동작(싱크, sink)을 연결할 수 있습니다. 매력적인 이유는 분명합니다 — 중간에 프로그램을 하나도 두지 않고 브로커 설정만으로 데이터를 다음 시스템까지 보낼 수 있기 때문입니다.

이 교본의 원문 자료도 바로 그 점을 근거로 삼습니다. "EMQX와 같은 고성능 분산 MQTT 브로커는 내부 규칙 엔진(Rule Engine)과 내장 Kafka 프로듀서 커넥터를 제공하여, 중간 변환 오버헤드 없이 수십 만 TPS의 데이터를 Kafka 토픽으로 즉각 분배할 수 있다." 여기서 TPS는 Transactions Per Second, 초당 처리 건수입니다. 수십만 TPS면 초당 수십만 건입니다(참고로 AR-100의 실측 유입은 초당 12건입니다). 최종 권고안에서도 "EMQX의 데이터 통합 브리지를 통해 수신된 센서 텔레메트리는 지체 없이 Apache Kafka 클러스터의 sensor.telemetry.raw 토픽으로 적재된다"고 못박습니다.

실사 결과 — Kafka Sink는 Enterprise 전용입니다

AR-100에서는, 구현에 들어가기 전 이 전제를 확인했고, 성립하지 않았습니다. EMQX 문서에서 Kafka 연동 페이지는 전부 상용 판 경로에 있습니다.

<https://docs.emqx.com/en/enterprise/v5.1/data-integration/data-bridge-kafka.html>

오픈소스 EMQX 5.x에는 Kafka 프로듀서 커넥터 자체가 없습니다. 규칙 엔진은 있지만 동작(싱크)으로 Kafka를 고를 수 없고, 대시보드의 데이터 통합 메뉴에도 나타나지 않습니다. 원문 자료가 "EMQX가 하면 된다"고 적은 그 자리가, 오픈소스만으로는 비어 있는 자리였습니다 (docs/journal/08-decisions.md ADR-001).

주의. 이것이 이 프로젝트 전체에서 가장 값비싼 교훈입니다. "X가 Y를 지원한다"는 서술을 만나면 그것이 오픈소스 판(OSS)인지 상용 판(Enterprise)인지를 반드시 확인해야 합니다. IIoT 스택에는 핵심은 무료로 열어 두고 연동 기능만 유료로 파는 오픈코어(open core) 모델이 특히 많습니다. 이 한 가지를 모르고 구현에 들어갔다면 수집 계층을 통째로 다시 만들어야 했습니다.

빈자리를 무엇으로 채울지가 이 계층의 핵심 결정이었습니다. 후보는 넷이었습니다.

대안	장점	단점	판정
EMQX Enterprise	원문 권고 그대로 구현 가능	상용 라이선스, 체험판 제약	기각 — "오픈소스 예제" 목표와 정면 충돌
Kafka Connect + MQTT 소스 커넥터	Kafka 생태계 표준 방식	Confluent 판은 독점 라이선스. 대안 커넥터는 설정 복잡도가 더 높음	보류
Telegraf	TOML 설정만, 공식 이미지, Apache 2.0	컴포넌트가 하나 늘어남	채택
자체 브리지 코드 작성	완전한 제어	코드가 늘어 "설정만으로 동작" 목표 위반	기각

AR-100에서는, Telegraf가 MQTT→Kafka 브리지를 말합니다. 입력은 mqtt_consumer, 출력은 kafka입니다. 채택 근거 중 하나인 TOML은 Tom's Obvious Minimal Language의 약자로, [섹션]과 키=값으로만 이루어진 설정 파일 형식입니다. 윈도우의 .ini 파일과 비슷하지만 문자열·숫자·참거짓·배열을 구분해 적을 수 있습니다. "코드를 짜지 않고 설정만으로 동작한다"는 이 프로젝트의 목표에 맞는 형식이었습니다. 결정적 이유는 새 기술을 들이는 것이 아니라는 점이었습니다 — 원문 자료가 이미 Tier 4(저장 계층)에서 "Telegraf 컨슈머"를 언급하고 있으므로 스택 일관성이 깨지지 않습니다.

입력은 inputs.mqtt_consumer이며 servers=["tcp://emqx:1883"], topics=["iiot/+/+"], qos=1, client_id="telegraf-bridge-lite", persistent_session=true, data_format="json_v2"입니다. 출력은 outputs.kafka이며 brokers=["kafka:9092"], topic="sensor.telemetry.raw", routing_tag="tag", compression_codec=3, required_acks=-1, max_retry=10, data_format="json", json_timestamp_units="1ns"입니다. json_transformation은 2장에 정리한 여섯 필드 계약으로 바꿉니다. 아래에서 Kafka 출력 설정의 뜻과 파서 함정을 확인합니다.

이 파일에는 다음 장(Kafka)의 용어가 미리 섞여 있습니다. 여기서 뜻만 풀어 두겠습니다.

출력 설정 읽기: routing_tag는 같은 태그의 메시지를 같은 Kafka 파티션으로 보냅니다. required_acks=-1은 동기화된 복제본의 기록 확인을 기다리지만, 이 예제는 복제 개수 1이라 실제 확인 대상도 한 대입니다. JSONata는 입력 JSON에서 어느 값을 꺼내 어떤 결과 구조로 내보낼지 적는 식 언어입니다. 파티션·복제·오프셋의 전체 원리는 2권 4장에 모았습니다.

치른 대가도 정직하게 적어 둡니다. 컨테이너가 하나 늘었고, Telegraf의 json_v2 파서에 함정이 둘 있었습니다. json_v2는 Telegraf가 JSON을 읽어 들이는 두 번째 세대 파서로, 중첩 JSON과 평면 JSON에 서로 다른 문법을 씁니다.

함정 1 — object.fields 는 하위 테이블이 아니라 맵입니다. TOML 에서 [a.b.c] 처럼 대괄호로 한 단계 더 들어가는 것을 하위 테이블이라 하고, 키 = 값을 여러 줄 나열한 것을 맵이라 합니다. 여기서 는 항목마다 [...fields.detail] 로 파고들면 cannot unmarshal TOML table into string 오류가 나고, value = "float" / detail = "string" 처럼 한 블록 안에 나란히 적어야 합니다. 함정 2 — 평면 JSON 에는 object 파서를 쓸 수 없습니다. 평면(flat) JSON 이란 {"ts":..., "tag":..., "value":...} 처럼 중첩이 없는, 한 단계짜리 객체를 말합니다. object 파서는 path 로 "이 배열 안을 훑어라" 를 지정하는 방식이라 루트 자체를 가리킬 수가 없습니다(path = "" 는 the GJSON path is required 로 거부됩니다). 평면 JSON 은 tag / field 항목으로 하나씩 기술해야 합니다. 그래서 원문의 저장소 는 EdgeX 경로(중첩, readings[])와 lite 경로(평면)에서 서로 다른 파서 문법을 씁니다 — 위 발체의 bridge-lite.conf 가 tag / field 방식입니다.

그리고 이 선택이 모든 규모에서 옳다는 보장은 없습니다. 원문의 저장소의 실측 부하는 초당 12 건이고 그 구간 에서 지연은 관측되지 않았습니다. 원문 자료가 말한 "수십만 TPS" 환경에서 Telegraf 한 프로세스가 Kafka Connect 의 분산 워커 모델만큼 버틸지는 이 프로젝트가 확인하지 않은 범위입니다. ADR-001 도 이 것을 "토론거리" 로 남겨 두었습니다.

주의 — 원문 자료가 IoT 미들웨어 존치를 결론 1 로 둔 이유는 "Kafka 클라이언트가 무겁다" 가 아닙니다. 결론 1 의 문장은 이렇습니다. "IoT 미들웨어의 존치: 센서 네트워크와 Kafka 사이에 EdgeX Foundry 나 고성능 브로커(EMQX)를 반드시 유지하여 프로토콜 정규화와 에지 Store-and-Forward 유실 방지 체계를 수립해야 한다." 근거로 적힌 것은 프로토콜 정규화와 에지 저장 후 전송 두 가지 이고, 클라이언트의 무게나 연결 관리의 어려움은 원문에 없습니다. 그럴듯한 다른 이유를 대신 끼워 넣으면 그 순간 원문이 실제로 무엇을 근거로 삼았는지가 사라집니다. 인용할 때는 근거까지 인용해야 합니다.

이 권에서 통합한 반복 설명

반복 주제	편집본의 정본 위치
계측 태그·정상 운전점	1 장 표를 정본으로 유지. 2 장·8 장 등 재등장 표는 중복 설명 축소
제어점·주소·타입	2 장에 전체 주소 표와 0/1-based·읽기 전용의 실제 경계 유지
세션·저장 후 전송	2 장의 에지 저장과 3 장의 MQTT 세션으로 책임 분리
공통 계약·브리지	2 장에 실제 계약, 3 장에 브리지 채택과 설정 함정

분량 감소는 반복 문장·표·설정의 통합과 연속 페이지 배치에서 만들었습니다. 원문에 없던 기능을 보충하거나, 서로 다른 측정을 임의로 하나의 정답으로 맞추지는 않았습니다. 원문 전체와 이 편집본은 설치·운영 시 함께 확인할 수 있는 별개의 자료입니다.